

Università di Pisa

Dipartimento di Informatica

Corso di Laurea Magistrale in Computer Science

A SSI-based Governance Model for Decentralized Autonomous Organizations

Candidato:

Mattia Prestifilippo Colombrino

Relatori:

Prof.ssa Laura Ricci

Dott. Calogero Turco

Anno Accademico 2025–2026

Contents

1 Introduction **5**

2 Background **11**

2.1 Blockchain and Smart Contracts 11

2.1.1 Definition of Blockchain 11

2.1.2 Consensus: Proof of Work and Proof of Stake 13

2.1.3 Ethereum: Ethereum Blockchain, Smart Contracts, Transactions, ERC-20 Tokens and Gas 14

2.1.4 Ethereum Storage Model 17

2.1.5 Solidity 18

2.1.6 ERC-20 Fungible Tokens 19

2.2 Decentralized Autonomous Organizations 21

2.2.1 Characteristics of a DAO 22

2.2.2 Governance Mechanisms 23

2.2.3 Timelock Mechanism 24

2.2.4 Snapshot 24

2.3 Cryptographic Concepts 25

2.3.1 Hash Functions 25

2.3.2 Asymmetric Cryptography 26

2.3.3 Digital Signatures 26

2.4 Self-Sovereign Identity 27

2.4.1 Definition and Principles 28

2.4.2 SSI Actors 29

2.4.3 Verifiable Credentials 29

2.4.4 Structure of a Verifiable Credential 31
2.4.5 Verifiable Presentation and Selective Disclosure 32
2.4.6 Decentralized Identifiers 33

3 Related Work **36**

4 Design of the Governance Model **42**
4.1 Problem 42
4.2 Design of Expertise-Based Voting Power 43
4.2.1 Issues in Using Tokens to Represent Composite Voting Power 43
4.2.2 Design of Topic-Dependent Voting Power 44
4.2.3 Formula for Calculating a Member's Voting Power 45
4.2.4 Representation of Member Skills 46
4.3 SAGE Actors and Components 47
4.3.1 SAGE Actors 47
4.3.2 SAGE Components 48
4.4 SAGE Protocol Architecture 48
4.4.1 DAO Membership Protocol 50
4.4.2 Stake Protocol 50
4.4.3 Competence Certification Protocol 51
4.4.4 Governance Protocol 52
4.4.5 Investment Execution Protocol 54

5 DAO Governance and SSI Management Tools **56**
5.1 OpenZeppelin Contracts and Governance 56
5.1.1 Governor 58
5.1.2 ERC20Votes and GovernorVotes 59
5.1.3 Governor Configuration and Extensions 61
5.1.4 TimelockController and GovernorTimelockControl 62
5.2 Veramo 63
5.3 Hardhat 64

6 Implementation **65**
6.1 Implementation Versions 65
6.2 Project Structure 66
6.3 DAO Membership Protocol 69
6.4 Stake Protocol 70
6.5 Competence Certification Protocol 73
6.5.1 VC Issuance and On-Chain Cryptographic Verification . . 83
6.6 Governance Protocol 95
6.7 Investment Execution Protocol 101

7 Experimental evaluations **104**
7.1 Gas Measurement 104
7.1.1 Experimental Setup 105
7.1.2 Gas Measurement of V0 106
7.1.3 Gas Measurement of V1 108
7.1.4 Gas Measurement of V2 109
7.1.5 Gas Measurement of V3 110
7.1.6 Bitmap and No-Bitmap Comparison 113
7.2 Analytical Evaluation of Voting Power Behavior 116
7.2.1 Influence of Stake and Competence Weights 117
7.2.2 Whale and expert minority 119
7.2.3 Sensitivity to the Proposal Topic 120

8 Security Analysis **123**
8.1 Actors and Security Assumptions 124
8.2 Deployer Privileges and Risks During Bootstrap 124
8.3 Protection of Stake-Based Voting Power 125
8.4 Security of Historical Voting Power After a Snapshot 126
8.5 DID Binding and Holder Authentication 127
8.6 Addition and Removal of Trusted Issuers 128
8.7 VC Integrity 128
8.8 Security of Topic-Aware Governance 129
8.9 Treasury Security and Reentrancy Protection 129

8.10 Residual Risks and Limitations 130

9 Conclusions and Future Work **133**

9.1 Conclusions 133

9.2 Future Work 136

9.3 Declaration of Generative AI and AI-assisted technologies in the writing process 137

Bibliography **138**

Chapter 1

Introduction

Web 2.0 made the Internet an interactive platform, where users do not only access content, but can also create it, share it, and interact with other users. However, most Web 2.0 services are based on centralized infrastructures, in which data, accounts, and application rules are managed by a central organization.

Web3[1] was created with the goal of reducing this dependence on intermediaries through the use of decentralized technologies, in particular blockchain and smart contracts. In this model, users can interact with decentralized applications through cryptographic wallets and maintain greater control over their digital assets, while the rules of the system can be executed directly through smart contracts on the blockchain. The transition from Web 2.0 to Web3 therefore represents an evolution from applications mainly controlled by central platforms toward systems in which part of the control and management of resources can be distributed among the users of the network.

One of the main technologies underlying Web3 is blockchain[2], which makes it possible to record and share information in a ledger shared among the nodes of a distributed network, without relying on a single central authority. Blockchains such as Ethereum[3] have extended this model beyond the simple transfer of value by introducing smart contracts, programs executed on the blockchain that make it possible to define rules and operations that are applied directly by the network. Thanks to smart contracts, it has become possible to develop not only decentralized applications, but also actual digital organizations, known as Decentralized

Autonomous Organizations (DAOs).

A DAO[4] is an organization in which decisions are made collectively by its members according to rules defined through smart contracts, in a decentralized way. Participants can submit proposals, vote, and, when a proposal is approved, execute the planned actions without the need for a central authority that directly controls the process. DAOs therefore represent a concrete application of the principles of Web3, since they make it possible to distribute control and decision-making among the participants of the organization.

DAOs no longer represent only an experimental application of blockchain technology, but have become economically relevant organizations. At the beginning of 2025, more than 10,000 active DAOs and over 3.3 million total participants had been identified. The total value of assets held in DAO treasuries increased from approximately $520.7 million in 2021 to approximately $22.5 billion in 2025 [5]. These figures show that DAO governance mechanisms can control significant economic resources.

As the economic relevance of DAOs increases, the way in which decision-making power is distributed among their members becomes particularly important. The decentralization of the infrastructure does not necessarily imply a balanced distribution of decision-making power[6]. In many governance models, the voting weight of each member depends on the amount of governance tokens owned or on the capital invested in the organization. This approach makes it possible to quantify voting power in a simple and transparent way, but it introduces an important limitation: the economic investment of a participant does not necessarily represent their ability to correctly evaluate a specific decision.

The problem becomes particularly evident when a DAO has to make decisions concerning different domains. Consider, for example, a DAO that manages investments in technological projects. One proposal could concern an Artificial Intelligence project, another a Cybersecurity project, and another a Blockchain project. A member who has invested a large amount of capital could have a high voting weight in all these decisions, even without necessarily having competences in the specific sectors involved. In contrast, a member with a lower economic investment but with competences directly relevant to a proposal could have very limited influence.

This observation leads to the idea developed in this thesis: the decision-making power of a member could be determined not only by their economic contribution, but also by the competences they possess with respect to the topic of the decision. However, this introduces a new question: how can a DAO verify that a participant actually possesses the competences they claim to have?

One possible solution would be to store this information in a database managed by a central authority that keeps and controls the users' data. However, such a solution would not be fully consistent with the decentralized nature of a DAO, because it would once again introduce an entity on which the organization would have to depend in order to verify the information of its members.

Self-Sovereign Identity (SSI)[7] offers a different approach. The objective of SSI is to allow individuals to maintain control over their digital information and to use it in different contexts without continuously depending on the system that originally produced it. Within this paradigm, a Verifiable Credential (VC)[8] allows a trusted entity, called the Issuer, to digitally certify specific information related to a user, who takes the role of Holder. The Holder can then present the credential to a Verifier, which can verify its authenticity and integrity.

This model naturally fits the problem addressed in this thesis. A university, a training institution, or another organization considered trustworthy could, for example, certify specific competences of a user through a VC. The member keeps their own credential and can later use it within the DAO. The DAO can then verify that the credential comes from an Issuer considered trustworthy and use the certified competences in the calculation of voting power, without having to maintain a centralized system responsible for directly certifying those competences.

The objective of this thesis is to develop a decentralized governance model in which the voting power of members depends in a combined way both on the competences of the member with respect to the topic of a vote and on the capital deposited. Competences are not considered in the same way for every decision. Each proposal is associated with a specific voting topic, and the competences of a member contribute to their voting power according to their relevance to that topic. In this way, the same participant can have a different weight in decisions belonging to different domains. Stake therefore continues to represent the economic involvement of the member in the organization, while competences introduce a

second dimension related to the knowledge relevant to the decision. The model also includes the on-chain verification of the VC used by the member to certify their competences, avoiding delegating the verification to a centralized off-chain component. To develop the proposed model, the state of the art concerning the main DAO governance mechanisms and the approaches that use competences, reputation, and VCs was first analyzed. Subsequently, the architecture of the solution was designed and the formula for calculating the voting power of a member was defined.

Based on the designed architecture, a complete DAO prototype was implemented on Ethereum. The implementation reuses and extends some OpenZeppelin contracts already employed in traditional DAO governance, such as Governor, ERC20Votes, and TimelockController, and introduces new components to integrate certified competences into the governance process. In particular, the registration of member DIDs and competences, a scoring system that assigns a weight to competences according to the topic of the proposal, the management of historical voting power derived from competences, the issuance of VCs by Issuers, and their on-chain cryptographic verification were implemented.

During the development process, four versions of the system were implemented in order to evaluate different design choices and optimizations. The different versions were experimentally compared by measuring the gas consumption of the main DAO operations and of the entire governance cycle. The scalability of the representation was also evaluated as the number of managed skills increased. The behavior of the voting power was then analyzed by varying the stake, the competences, the weights assigned to the two components, and the topic of the proposal. Finally, a security analysis was conducted by considering the main malicious behaviors and possible attacks, the adopted countermeasures, and the residual risks of the prototype.

The thesis is organized as follows:

- Chapter 2 introduces the background required to understand the work, presenting the concepts related to blockchain, Ethereum, smart contracts, DAOs, cryptography, and Self-Sovereign Identity.

- Chapter 3 analyzes the main related works in the field of DAO governance, comparing the proposed model with approaches based on governance tokens, member reputation, competences, and the use of VCs to determine the voting eligibility of members.
- Chapter 4 describes the design of the governance model developed in the thesis, starting from the problem of DAO systems in which voting power mainly depends on stake and introducing a model that combines the economic component with competences certified through Verifiable Credentials. It describes how a member joins the DAO, how they update their competences, how these are verified on-chain, and how the governance process is carried out.
- Chapter 5 presents the main libraries and technologies used to implement the system, including OpenZeppelin, Veramo, and Hardhat.
- Chapter 6 describes how the proposed governance model is concretely implemented, explaining how the governance flow, voting power calculation, quorum, and superquorum are managed, how competences are handled by the system, and how VCs are generated by Issuers and verified on-chain by the DAO.
- Chapter 7 presents the experimental evaluation of the implementation, analyzing the gas consumption of the main DAO operations and the impact of the changes introduced over time across the different solutions developed. It also studies the behavior of a member's voting power as stake, certified competences, and the topic associated with the proposal change.
- Chapter 8 analyzes the security mechanisms used to protect the data and operations of the proposed solution. It examines, from a security perspective, the voting power derived from stake and competences, the relationship between members and their DIDs, VC verification, and the management of trusted Issuers. It also considers the security of topic-based governance and the protection of Treasury funds. Finally, it discusses the main remaining limitations of the prototype, with particular attention to credential revocation and on-chain privacy.

- Chapter 9 summarizes the main contributions of the thesis, discusses the main limitations of the current prototype, and identifies possible future developments, with particular attention to credential revocation and expiration, selective disclosure, and Zero-Knowledge Proofs.

Chapter 2

Background

This chapter introduces the theoretical background required to understand the work developed in the thesis. In particular, it presents the fundamental concepts related to blockchain, Ethereum, smart contracts, DAOs and their governance mechanisms, together with the main cryptographic concepts. The chapter also introduces Self-Sovereign Identity and its main components, including Verifiable Credentials, Verifiable Presentations, and Decentralized Identifiers, which form the basis of the model proposed in the following chapters.

2.1 Blockchain and Smart Contracts

2.1.1 Definition of Blockchain

A **ledger**[2] is a register maintained in a network of distributed nodes that keeps track of an ordered sequence of recorded events. It can be *append-only*, meaning that it is only possible to add new events without ever modifying them.

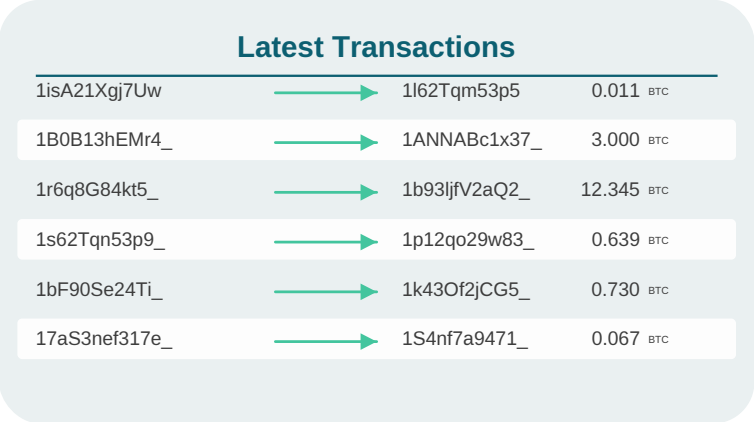


Figure 2.1: Example of a transaction register (Source: [9]).

A **blockchain**[2] is a distributed and immutable ledger, composed of a list of blocks connected to each other through hash pointers. Each block in the chain contains the transactions performed by the network. The block header contains the metadata of the block, a random nonce value, and the hash of the previous block, which represents a unique reference to that block.

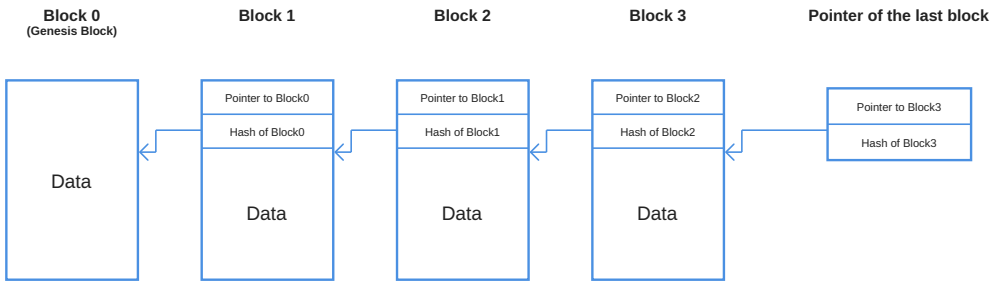


Figure 2.2: Example of a blockchain (Source: [10]).

The blockchain is maintained by a network of distributed nodes, each of which keeps the same complete and updated copy of the ledger. If an attacker modifies the data in a block k, its hash changes, and therefore the hash pointer contained in the following block k+1 no longer matches the previous one. This creates a chain effect: block k+1 no longer has a valid hash for the previous block. As a

consequence, block k+2 will also have an incorrect hash pointer, and so on. In the end, the entire blockchain will be compromised, and the modification will be easily detectable.

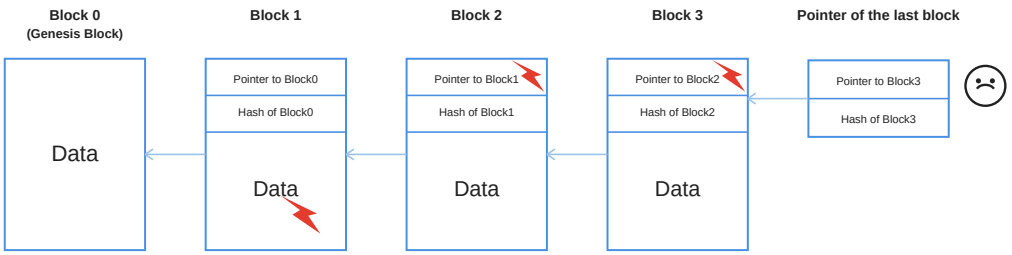


Figure 2.3: Example of blockchain immutability (Source: [10]).

To make a malicious modification possible, the attacker must modify all the following hash pointers and, for each following block, recalculate the Proof of Work.

2.1.2 Consensus: Proof of Work and Proof of Stake

In a blockchain, consensus is the process through which the nodes of the network agree on which transactions or blocks are valid and in which order they must be added to the shared ledger. In the Bitcoin blockchain, consensus is based on Proof of Work, while in Ethereum it is based on Proof of Stake.

In a solution based on **Proof of Work**[11], instead of voting according to the number of participating nodes, consensus is determined according to the amount of computational power that each node provides. Even if an attacker registers multiple identities, these identities do not add computational power, so the attacker cannot vote multiple times. A cryptographic puzzle that is computationally expensive is defined. A miner node must find, through brute force, a random value x, called a nonce, such that the concatenation between the hash of the header of the created block and the random nonce value is lower than a difficulty threshold T, $H(\textbf{header} \parallel x) < T$.

The node that solves the problem first becomes the ”leader” in the selection of the block and ”wins” the right to unilaterally decide which valid transactions

to include in the next block of the blockchain and in which order. In addition, it receives a monetary reward for its work.

Once each node in the network receives the block, it quickly computes the hash of the received block header and checks that this hash is lower than the established difficulty target. It also checks that the transactions contained in the block are valid. If they are not valid, the block is not accepted. If the block passes the verification, each node adds it to its own copy of the blockchain, updating the transaction ledger.

Proof of Stake (PoS)[12], instead, is based on the capital, expressed in ETH, that participants put at stake as a guarantee of their behavior. Ethereum currently uses Proof of Stake as its consensus mechanism.

The participants in the consensus process are called validators. To directly activate an Ethereum validator, it is necessary to deposit 32 ETH as stake. This deposit acts as a guarantee: a validator that correctly follows the protocol can receive rewards, while incorrect behavior can cause penalties and, in the most serious cases, the loss of part of the stake.

Time in Ethereum is divided into intervals called slots, each lasting about 12 seconds. A set of 32 slots forms an epoch. In each slot, the protocol selects a validator as the block proposer, responsible for building a new block containing the transactions to be added to the blockchain.

Once the proposed block is received, the other validators check that the block and the transactions contained in it follow the rules of the protocol. If they consider the proposal valid, they produce an attestation, that is, a cryptographically signed vote through which they indicate which block they consider correct and which chain they consider to be the valid one.

2.1.3 Ethereum: Ethereum Blockchain, Smart Contracts, Transactions, ERC-20 Tokens and Gas

Ethereum[3] is a blockchain that is used not only as a distributed transaction ledger, but also as a platform for executing programs on nodes distributed across its network. Ethereum maintains a shared global state that can be modified through

transactions and through the execution of programs called smart contracts. The native cryptocurrency of the network is Ether (ETH), which is used to transfer value, participate in the Proof of Stake consensus mechanism, and pay the gas related to operations executed on the network. Each computational operation requires a certain amount of gas, which represents the amount of resources needed to execute it on all the nodes of the network.

Code execution takes place on all nodes through the **Ethereum Virtual Machine** (EVM), which allows Ethereum nodes to execute smart contracts deterministically, so that, starting from the same state and the same transaction, the nodes can obtain the same result and agree on the new state of the blockchain.

An **Ethereum address**[13] is a unique identifier associated with an account and is represented by a hexadecimal string beginning with `0x`. It can be used to receive and send ETH, interact with smart contracts, and identify an entity on the blockchain. Ethereum distinguishes between addresses controlled by users, known as **Externally Owned Accounts** (EOAs), and addresses associated with smart contracts.

A **wallet** in Ethereum is a software application that allows a user to manage the cryptographic keys required to interact with the Ethereum network. In particular, the wallet manages a **private key** associated with a corresponding **public key**.

The Ethereum address of an account is derived from the public key by applying a cryptographic hash function and taking part of the resulting hash. The private key is used to authorize transactions and prove control over the associated account.

A **smart contract**[14] is a program stored on the Ethereum blockchain that contains rules and functions executed when a user or another smart contract interacts with it. A smart contract is usually written in a language such as Solidity and then deployed on the blockchain. After deployment, it has its own address, through which it can be used.

Users interact with the smart contract through transactions or calls to its functions. If a function modifies the data of the contract, the modification is verified by the Ethereum network and recorded on the blockchain.

The execution of operations requires the payment of gas, which depends on the computational resources used. The main characteristic of smart contracts is that the rules defined in the code are automatically applied, without the need for a central intermediary.

Gas[15] in Ethereum is the unit used to measure the amount of computational work required to execute a transaction or a function of a smart contract. Each operation executed by the EVM has a specific gas cost.

Gas is mainly used to prevent users from using the computational resources of the network for free and to protect Ethereum from excessively expensive operations or infinite execution loops. For this reason, the sender of a transaction pays a `gas fee` in ETH.

The final cost of a transaction depends on the amount of gas actually used and on the price paid for each unit of gas.

The transaction fee is calculated as follows: $\textit{Transaction Fee} = \textit{Gas Used} \times \textit{Gas Price}$. The gas price is usually expressed in `gwei`, where 1 gwei corresponds to 10^{-9} ETH.

Each transaction also specifies a `gas limit`, which is the maximum amount of gas that can be used for its execution. If the execution requires more gas than the available limit, the transaction fails; however, the gas already consumed during execution is not refunded.

An Ethereum **transaction**[16] is an operation sent from an account to the network to modify the state of the blockchain. A transaction can be used, for example, to transfer ETH from one account to another, execute a function of a smart contract, or deploy a new smart contract.

Each transaction is created and digitally signed by the sender using their private key. The signature allows the network to verify that the transaction was actually authorized by the owner of the account.

A transaction contains several pieces of information, including the sender account that sends the transaction, the recipient account, the amount of ETH, or `value`, that may be transferred, the additional data, the progressive nonce associated with

the transactions of the account, and the maximum gas limit that the sender is willing to spend.

2.1.4 Ethereum Storage Model

Ethereum stores a global state, called the `world state`, which contains the information associated with all Ethereum accounts. Each Ethereum account is represented in memory by an address, composed of a sequence of 42 alphanumeric characters, a nonce, which keeps track of the operations associated with the account, a balance in ETH, a `storageRoot`, which in the case of a smart contract represents the Merkle root of the persistent storage of the contract, and the hash of the smart contract code, called `codeHash`.

When a transaction is executed, it can modify the global state of Ethereum. The result of the execution therefore produces a new global state.

In addition to the global state, each block also contains a set of transactions and the related transaction receipts. These receipts contain information about the result of the execution of a transaction, such as the success or failure status, the gas used, and any events generated by smart contracts.

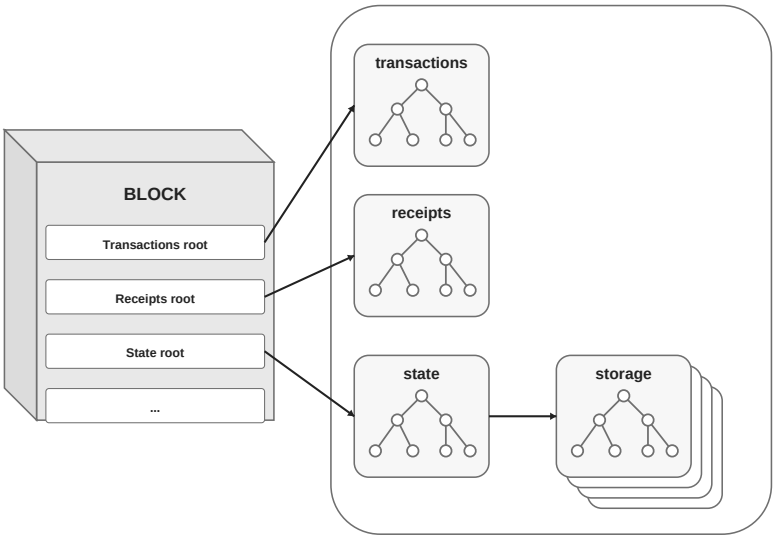


Figure 2.4: Ethereum Storage Model (Source: [17]).

For each block, Ethereum uses three main associated cryptographic roots[18] to store the state of Ethereum. The `stateRoot` represents the Merkle root of the world state resulting after the execution of the block. The `transactionsRoot` represents the Merkle root of the transactions included in the block. The `receiptsRoot` represents the Merkle root of the receipts produced by the execution of the transactions.

Regarding the actual storage of the data used for smart contract execution, the Ethereum Virtual Machines[19] running on the nodes store the data in the following way. The bytecode is stored in the form of a hash in the world state of the Ethereum block, while the actual code is stored in a database on the node where the EVM is running. The same happens for the persistent storage of the contract: the Merkle root of the data is stored in the world state, while the actual data is stored in an internal database of the node.

Temporary data is stored in volatile memory, similar to RAM, which is cleared at the end of the contract execution. In particular, there is a LIFO stack in which instruction operands are pushed and from which they are then read to compute intermediate results, a linear byte buffer called `Memory` containing temporary data and computation results, and a read-only area called `Calldata` containing the input data sent by the caller of the contract.

2.1.5 Solidity

Solidity[20] is a high-level, statically typed programming language used to develop smart contracts compatible with the EVM. Solidity code is compiled into bytecode, then deployed on the blockchain and executed by the EVM. Through Solidity, it is possible to define state variables, data structures, functions, and events that determine the behavior of a smart contract.

State variables allow persistent data to be stored in the contract storage, while functions define the operations that can be executed by users or by other smart contracts. Solidity provides several data types, including integers (`uint`, `int`), Boolean values (`bool`), Ethereum addresses (`address`), arrays, and user-defined types. Among the most commonly used data structures are mappings, which allow a key to be associated with a value, for example, to associate each Ethereum

address with its deposited balance.

Functions in Solidity can have different visibility levels, which determine where they can be called from. If a function is `public`, it can be called both from inside the contract and from outside it; if it is `external`, it can be called from outside the contract; if it is `internal`, it can be called only by the contract itself and by contracts that inherit from it; and if it is `private`, it can be called only from within the contract in which it is defined. Functions can also be declared `view` when they read the state of the contract without modifying it.

Modifiers are constructs that allow conditions to be added before the execution of a function. For example, a modifier can check that the caller of a function is an authorized address, and if the condition is not satisfied, the execution is blocked.

Events allow a smart contract to record information in the blockchain logs. These logs can be read by external applications to obtain the information transmitted by the emitted events.

Solidity also supports inheritance, interfaces, and libraries, which allow code to be better organized and reused. Inheritance allows a smart contract to inherit variables, functions, and behaviors from another contract, avoiding the need to rewrite existing code. The derived contract can add new functionalities or modify some behaviors of the base contract. Interfaces, instead, define which functions a contract makes available, without necessarily specifying their implementation.

Libraries are collections of reusable functions that can be used by multiple smart contracts to avoid rewriting the same logic. A library can contain, for example, functions to perform calculations, manipulate data structures, or apply common operations. These functions are then copied inline into the contracts that use them.

2.1.6 ERC-20 Fungible Tokens

ERC-20 tokens are fungible tokens implemented on the Ethereum blockchain, that is, digital assets in which each unit is equivalent to and interchangeable with another unit of the same type. For example, BAT (Basic Attention Token)[21] is an

ERC-20 token used within the Brave browser ecosystem, where users can receive BAT as a reward for viewing advertisements. Another example is MANA[22], an ERC-20 token used within the Decentraland virtual platform to purchase digital goods and services, such as items for avatars. In both cases, the tokens are fungible: each BAT token is equivalent to and interchangeable with any other BAT token, just as each MANA token is equivalent to and interchangeable with any other MANA token.

The ERC-20 standard[23] defines a set of common functions and rules that a smart contract must implement to manage tokens consistently. The main functions are `totalSupply()`, which returns the total amount of existing tokens; `balanceOf(address)`, which returns the token balance owned by an address; `transfer(address, amount)`, which transfers tokens from one account to another; `approve(address, amount)`, which authorizes another address to spend a certain amount of its tokens; `allowance(owner, spender)`, which indicates how many tokens a spender address is authorized to spend; and `transferFrom(from, to, amount)`, which allows a subject authorized through `approve()` to transfer tokens from the `from` address to the `to` address.
An ERC-20 contract maintains a balance table, represented by a mapping between Ethereum addresses and the number of tokens owned by each address. For example, it can contain [Alice $\rightarrow$ 100 ERC-20 tokens, Bob $\rightarrow$ 50 ERC-20 tokens, Carol $\rightarrow$ 25 ERC-20 tokens].

The contract also maintains an allowance table, represented by a mapping that associates a holder address, which owns the tokens, with an allowed address authorized to spend the tokens, and with an allowance integer representing the maximum number of tokens that the authorized address can still spend. A given entry is updated when the `approve()` and `transferFrom()` functions are called.

Tokens can be created through an operation called minting, which increases the `totalSupply`, and can also be destroyed through burning, which reduces the totalSupply.

ERC-20 tokens can also be used as governance tokens, that is, tokens that give

the members of a DAO a certain voting power. However, the ERC-20 standard mainly records the current token balance owned by each account and, by itself, does not make it possible to know how many tokens a user owned at a previous point in time. In governance systems, instead, it is important to be able to determine the voting power held at a specific point in time, for example when a proposal is created. For this reason, extensions such as `ERC20Votes` can be used, which add checkpoint mechanisms through which it is possible to record and retrieve the historical voting power of an account.

2.2 Decentralized Autonomous Organizations

A **Decentralized Autonomous Organization**[4] is a digital organization composed of multiple members, without a central leadership, and governed by its members in a decentralized way. The rules, fund management, and decision-making processes are not entrusted to external central actors, but to a smart contract executed on a blockchain.

A DAO is created when its smart contract is deployed on the blockchain. This smart contract implements in code the statutes containing the rules for managing the organization. It defines how members join the DAO, how funds are spent, and how the decision-making process works.

A DAO is called autonomous because, once deployed, its governance rules are automatically enforced by smart contracts. Members still interact with the DAO by submitting proposals, voting, and executing allowed actions, but the execution of these rules does not require a central authority to manually validate each operation.

A DAO is called decentralized because there is no CEO or board of directors governing the organization, but power is distributed among the members, proportionally to the funds they deposit in the DAO.

Within DAOs, governance tokens are commonly used to quantify a member's voting power. A `governance token` is a cryptographic token that grants its holder the right to participate in the decision-making processes of a DAO. Depending on the governance model, token holders may submit proposals, delegate their voting

power, or vote on proposals regarding the organization's governance and activities. Voting power is typically proportional to the number of tokens held or delegated.

Depending on the governance mechanism adopted by the organization, these tokens can be obtained in different ways. The most traditional distribution mechanism consists of assigning governance tokens in proportion to the funds that members contribute or deposit into the protocol. However, other distribution mechanisms are also possible. For example, Uniswap's UNI tokens can be purchased on cryptocurrency exchanges and were initially distributed to eligible users through an airdrop.

Members who meet specific requirements can create a proposal, which asks the other members to make the organization perform a specific action, such as spending DAO funds, changing a governance rule, or adding a new member.

Members vote using their governance tokens. Each token represents a share of voting power, and the proposal that receives the majority of the votes wins. The vote is transparent and verifiable on the blockchain. Once the voting process is closed, the result is binding. If a proposal passes, the smart contract automatically executes the decision.

For example, a possible scenario can be represented by a group of people creating a DAO to invest together in startups. Members who invest funds receive governance tokens proportionally. These invested funds are locked in a smart contract, so that they can only be withdrawn through an approved governance proposal. In this scenario, a member proposes investing 100 ETH in startup X. The members of the DAO vote using their tokens. If the proposal receives the majority of the votes, the smart contract sends the funds to startup X.

2.2.1 Characteristics of a DAO

In a DAO, the deposit and transfer of value take place in the form of cryptocurrencies.

A DAO has the ability to organize and store voting processes. The functionalities offered by a DAO and its rules are encoded on a blockchain through a smart contract.

Decision-making power is not in the hands of a single central actor who can make all decisions on behalf of the organization. Decisions are made autonomously and in a decentralized way by the members of the organization.

The proposals, votes, and actions performed by the DAO are transparent on the blockchain, together with the smart contract that defines its governance. Consensus is reached through voting processes based on predefined rules, such as majority voting, superquorum, and the validity of votes based on reaching a quorum. The decision-making process always begins with a proposal. A proposal is a formal proposal submitted to the governance system that describes a decision to be made and the actions to be executed if it is approved by the majority of the DAO members. To create a proposal, a member must meet a Proposal Threshold, that is, a minimum requirement needed to create a proposal, such as owning at least a certain amount of tokens.

Voting rights are assigned to each member based on the number of tokens they own. The quorum is the minimum amount of participation required for a vote to be considered valid. A DAO member can delegate their tokens to themselves or to another member, transferring their voting power to the delegate without losing ownership of the delegated tokens.

2.2.2 Governance Mechanisms

The governance of a DAO is the set of rules and mechanisms through which members participate in the decisions of the organization. It defines who can propose a decision, who can vote, how much each vote weighs, which conditions must be satisfied for a proposal to be approved, and how the decision is executed.

The **Governance Contract** is the smart contract that implements the governance rules, that is, it implements the creation of proposals, the management of voting, the counting of results, and the final approval or rejection of a proposal.

In the decision-making process, a proposal is generally created in which a member provides a description, the actions to be executed in case of approval, the duration of the voting process, the approval threshold, and the validity quorum.

Members vote on the proposal. There is a voting period, that is, a period of

time during which eligible members can vote on a proposal before it is closed and the votes are counted. At the end of the voting period, the votes received by the proposal are counted. If the quorum is reached, the proposal is `valid`. If the proposal exceeds the threshold of affirmative votes, it becomes `approved`. In this case, the predefined actions are executed. If it does not reach the quorum or the voting threshold, it becomes `rejected`.

Execution is the phase in which a proposal approved by the governance is actually applied by executing the planned operations through smart contracts.

Execution can be automatic and on-chain, automatically starting an action defined by the smart contract, or manual and off-chain. In the latter case, after the proposal has been approved, the action is not automatically executed by the smart contract but requires the intervention of external actors, who perform it manually, for example by manually sending funds, calling a function on a contract, or carrying out activities in the real world.

2.2.3 Timelock Mechanism

A **Timelock** is a contract that imposes a mandatory delay between the approval of a proposal and the actual execution of the action. After a proposal is approved, the action is queued in the Timelock. It must wait for a predefined amount of time before being executed by the Timelock. This allows the community to notice malicious proposals and gives it time to react.

2.2.4 Snapshot

A **snapshot** is a mechanism through which the DAO records the voting power of participants at a specific point in time, typically at block number X or at a specific timestamp.

When votes are counted, the token balance at the time when a participant votes is not used; instead, the balance at the point in time when the proposal was created is used. If a participant moves tokens or buys tokens only to vote, in order to influence the voting process, after the block in which the snapshot was taken, this action becomes ineffective.

2.3 Cryptographic Concepts

2.3.1 Hash Functions

A **hash function** is an algorithm that transforms an input of arbitrary length n into a hash value of fixed length. A hash function has the following fundamental properties. The same input must always produce the same output. The hash computation must be fast. Hash values should be uniformly distributed over the output space. The probability that two different inputs produce the same hash should be low.

A **cryptographic hash function**[24] is a hash function that makes it impossible to find collisions or determine the original input from the hash value. The property of *collision resistance* states that, given an input value x_1, it must be computationally difficult to find another distinct input value x_2 such that $H(x_1) = H(x_2)$.

The property of *preimage resistance* states that, given an output h, it must be computationally impossible to determine the corresponding input value x such that $H(x) = h$, except by using brute force.

An *avalanche effect* occurs when changing even a single bit in the same document produces a completely different hash value.

The *hiding property* states that if a secret value is combined with a random value and then hashed, the output should not reveal information about the secret value.

The *puzzle-friendliness property* states that it must be impossible to find an input that produces a hash with specific properties without trying many combinations.

The *pigeonhole principle* states that if there are more elements than available containers, at least one container will contain more than one element. Since the number of possible inputs is much greater than the number of possible outputs of a hash function, it is inevitable that at least two different inputs are mapped to the same hash.

2.3.2 Asymmetric Cryptography

Asymmetric cryptography[25] uses different keys to encrypt and decrypt messages. Each user has a **public key** and a **private key**. Messages can be encrypted using one of the two keys, but they can only be decrypted using the other key. Public keys can be published by the key owner, and it is not necessary to transmit a key securely.

To guarantee confidentiality, the sender encrypts a message using the recipient's public key. The recipient decrypts the message using their private key, which only they know. Only that recipient can decrypt the message.

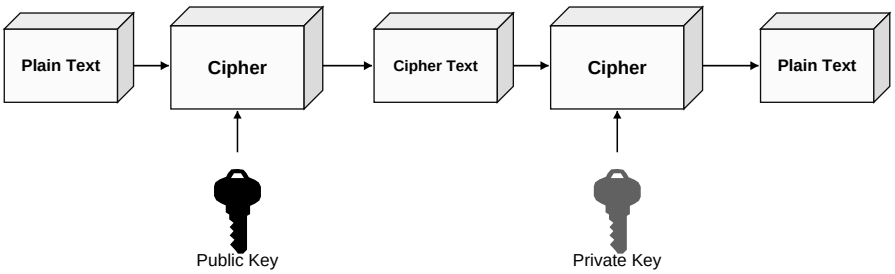


Figure 2.5: Example of asymmetric-key cryptography (Source: [26]).

2.3.3 Digital Signatures

A **digital signature** is a digital hash fingerprint of some data that accompanies the data itself, providing proof that the message actually comes from the legitimate sender.

The message is first transformed into a hash value. This hash is then signed using the sender's private key. The message and the digital signature are sent to the recipient.

The recipient receives the message and calculates the hash value of the received data using the same hashing algorithm. Using the sender's public key, the recipient decrypts the received fingerprint and checks that it is equal to the calculated hash value.

If the digests match, this means that the message has not been altered and comes from the legitimate sender.

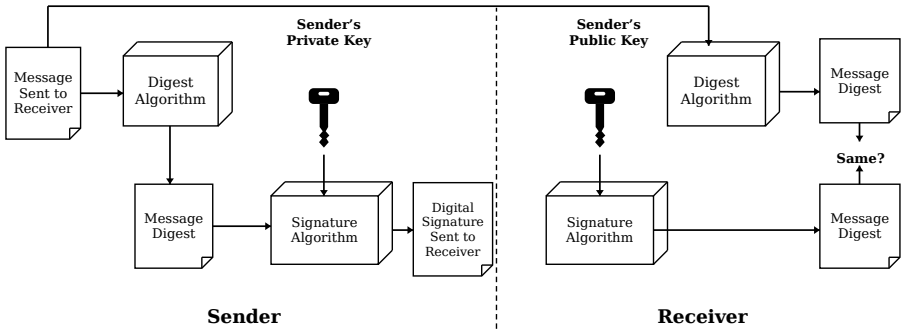


Figure 2.6: Example of a digital signature process (Source: [26]).

A digital signature does not guarantee data confidentiality. To guarantee both confidentiality and integrity, the sender encrypts the encrypted message plus the signature using the recipient's public key in order to guarantee confidentiality. The recipient decrypts the message using their private key. The process explained previously is then used.

2.4 Self-Sovereign Identity

In centralized digital identity systems, information related to a user's identity is generally managed by an external central authority, such as a public institution or a university. This authority is responsible for recording the user's data, verifying their identity, and, when necessary, confirming information about the user to third parties.

The problem with this approach is that the user's data is managed by a single authority, on which the user depends to store, update, and prove this information to third parties.

This reduces the user's direct control over their own data and introduces a single point of failure. In fact, if the central system becomes unavailable, access to and verification of identities can also be compromised.

For example, if a degree can only be verified through the university database, the student continuously depends on the university to prove their qualification. Using this approach, proving a single attribute may require providing more data than necessary. For example, to prove that a person is over 18 years old, it may be necessary to provide a document that also shows their name, address, and full date of birth.

In addition, information stored on a specific platform may be difficult to use on another one.

2.4.1 Definition and Principles

Self-Sovereign Identity (SSI)[7] is a digital identity management model in which the individual is the sole owner of their identifying data, maintaining direct control over their information without having to depend on a central authority that stores and manages their data.

In the SSI model, the user receives credentials from a trusted entity that certify information about the user. The user independently stores their credentials in a digital wallet and decides which information to share, with whom, and for how long. In this model, identity must exist independently of central authorities. The user must have full control over their personal data, deciding what to share and with whom. Furthermore, the user must always be able to access their data. The identity must be persistent over time and must not depend on the lifetime of a single platform.

Credentials must be transferable across different systems, making it possible to use the same identity on multiple platforms. Data sharing must take place only with the user's explicit consent. Only the data strictly necessary for the requested service should be shared. For example, if entering a venue requires proving that a person is over 18 years old, there is no need to share their name, address, or full date of birth.

The user's rights must take priority over the interests of organizations. Privacy is central, not optional.

2.4.2 SSI Actors

In the SSI model, there are three main actors. There is an **Issuer**, which is the entity that certifies information about the user and issues a Verifiable Credential, digitally signed with its private key. For example, a university can certify that a person has a degree in Computer Science. There is a **Holder**, which is the entity that receives the credential from the Issuer, stores it, and can later decide to present it to a Verifier.

There is a **Verifier**, which is the entity that receives the credential and verifies its validity, checking that it was actually issued by an Issuer considered trustworthy and that its content has not been modified.

For example, a university issues a digital credential to a student certifying that the student has obtained a Master's degree in Computer Science. The university is the Issuer, while the student is the Holder. Later, the student wants to prove this competence to a DAO. The student therefore presents the credential to the DAO, which takes the role of Verifier and checks that the credential has not been modified and that it was issued by an Issuer considered trustworthy.

2.4.3 Verifiable Credentials

Verifiable Credentials (VCs)[8] are digital credentials that allow an Issuer to verifiably certify a set of information about another subject, the Holder. This information is represented through claims, that is, statements about attributes related to the subject of the credential. For example, a university can issue a VC certifying that a student has obtained a Master's degree in Computer Science. The claims can include the name of the subject, the faculty, the final degree grade, and the date of graduation.

A VC contains both the information certified by the Issuer and the metadata needed to cryptographically verify that the credential actually comes from the declared Issuer and that its content has not been modified after issuance.

A VC contains information such as the identity of the Issuer, the type of credential, the subject to which the claims refer, the certified attributes, the possible validity period, and a reference for verifying its status, that is, whether it is valid, expired, or revoked.

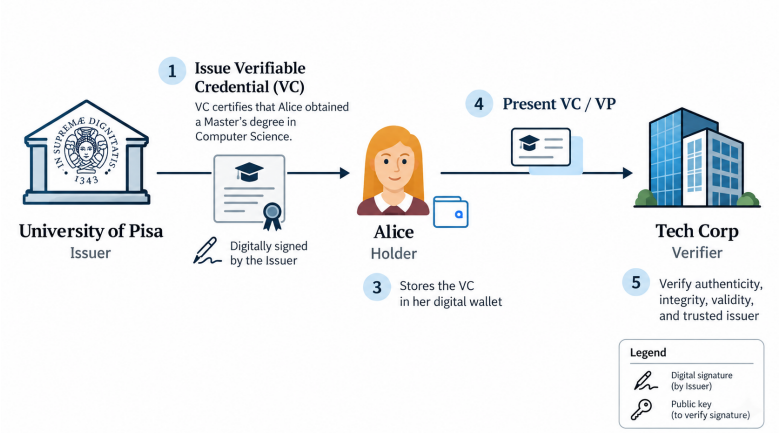


Figure 2.7: Verifiable Credentials Model

The process begins with the issuance of the credential. The Issuer creates a VC containing the claims that it intends to certify and uses a cryptographic mechanism based on hashing and digital signatures so that the origin and integrity of the credential can later be verified. The credential is then delivered to the Holder, who can store it and later use it to prove the information contained in it. When the Holder wants to use a credential, they can present it to the subject interested in verifying the certified information, that is, the Verifier.

When the Verifier receives the credential, it checks that the credential has not been altered and that it was actually issued by the declared subject.

However, this does not automatically mean that the Verifier must consider that Issuer trustworthy. For example, a credential certifying a degree may be technically authentic, but the Verifier may decide to accept it only if it was issued by a recognized university.

An example of the use of a VC is a university issuing a VC to Alice certifying that she has obtained a Master's degree in Computer Science. The university acts as the Issuer, Alice acts as the Holder and stores the credential in her digital wallet.

When Alice needs to prove her qualification to an organization, she can present the credential to the corresponding Verifier. The Verifier checks that the credential is authentic, that it has not been modified, that it is still valid, and that it comes from an Issuer considered trustworthy. In this way, Alice can reuse a verifiable digital certification without asking the university for a new attestation every time she needs to prove the same qualification.

2.4.4 Structure of a Verifiable Credential

A VC is composed of a set of information that describes who issued the credential, the subject to which it refers, which information it certifies, and for how long it is valid. The W3C defines some standard attributes that can be used within a VC.

There is an Issuer that identifies the subject that issued the credential. There is a Credential Subject that contains the information related to the subject. The claims, that is, the information certified by the Issuer, are included in this section.

There is a Type field, which indicates the type of credential, for example whether it is a university credential, a professional credential, or another specific type of credential.

There is a validity period through which it is possible to indicate from which moment and until when the credential should be considered valid.

There is also information for cryptographic verification, through which a VC is protected by a cryptographic mechanism that allows the Verifier to check its authenticity and integrity.

```
{
  "@context": [
    "https://www.w3.org/2018/credentials/v1",
    "https://www.w3.org/2018/credentials/examples/v1",
    "https://w3id.org/security/suites/ed25519-2020/v1"
  ],
  "id": "http://example.edu/credentials/3732",
  "type": [
    "VerifiableCredential",
    "UniversityDegreeCredential"
  ],
  "issuer": "https://example.edu/issuers/565049",
  "issuanceDate": "2010-01-01T00:00:00Z",
  "credentialSubject": {
    "id": "did:example:ebfeb1f712ebc6f1c276e12ec21",
    "degree": {
      "type": "BachelorDegree",
      "name": "Bachelor of Science and Arts"
    }
  },
  "proof": {
    "type": "Ed25519Signature2020",
    "created": "2022-02-25T14:58:43Z",
    "verificationMethod":
      "https://example.edu/issuers/565049#key-1",
    "proofPurpose": "assertionMethod",
    "proofValue":
      "zeEdUoM7m9cY8ZyTpey83yBKeBcmcvbyrEQzJ19rD2UXArU2U1jPGoEt"
      "rRvGYppdiK37GU4NBeoPakxpWhAvsVSt"
  }
}
```


Figure 2.8: Example of a Verifiable Credential (Source: [27]).

2.4.5 Verifiable Presentation and Selective Disclosure

A **Verifiable Presentation (VP)** is a digital presentation created by the Holder to provide a Verifier with one or more pieces of information coming from their VCs. A VP can contain data derived from a single VC or combine information coming from multiple credentials. Its purpose is to allow the Holder to present to the Verifier the information needed for a specific verification, without necessarily providing all the credentials they possess. VPs are generally created for a specific interaction and are not intended to be permanent credentials stored over time.

For example, Alice has a VC related to her degree in her wallet and a second credential certifying her knowledge of the English language. To apply for a job that requires both qualifications, Alice can create a VP containing the necessary information from the two credentials and present it to the company, which acts as

the Verifier.

Selective Disclosure is a mechanism that allows the Holder of a VC to reveal to the Verifier only part of the information contained in the credential, avoiding the sharing of data that is not necessary for the required verification.

For example, a digital credential can contain first name, last name, date of birth, address, nationality, and age. If the Verifier only needs to check that the subject is over 18 years old, it is not necessary to show all this information. The Holder can present only the age, without revealing the other attributes contained in the credential.

2.4.6 Decentralized Identifiers

A **Decentralized Identifier (DID)**[28] is a digital identifier designed to identify a subject without necessarily depending on a central authority that assigns and manages the identifier. A DID can refer to a person, an organization, a device, or another entity, and it is associated with cryptographic mechanisms that allow its holder to prove control over it. However, a DID by itself does not certify the real identity or the characteristics of the subject.

A DID generally has the following structure. The first part, `did`, is a fixed string that indicates that it is a Decentralized Identifier. The second part, the DID method, specifies how the DID is registered, for example through a blockchain or the web. The third part is the unique identifier value.

For example, a DID can be "did:etheraum:123456789abcdef". When a DID is created, it is usually associated with a cryptographic key pair, one private and one public.

A DID Document is the data structure associated with a DID that contains the public information needed to cryptographically verify who controls that identifier and to understand how it can be used.

To verify a credential associated with that DID, a Verifier must first perform DID Resolution, that is, retrieve the corresponding DID Document. Inside the document, the Verifier finds one or more verification methods, which indicate

which public keys or other cryptographic materials are authorized to be used for that DID.

The verification process can be described as follows. The controller of the DID owns a private key, which is kept secret, and uses this key to digitally sign a message or a credential. The Verifier receives the signed data, retrieves the DID of the author of the signature, and resolves that DID to obtain the corresponding DID Document. From the document, the Verifier then identifies the verification method used and retrieves the corresponding public key. With this key, the Verifier verifies the received digital signature.

If the signature is valid, the Verifier can conclude that the data was signed by the subject who owns the private key corresponding to the public key authorized in the DID Document and that the signed content was not modified after the signature.

The document can also contain service endpoints, that is, addresses of services associated with the DID.

```
"verificationMethod": [
  {
    "id": "did:example:123#key-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:example:123",
    "publicKeyMultibase": "zExampleKeyValue"
  }
]
```

Figure 2.9: Example of a DID Document (Adapted from: [28]).

To retrieve the information associated with a DID, a process called DID Resolution is used. A component called a DID Resolver receives a DID as input, identifies the corresponding DID method, and applies the procedure defined by that method to obtain the corresponding DID Document and related metadata. The way in which the document is retrieved depends on the DID method.

In the context of Verifiable Credentials, DIDs can be used to identify the Issuer, the Holder, or the Credential Subject. For example, when a Verifier receives

a credential signed by an Issuer identified through a DID, the Verifier can resolve the Issuer's DID, obtain the corresponding DID Document, and retrieve the public key needed to cryptographically verify the credential.

Correct verification of the signature proves that the credential was produced using the cryptographic material associated with the Issuer, but it does not automatically imply that the Issuer should be considered trustworthy.

Chapter 3

Related Work

This chapter analyzes the main existing works on DAO governance, with particular attention to the limitations of models based exclusively on governance tokens and to alternative approaches based on competences, reputation systems, and solutions that use VCs to verify a user's voting eligibility.

Many DAO governance systems assign voting power according to the governance tokens owned by each participant.

An example of a token-based DAO is **Compound**[29], a *Decentralized Finance* (DeFi) protocol that allows users to supply digital assets to the protocol in order to earn a return, or to borrow assets by providing collateral and paying an interest rate. Lending and borrowing operations are managed through smart contracts, without the need for a central financial intermediary.

The evolution of the protocol is managed through **Compound Governance**, an on-chain governance system based on the **COMP** token. Through governance, participants can propose and vote on changes to the protocol, such as the introduction of new supported assets, modifications to the parameters and models that determine interest rates, and updates to some functionalities.

The voting power in Compound governance is determined by the amount of COMP tokens delegated to an address. An address with a larger amount of delegated COMP therefore has greater weight in governance decisions.

A proposal is approved if the votes in favor exceed the votes against and the

quorum required by the governance is reached. Once approved, the proposal is added to the `Timelock`, which introduces a time interval before its on-chain execution. After this interval has elapsed, the changes defined by the proposal are applied to the smart contracts of the protocol.

The LAO[30] is a decentralized *for-profit* organization that allows its members to pool capital and collectively decide which blockchain projects to invest in. Its operation is similar to that of a decentralized venture capital fund: a project can be submitted to the DAO and, for the related proposal to proceed through the governance process, it must be sponsored by a member. The members can then evaluate the project and vote on whether to use part of the common funds to make the investment.

Members of The LAO can contribute capital in Ether and receive *membership units*. These units represent both the economic participation of the member in the organization and a corresponding share of the voting rights. A participation equal to 1% therefore corresponds to 1% of the voting rights and of the economic rights on profits.

If an investment proposal is approved, a waiting period is required before its execution, during which members who disagree with the decision can use a `ragequit` mechanism, leaving the DAO and withdrawing their share of the funds that have not yet been allocated. This mechanism therefore allows a member not to remain economically bound to a decision approved by the majority with which they disagree. In addition to investments, governance can also concern organizational aspects, such as the admission or removal of members.

This example presents some similarities with the solution developed in this thesis, in which DAO members deposit capital in ETH and obtain voting power, and can create and vote on investment proposals directed toward startups registered in a dedicated registry. The addition of new startup addresses to the registry, which may later become recipients of investments, also takes place through a proposal submitted to the governance process.

Another example is **Uniswap**[31], a DeFi protocol that allows users to exchange digital assets directly through smart contracts, without the need for a centralized

intermediary. The exchanges are performed using liquidity pools, in which the assets used by the protocol are deposited.

UNI holders can use their tokens to participate in the governance of the protocol. Proposals can concern, for example, the management of protocol fees, the use of treasury funds, and the modification of some system parameters or functionalities.

Before reaching the on-chain governance phase, proposals can be discussed by the community through the governance forum and can go through preliminary stages of discussion and off-chain consensus. UNI holders can delegate their voting power to themselves or to another address in order to participate in governance. The weight of the vote depends on the amount of UNI delegated to the address that casts the vote.

A further example is **Nouns DAO**[32], a decentralized organization based on a collection of NFTs called *Nouns*. Each Noun is an NFT depicting a unique digital character, similar to a collectible card, that also grants its owner one vote in the governance of Nouns DAO. A newly created Noun is auctioned every day, with the proceeds going to the DAO's treasury to fund projects approved by the community. Holders can vote directly or delegate their vote to another participant.

Proposals can concern, for example, the funding of community projects through treasury funds, modifications to governance parameters, or the introduction of new graphical elements used in the generation of Nouns.

If a proposal obtains the required majority and reaches the required quorum, it can be approved and executed by the DAO.

Another example is the **Ethereum Name Service (ENS) DAO**[33], which governs the Ethereum Name Service, a decentralized system based on Ethereum that allows human-readable names, such as `alice.eth`, to be associated with blockchain addresses.

The governance is based on the **ENS** token, which was initially distributed to users who owned an ENS name and later also became available on the market. Proposals can concern concrete decisions related to the management of the ENS ecosystem, such as updating the protocol smart contracts, allocating treasury

funds to finance DAO projects, and modifying some system rules, for example those related to name registration costs. The voting weight of each participant depends on the amount of ENS tokens delegated to their address.

Although token-based governance provides a relatively simple mechanism for assigning voting power, it can lead to a strong concentration of decision-making power. Fritsch, Müller, and Wattenhofer[6] analyzed the distribution of voting power in Compound, Uniswap, and Ethereum Name Service (ENS), observing that a significant share of voting power is concentrated in a limited number of addresses.

This problem is also relevant to this thesis, because owning a large amount of tokens does not necessarily imply that a participant has the competences required to evaluate a specific proposal.

To overcome some of the limitations of voting systems based exclusively on tokens, several alternative mechanisms have been proposed. Among these, Fan et al.[34] discuss *Knowledge-Extractable Voting*, an approach in which experts in a specific domain can receive a greater weight in the voting process. The authors then place this approach within the broader concept of **Weighted Voting**, in which additional weights can be associated with the voting power of participants, derived, for example, from reputation or knowledge. The objective is to give greater influence to members who possess experience or knowledge relevant to the decision, rather than considering only the number of tokens they hold. This approach has some similarities with the model developed in this thesis, but it remains at a conceptual level and does not address the problem of certifying the competences of participants through verifiable credentials.

Another approach is based on **reputation**. Saito and Rose[35] propose a DAO in which voting power, instead of depending directly on governance tokens, derives from the reputation acquired by members through their contributions to the DAO. Reputation is represented through non-transferable Soulbound Reputation Tokens, which are used both as membership tokens and as containers of the person's reputation history. These tokens are used to determine the weight of par-

ticipants in the decision-making process. The authors also discuss the possibility of separating reputation and voting into different domains, so that reputation acquired in a specific domain can be used mainly in decisions related to that domain. This extension is proposed as a possible evolution of the system and is not part of the implemented prototype.

A concrete example of governance that explicitly considers the experience of participants is **DeXe Protocol**[36]. DeXe distinguishes ordinary members from *experts*, who are recognized as such by the DAO through a proposal and a vote. Once approved, the expert status is represented through a non-transferable NFT.

The protocol also uses a weighted and non-linear voting model, in which voting power does not simply increase proportionally to the number of tokens and experts can be treated differently from ordinary members.

In this case, however, a participant's expertise is recognized internally by the DAO. In the model proposed in this thesis, instead, a change in the competence level of a participant does not require a governance proposal but relies on a verification process. The competences are certified by external Issuers considered trustworthy and are represented through VC.

VCs have also been used in voting systems based on Self-Sovereign Identity. Castellano, De Prisco, and Faruolo[37] propose an e-voting system based on SSI and the Algorand blockchain, in which credentials are used to certify that a subject has the right to participate in the voting process.

In this case, the credential mainly determines the *eligibility* of the voter. In the model developed in this thesis, instead, a VC certifies specific competences of the member, and this information is used to contribute to the calculation of their voting power.

A work that is particularly close to the model developed in this thesis is the one by Keršič et al.[38], which integrates Self-Sovereign Identity and VCs into DAO governance. The authors use DIDs and VCs to represent the competences and experience of participants and propose a prototype based on Veramo and MetaMask Snaps, integrated with the Snapshot voting system. The credentials are used as a

requirement to participate in specific voting processes: for example, a VC can certify a specific competence and allow its Holder to vote on proposals that require that competence.

In the model proposed in this thesis, instead, the VC is not used only to determine whether a member can participate in the voting process. The certified competences directly contribute to the calculation of voting power, and their weight depends on the topic associated with the proposal. This component is also combined with the economic stake of the participant.

The analyzed works therefore show different approaches for integrating competence and reputation into DAO governance: assigning greater voting weight to participants with more experience, using reputation acquired through activity within the DAO, recognizing expert status through the community, and using verifiable credentials to determine voting eligibility.

The model proposed in this thesis instead combines an economic component, derived from stake, with a component based on competences certified through VCs. The competences are considered according to the topic associated with the proposal and directly contribute to the calculation of voting power.

The VCs are issued by trusted Issuers and are verified directly within the governance system. The credential is not used to determine whether a member can participate in the voting process, but instead contributes quantitatively to the weight of their vote.

Approach	**Competence-weighted VP**	**Competence-based eligibility**	**Reputation-based VP**	**Topic-dependent**	**VC-based verification**
Token-based governance [29] [30] [31] [32] [33]	–	–	–	–	–
Weighted / Knowledge-Extractable Voting [34]	✓	–	*Partial*	✓	–
Reputation-based governance [35]	–	–	✓	*Proposed*	–
DeXe Protocol [36]	✓	–	–	–	–
SSI-based e-voting [37]	–	✓	–	–	✓
SSI-based DAO governance [38]	–	✓	–	✓	✓
Proposed model	✓	–	–	✓	✓

Table 3.1: Comparison of the governance approaches analyzed in the related work according to competence-based VP and eligibility, reputation VP, topic dependency and the use of VC.

Chapter 4

Design of the Governance Model

This chapter first describes the design of the voting power model, including the separation between stake-based and skill-based voting power, the use of topic-dependent competences, and the formula used to combine the two components. It then introduces the main actors and components of the system and explains how the system operates through five main protocols.

4.1 Problem

The classical approach used in DAOs, in which the voting power of a member is proportional to the amount of money deposited, has an important limitation: owning more tokens does not necessarily mean having greater competences related to the topic being voted on. An incompetent member who has deposited a very large amount of money can centralize voting power and influence decisions, voting for and approving proposals that go against the interests of the organization.

We propose a DAO model called **Self-sovereign Attestations for Governance with Expertise** (SAGE), in which voting power is based both on the amount of money deposited by a member and on their competences. These competences are verified through VCs issued by trusted Issuers. If a member is highly competent in a specific domain, they obtain greater voting power.

As a concrete example, we consider an investment DAO in which participants

collectively decide how to allocate the DAO's funds by voting on investments in startup addresses. However, the mechanisms introduced by SAGE, including competence-based voting power and the verification of competences through VCs, are general and can be applied to different types of DAOs.

4.2 Design of Expertise-Based Voting Power

The proposed model defines voting power based on both the amount of stake deposited by a member and their competences. SAGE defines the skills considered relevant to the governance process and uses them to determine the competence score of each member.

To determine the voting power of a member at the time of voting, a weighted formula is used that takes into account the amount of stake deposited, the weight assigned to the stake component, the competence score derived from the member's skills, and the weight assigned to the competence component.

The weights of the stake-based and skill-based components are defined immutably when the DAO is created and are expressed in basis points. Their values determine the relative contribution of the two components to the voting power of each member.

4.2.1 Issues in Using Tokens to Represent Composite Voting Power

To represent the voting power of a member, a possible approach would be to use a single governance token for both the stake-based and the skill-based components. Under this approach, tokens could be minted at two different moments: when a member deposits ETH, to represent the economic component, and when their competences are verified, to represent the skill component. However, this approach introduces an important problem.

Tokens representing voting power derived from `stake` can be transferable, while voting power derived from `skills` should remain associated with the member whose competences have been certified and therefore should not be transferable.

To solve this problem, the governance token is used only to represent the stake-based component. The total voting power of a member is not represented only by the token, but by the combination of their governance tokens, which represent the stake component, and the skill component, which is obtained separately from a state stored inside the contract.

To achieve this, the smart contract maintains additional `checkpoints`. A checkpoint stores the value of a variable at a specific timepoint, making it possible to retrieve its historical value later. In this case, at each competence upgrade, checkpoints store both the skill component of a user's voting power and the total skill-based voting power of all users at that timepoint.

When a proposal enters the voting phase, the governance system uses the proposal snapshot timepoint, determined at the beginning of voting, to retrieve the historical values stored in the checkpoints.

The voting power of a user is therefore derived from the weighted sum of the tokens owned by that user at the snapshot timepoint and the skill level stored in the corresponding checkpoint.

The `total voting power supply` is therefore calculated as the sum of the `total supply` of the governance tokens related to stake and the total skill-based voting power of all users, both considered at the snapshot timepoint.

4.2.2 Design of Topic-Dependent Voting Power

Our goal is to design a DAO in which a user's skills provide different voting power depending on the topic of the vote.

A **topic** represents the domain to which a proposal belongs and determines which competences are relevant to that specific decision.

For example, in an investment DAO, proposals can concern startups operating in different domains, such as AI & Data, Cloud & Cybersecurity, FinTech & Blockchain, or Enterprise Software. Each proposal can therefore be associated with one of these topics, and the same skill can contribute differently to the voting power depending on the topic associated with the proposal.

Both approaches analyzed above, the approach that uses a single token to represent voting power and the approach based on the combination of tokens and

checkpoints, are not sufficient to support topic-based governance.

To make the skill-based component topic-dependent, two checkpoints are required for each topic: one that keeps track, for each user, of the skill value for that topic, and one that keeps track of the sum of the voting power derived from skills of all users for that topic.

4.2.3 Formula for Calculating a Member's Voting Power

To calculate the voting power of a DAO member at a given timepoint, the following formula is used:

$$\begin{aligned}\textbf{VotingPower}(account, topic, timepoint) = \textbf{VotingPowerStake}(account, timepoint) \\ + \textbf{VotingPowerSkill}(account, topic, \\ timepoint)\end{aligned} \tag{4.1}$$

The economic component uses the following formula:

$$\begin{aligned}\textbf{VotingPowerStake}(account, timepoint) = \textbf{weightStake} \\ \times \textbf{scoreStake}(account, timepoint)\end{aligned} \tag{4.2}$$

$$\textbf{scoreStake}(account, timepoint) = \min\left(\frac{stakeDeposited}{MAX_DEPOSIT}, 1\right) \times 100 \in [0, 100] \tag{4.3}$$

A maximum limit for the deposited stake is defined. The score derived from stake has a range from 0 to 100.

If more than the maximum allowed amount is deposited, the score takes the value 100; otherwise, its value is the ratio between the deposited stake and the maximum allowed deposit, multiplied by 100.

The resulting score is multiplied by a coefficient that determines, as a percentage, the weight of the voting power derived from stake in the total voting power.

The use of a maximum deposit limit in the formula makes it possible to limit the influence of the economic component on a member's voting power. Once `MAX_DEPOSIT` is reached, additional deposits do not produce any increase in the voting power derived from stake.

The skill component uses the following formula:

$$\textbf{VotingPowerSkill}(account, topic, timepoint) = \textbf{weightSkill} \times \textbf{scoreSkills}(account, topic, timepoint) \tag{4.4}$$

The score is calculated based on the value of the skill possessed by the member with respect to the topic of the vote.

To obtain the voting power derived from skills, the score is then multiplied by a coefficient, called `weightSkill`, that determines, as a percentage, the weight of the voting power derived from skills in the total voting power.

The sum of `weightSkill` and `weightStake` must be equal to 100%. These parameters can be customized when the DAO is deployed. For example, the following combinations can be used: 100% stake / 0% skill, 75% / 25%, 50% / 50%, and 25% / 75%.

4.2.4 Representation of Member Skills

SAGE uses a **bitmap** to represent the skills possessed by DAO members, instead of a mapping-based structure, with the goal of reducing the cost of storing them on-chain.

A traditional solution would be to store each skill possessed by a member separately within an array. However, this approach would require a larger number of storage entries and, consequently, a greater amount of data to be maintained on-chain.

The bitmap solution instead makes it possible to represent all the skills of a member with a single integer. Each skill is assigned a fixed position corresponding to a bit. The presence of a skill is represented by setting the corresponding bit to 1, while its absence is represented by 0.

For example, considering the set of skills `machineLearning`, `dataEngineering`, `cyberSecurity`, `cloudArchitecture`, `distributedSystems`, `blockchain`, `softwareArchitecture`, and `startupFinance`, occupying bits 0 to 7 respectively, a bitmap with the binary value `00101000` represents the skills `cloud-`

`Architecture` and `blockchain`.

The main advantage of using a bitmap is the reduction in storage operations and, consequently, in the gas required to update skills on-chain. The trade-off is the need to assign a stable position to each skill in the bitmap. The skills must be defined by the system in advance, and once a bit has been associated with a skill, that position must not be changed.

4.3 SAGE Actors and Components

4.3.1 SAGE Actors

SAGE involves several actors, each with a specific role in the operation of the DAO. The main actors are the DAO members, the Issuers, the startups, the external accounts, and the deployer.

- The **DAO members** are the users who participate in the governance process. They join the DAO by depositing an amount of ETH and can increase their voting power either by increasing their deposited stake or by presenting a credential that certifies their competences. The voting power obtained in this way is used to participate in the voting process of DAO proposals. DAO members can also act as proposers: a member can submit a proposal and assign it one of the topics defined by the DAO.

- The **Issuers** are external entities recognized as trusted by the DAO and authorized to certify the competences of members. They issue digitally signed VCs that contain information about their holders and the competences attributed to them.

- The **startups** represent the potential recipients of investments. Before receiving funds, they must be recognized by the DAO as valid and active destinations. Once an investment has been approved, the startup receives the amount defined in the proposal.

- The **external accounts** are Ethereum accounts that are not members of the DAO. They can interact with the publicly accessible functions of the system,

but not with those reserved for DAO members.

- The **deployer** is the actor responsible for initializing the system and configuring the dependencies required for the correct operation of the DAO. Once the initial setup has been completed, the deployer no longer directly manages the relevant operations of the system.

4.3.2 SAGE Components

The actors described above interact with the main system components, namely the DAO governance, the Treasury, and the Timelock Controller.

- The **DAO governance** is the component that represents the decentralized organization implemented through smart contracts. It coordinates the DAO decision-making process, manages the creation, voting, and approval of proposals, and authorizes the execution of operations such as investments, startup registration, and changes to governable parameters.
- The **Treasury** is the component that stores the funds of the DAO. It receives the ETH deposited by members and can use these funds only after a proposal has been approved through the governance process.
- The **Timelock Controller** is the component that acts as the executive authority of the DAO and is authorized to execute the operations approved by the governance on the components controlled by the system, such as the Treasury and the other governable modules. It also provides a **delay** mechanism, imposing a time interval between the approval of a proposal and its actual execution.

4.4 SAGE Protocol Architecture

The protocols described in this section define the main operational flows of SAGE. More specifically, the system can be described at a high level through five main protocols: *DAO Membership Protocol*, *Stake Protocol*, *Competence Certification Protocol*, *Governance Protocol*, and *Investment Execution Protocol*.

- The **DAO Membership Protocol** defines the process through which an external account acquires the status of DAO member. Joining the DAO requires an initial ETH deposit, which allows the user to access the functionalities reserved for members.

- The **Stake Protocol** defines the management of the economic component associated with DAO members. The deposited ETH are recorded and transferred to the Treasury, while the member receives an amount of stake tokens determined by the capital deposited.

- The **Competence Certification Protocol** defines the process through which a member proves their competences. The member associates their DID with the account used in the DAO and presents a VC issued and signed by a recognized Issuer. The DAO verifies the credential and, if it is valid, records the certified competences of the member.

- The **Governance Protocol** defines the process through which the DAO makes its decisions and makes them executable. It includes proposal creation, snapshot definition, the voting phase, the determination of the member's voting power at voting time, the verification of the conditions required for approval, and the subsequent execution through the Timelock Controller.

- The **Investment Execution Protocol** defines the process through which an investment decision approved by the governance is carried out. The protocol manages the transfer of funds from the Treasury to the recipient startup, verifying that the startup is recognized by the system as valid and active.

Together, these protocols describe the main operation of SAGE: a user joins the DAO, can certify their competences, obtains voting power derived from stake and competences, participates in the decision-making process, and, in the case of an approved investment, the funds are transferred from the Treasury to the recipient startup.

4.4.1 DAO Membership Protocol

The **DAO Membership Protocol** defines the process through which an external account acquires DAO membership status. Joining the DAO is permissionless and does not require approval through the governance process. To join the DAO, a user makes an initial ETH deposit. The governance first checks that the account is not already registered as a member. If the check succeeds, the account is registered as a member and receives governance tokens, gaining access to the functionalities reserved for DAO participants, as shown in Figure 4.1.

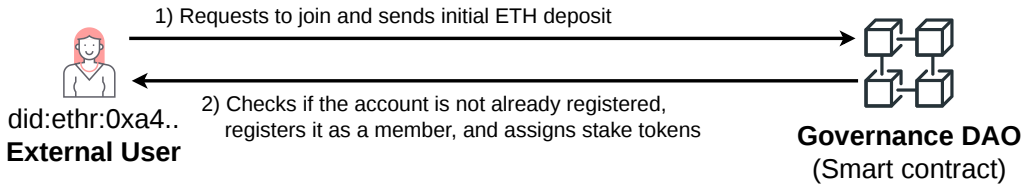


Figure 4.1: DAO Membership Protocol: An external user requests to join the DAO, while the governance verifies the account and registers the new member.

Once acquired, membership is also used by the system to restrict certain operations to DAO members only. In particular, governance tokens can be transferred or delegated exclusively to other members, preventing external accounts from acquiring voting rights without first obtaining membership status.

4.4.2 Stake Protocol

The **Stake Protocol** defines the management of the economic component of the voting power of DAO members, which is represented by a governance token. When a member deposits ETH, the amount is recorded and used to determine the number of governance tokens to assign to the member. The deposited amount is transferred to the DAO Treasury. The main steps of this process are illustrated in Figure 4.2.

After joining the DAO, a member can increase their stake by making additional ETH deposits. In this case, the system updates the total amount deposited and assigns new governance tokens according to the increase in stake.

The system defines two distinct limits: one related to the maximum amount of capital that each member can deposit and one related to the maximum amount of governance tokens that can be obtained through stake.

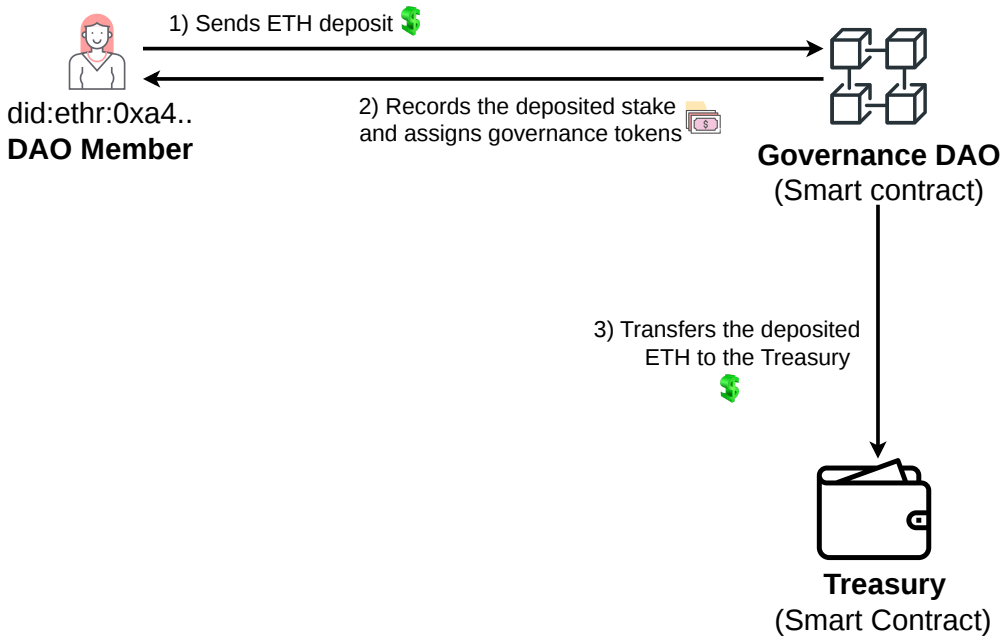


Figure 4.2: Stake Protocol: A DAO member deposits ETH, receives governance tokens, and transfers the deposited funds to the Treasury.

4.4.3 Competence Certification Protocol

The **Competence Certification Protocol** defines the process through which a DAO member proves their competences by verifying a VC.

In the considered model, an Issuer recognized as trusted by the DAO issues a VC to the member, who acts as the Holder of the credential.

The credential contains the identifier of the Issuer and the information related to the Holder, including the certified competences. The credential is digitally signed by the Issuer.

When the member wants to present these competences to the DAO, they associate their DID with the Ethereum account used to participate in the governance process and present the VC to the DAO, which acts as the Verifier.

The DAO verifies that the credential refers to the member presenting it, that it was issued by an Issuer recognized as trusted, and that it has not been modified.

If all the verification checks are successful, the certified competences contained in the VC are registered among the member's competences. The system then uses the updated set of competences to determine the component of voting power derived from competences. The main steps of the competence certification process are illustrated in Figure 4.3.

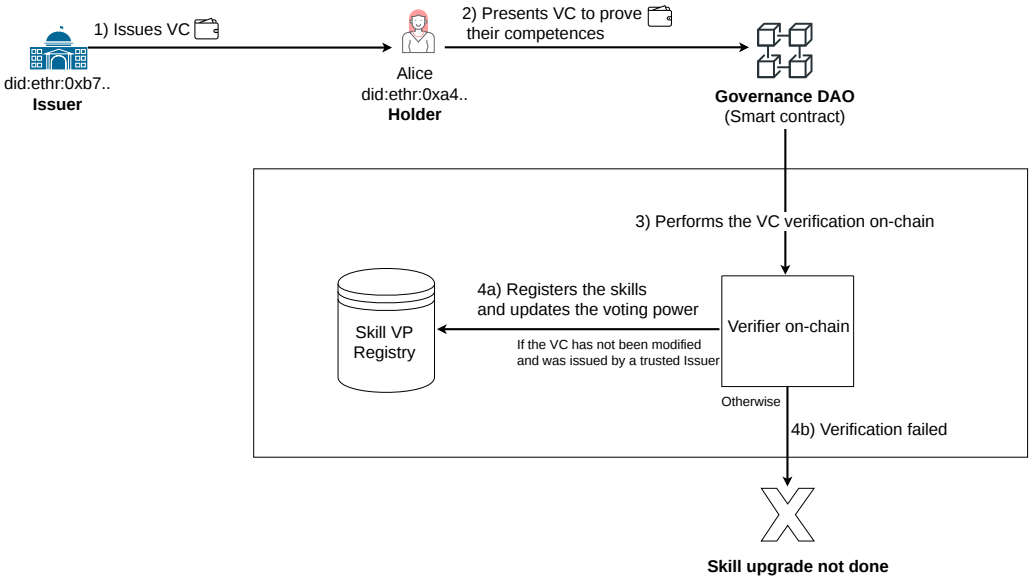


Figure 4.3: Competence Certification Protocol: A DAO member presents a VC issued by a trusted Issuer. The DAO verifies it on-chain and, if valid, updates the skill-based voting power.

4.4.4 Governance Protocol

The **Governance Protocol** defines the entire process through which the DAO makes a decision and makes its execution possible.

The process begins with the creation of a proposal, to which a topic identifying the domain of the decision is associated. Any member can create a proposal, without limits related to the amount of stake deposited or to the competences they possess. Once created, the proposal remains pending for the period defined by a `voting delay` and then enters the voting phase.

At the beginning of the voting phase, the `snapshot` of the proposal is defined and used as the reference for determining the voting power of the members throughout the entire voting process. For each member, the voting power is calculated by combining the component derived from stake with the component derived from competences in relation to the topic of the proposal. The use of the snapshot guarantees that any ETH deposits or competence updates performed later do not modify the voting power used for a vote that has already started.

During `voting period`, members can cast a vote in favor, against, or abstain. To be approved, a proposal must satisfy the conditions defined by the governance, including reaching the required `quorum` and majority.

The quorum is determined with respect to the total voting power available for the topic of the proposal at the snapshot. The system also includes a `superquorum`: if, during the voting period, a sufficiently high threshold of votes in favor is reached and these votes exceed the votes against, the proposal can be approved before the natural end of the voting period.

For example, the DAO can be configured with a quorum of 20%, meaning that for a proposal to be valid, at least 20% of the `totalSupply` must participate in the vote, and a superquorum of 70%, meaning that the proposal is approved immediately if 70% of votes in favor is reached, without waiting for the end of the voting period.

These values are therefore not fixed by the architecture and can be parametrically configured at deployment according to the governance requirements of the DAO.

An approved proposal is then added to the queue of the `Timelock Controller`. The operation remains pending for the `Timelock delay` and can be executed only after this interval has elapsed. If, instead, the proposal does not satisfy the required conditions, it is considered rejected. The main steps of the governance process are illustrated in Figure 4.4.

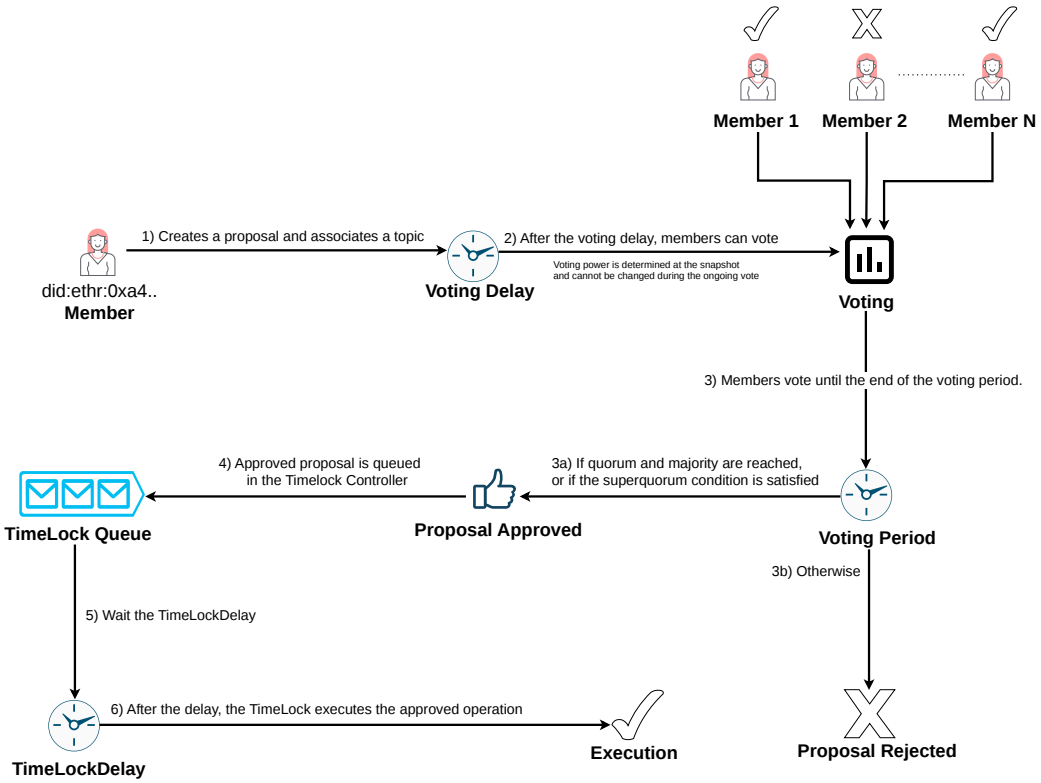


Figure 4.4: Governance Protocol: A DAO member creates a proposal, members vote, and, if approved, the proposal is queued and then executed after the Timelock delay.

4.4.5 Investment Execution Protocol

The **Investment Execution Protocol** defines the process through which an investment decision approved by the governance is actually carried out. The system maintains a registry containing the startups that can receive investments, their corresponding addresses, and their activity status. When the operation executed by the Timelock Controller concerns an investment, the Treasury verifies that the recipient address belongs to a startup registered in the system and that the startup is active. It also verifies that the funds required to execute the operation are available. If these conditions are satisfied, the Treasury transfers the amount defined in the proposal to the startup and records the investment made. The main steps of the investment execution process are illustrated in Figure 4.5. In this way, DAO funds

can be transferred only to recipients recognized by the system and only following a decision approved through the governance process.

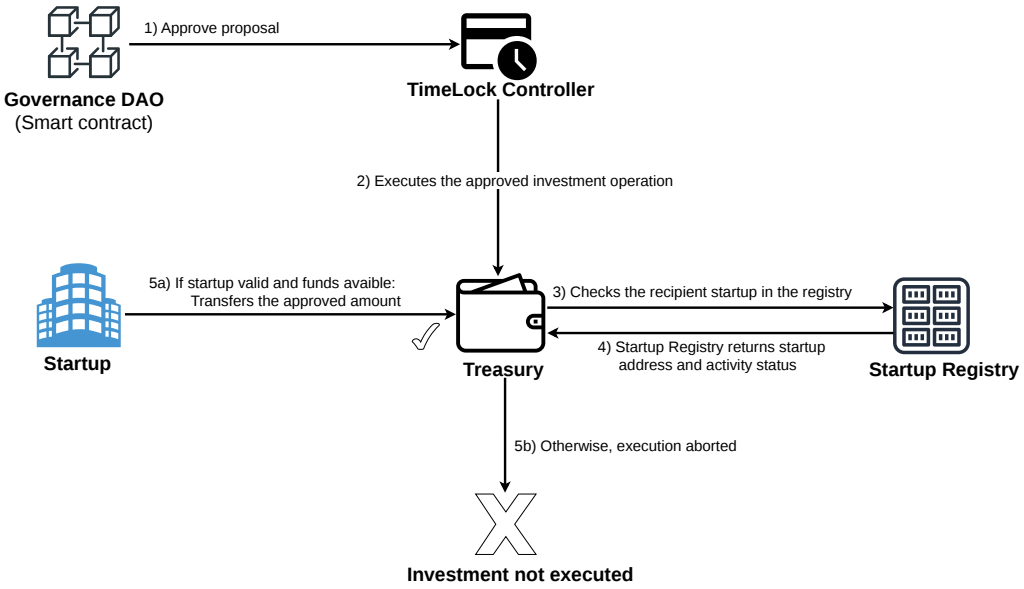


Figure 4.5: Investment Execution Protocol: The Treasury executes an approved investment after verifying the startup's eligibility and the availability of funds.

Chapter 5

DAO Governance and SSI Management Tools

This chapter presents the main libraries and technologies used to implement the system. In particular, it describes `OpenZeppelin` and the modules used to build the DAO governance, including `Governor`, `ERC20Votes`, `quorum`, `superquorum`, and `TimelockController`. It then introduces `Veramo`, used for the management of DIDs and VCs, and `Hardhat`, used to compile, test, and deploy the smart contracts.

5.1 OpenZeppelin Contracts and Governance

OpenZeppelin Contracts[39] is a library of smart contracts written in Solidity that provides modular and reusable components for implementing common functionalities in Ethereum applications, such as tokens, governance systems, and access control mechanisms.

Using these components makes it possible to develop new smart contracts starting from existing implementations, avoiding the need to rewrite common functionalities from scratch.

This approach is mainly based on Solidity inheritance: a new contract can inherit one or more OpenZeppelin modules, reusing their functionalities and, when necessary, extending or modifying the behavior of some functions through over-

riding.

To implement a governance system, OpenZeppelin uses a modular architecture composed of a main contract, called `Governor`, and a series of extensions that add specific functionalities, as shown in Figure 5.1.

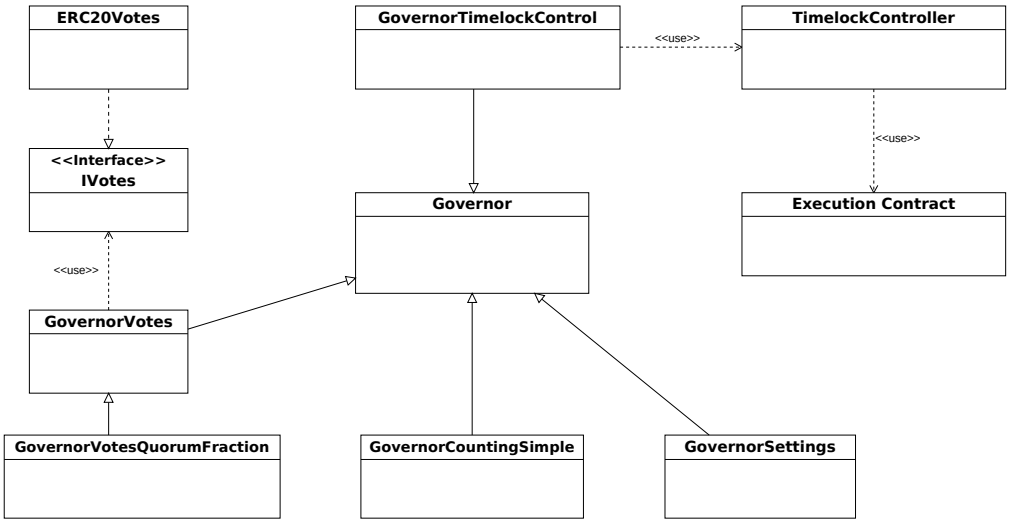


Figure 5.1: Class diagram of the OpenZeppelin Governor architecture and its main extensions.

The `Governor`[40] represents the central component of the system and manages the lifecycle of a proposal: its creation, the opening of the voting process, the recording of votes, the determination of the result, and the possible execution of the approved decision.

The different governance rules are then defined through additional modules. `GovernorVotes` determines the source from which the voting power of participants is obtained, for example from an `ERC20Votes` token.

`GovernorCountingSimple` defines how votes are expressed and counted, distinguishing between votes in favor, votes against, and abstentions. `GovernorSettings` allows aspects such as the time that must pass before the beginning of a voting process, its duration, and the minimum voting power required to create a proposal to be configured. `GovernorVotesQuorumFraction`, instead, defines the minimum amount of participation required for a vote to be considered valid, calculating it as a fraction of the total voting power.

The execution of proposals can finally be assigned to a `TimelockController`, connected to the `Governor` through `GovernorTimelockControl`. In this case, an approved proposal is not executed immediately, but is first queued and can only be executed after a predefined amount of time has passed.

5.1.1 Governor

The `Governor` contract provides the primitives needed to manage the lifecycle of proposals. A proposal is created through the `propose()` function.

Each proposal can contain one or more operations that must be executed if the proposal is approved. For each operation, the input specifies a target, that is, the address of the destination contract, an optional amount of ETH to send to that contract, and the calldata representing the function to execute with its parameters. A textual description is also associated with the proposal. When the `propose()` function is called, the `Governor` creates a new proposal containing the previously specified actions to be executed.

As an example, a gaming DAO proposes doubling the reward for winning a match from 100 to 200 tokens. The proposer calls the `propose()` function, specifying the game contract's address and the encoded call to `setWinnerReward(200)`, with no ETH attached to the proposed call. The reward changes only when the approved proposal is executed.

The `Governor` also determines the snapshot timepoint, that is, the reference point at which the voting power of participants is fixed and the quorum of the proposal is calculated. The snapshot is determined after the period defined by `votingDelay`, when the voting process begins. The `votingPeriod` defines how long the proposal remains open for voting. The deadline therefore indicates the point at which the voting period ends.

The creation of the proposal also generates the `ProposalCreated` event and produces a unique identifier, called `proposalId`, which is used to identify the proposal during all subsequent phases of the governance process.

The `Governor` provides a `state(uint256 proposalId)` function that returns the current state of a proposal identified by its `proposalId`. A proposal can be in

different states, defined by the `ProposalState` enum, including `Pending`, when the proposal has been created but voting has not yet started; `Active`, when voting is currently in progress; `Succeeded`, when voting has ended and the proposal has satisfied the conditions required for approval; `Defeated`, when the proposal has not reached the quorum or has not received the votes required for approval; `Queued`, when the approved proposal has been added to the queue for later execution; `Executed`, when the operations defined by the proposal have been executed; and `Canceled`, when the proposal has been canceled before its execution.

The `proposalSnapshot(uint256 proposalId)` function returns the reference timepoint of the proposal, that is, the point used by the governance system to determine the voting power of participants. In fact, the voting power of a participant is not calculated at the time of voting, but at the proposal snapshot. In this way, a participant cannot acquire additional voting power after the snapshot in order to influence the voting process.
The `proposalDeadline(uint256 proposalId)` function returns the closing timepoint of the voting period for the proposal. The `castVote(uint256 proposalId, uint8 choice)` function allows a participant to cast a vote on an active proposal, taking as input the proposal ID and the voting choice, which can be `Against`, `For`, or `Abstain`.

When the function is called, the `Governor` checks that the proposal is in the voting phase and that the account has not already voted on the same proposal.

It then retrieves the voting power of the account at the proposal snapshot timepoint and passes the voting power and the selected choice to the counting mechanism. The vote is then recorded, and its corresponding weight is added to the count of the selected choice.

5.1.2 ERC20Votes and GovernorVotes

OpenZeppelin provides the `ERC20Votes`[41] module, an extension of ERC-20 designed to support systems in which the voting power of governance members derives from a governance token.

Unlike a standard ERC-20 token, which mainly allows the current balance of an account to be known, `ERC20Votes` maintains information about voting power

over time, making it possible to retrieve the voting power associated with a specific point in the past.

Token ownership represents the available voting units, while the effective voting power is assigned through a delegation mechanism, represented by the `delegate(address delegatee)` function. An account can delegate its voting power to another address or delegate it to itself through self-delegation, in order to participate directly in governance or assign its voting power to another member.

To store the history of voting power, `ERC20Votes` uses `checkpoints`. Each time the voting power of an account changes, for example after a delegation or a change in the number of tokens that contribute to voting, a new checkpoint is recorded containing the new voting power and the corresponding timepoint.

It is therefore possible to retrieve both the current voting power, through `getVotes(address account)`, and the voting power held at a previous point in time through `getPastVotes(address account, uint256 blockNumber)`.

Similarly, `getPastTotalSupply(uint256 blockNumber)` makes it possible to retrieve the total amount of voting units available to all members at a specific timepoint.

In a typical use case, DAO members receive `ERC20Votes` tokens proportionally to the stake they deposit. They then delegate their votes so that checkpoints related to voting power are created. When a proposal enters the voting phase, the governance contract uses `getPastVotes()` to retrieve the voting power held by each account at the snapshot timepoint of the proposal, preventing later token transfers from affecting the weight of the vote.

The described functions are defined by the `IVotes` interface, which specifies a common set of operations that a governance contract can use to obtain information about votes.

The `GovernorVotes` module connects the `Governor` contract to a contract that implements the `IVotes` interface, such as `ERC20Votes`, allowing the governance system to retrieve the voting power of participants during a vote.

The `GovernorVotes` constructor receives as a parameter the contract implementing `IVotes` that must be used as the source of voting power.

`GovernorVotes` then stores a reference to this contract, allowing the `Governor` to retrieve the voting power of participants from it. The `getVotes(address account, uint256 timepoint)` function returns the voting power held by a specific account at a specific timepoint by reading the data from the token that implements `IVotes` and internally calling the `getPastVotes()` function.

When `GovernorVotes` is connected to an `ERC20Votes` token, the voting power is obtained from the historical checkpoints maintained by the token.

5.1.3 Governor Configuration and Extensions

`GovernorSettings` is an extension of `Governor` used to configure some fundamental parameters of the governance process, including the `votingDelay`, that is, the period of time that must pass between the creation of a proposal and the beginning of voting; the `votingPeriod`, that is, the duration of the period during which participants can vote; and the `proposalThreshold`, that is, the minimum voting power that an account must have in order to create a new proposal.

`GovernorSettings` also allows these values to be modified through the governance process itself, so that the DAO can update the configuration of its voting system without having to replace the `Governor` contract.

For quorum management, the `GovernorVotesQuorumFraction` module is provided, which defines the `quorum` as a fraction of the total supply of voting units at a specific timepoint. For a proposal to be valid, it must reach at least the required quorum of votes. For example, if the quorum numerator is equal to 10 and the denominator is 100, the minimum number of votes required corresponds to 10% of the total supply considered at the snapshot timepoint. The `quorum(uint256 timepoint)` function calculates the minimum number of votes required to reach the quorum at a specific timepoint.

OpenZeppelin also provides `GovernorVotesSuperQuorumFraction`, an extension that introduces the superquorum, that is, a higher threshold of votes in favor that can allow the early approval of a proposal. The superquorum is calculated as a fraction of the total supply of voting units at a specific timepoint.

Unlike the ordinary quorum, only `For` votes are counted to reach the superquo-

rum. If, during the voting period, the votes in favor reach this threshold and the proposal also has a majority of votes in favor, it can move to the `Succeeded` state before the normal deadline, without waiting for the end of the `votingPeriod`.

5.1.4 TimelockController and GovernorTimelockControl

The execution of decisions can be controlled through `TimelockController`, a smart contract that imposes a minimum time delay between the scheduling of an operation resulting from an approved governance proposal and its actual execution on the blockchain. This delay is intended to prevent a governance decision from producing its effects immediately, giving participants time to observe the scheduled operation and react before it is applied.

`TimelockController` works as follows. An operation is initially scheduled and associated with a specific timestamp. As long as the minimum delay has not passed, it remains pending, while once this interval has passed, it becomes executable. The contract also uses a role-based access control system. The `PROPOSER_ROLE` allows operations to be scheduled, the `EXECUTOR_ROLE` allows them to be executed when they become available, and the `CANCELLER_ROLE` allows pending operations to be canceled.

The `schedule()` function adds an operation to the queue to be executed after a certain minimum delay, the `execute()` function executes an operation that has already been scheduled and is ready to be executed once the delay has passed, while the `cancel(id)` function removes an operation identified by `id` from the queue before it is executed.

The connection between `Governor` and `TimelockController` is implemented through `GovernorTimelockControl`, which requires a proposal that successfully passes the voting process not to be executed directly by the `Governor`, but to be added to the `TimelockController` queue so that it can only be executed after the required delay has passed.

The `queue()` function takes a proposal approved by the vote and sends it to the timelock to be scheduled. The `execute()` function takes a proposal that has been scheduled in the timelock and, after the delay has passed, asks the time-

lock to execute the scheduled actions. The `cancel()` function cancels a proposal previously scheduled in the timelock, if it has not yet been executed.

5.2 Veramo

Veramo[42] is a modular framework used to develop applications based on Verifiable Data and Self-Sovereign Identity. It provides tools to create and manage Decentralized Identifiers, cryptographic keys, Verifiable Credentials, and Verifiable Presentations, providing the main components needed to implement the roles of Issuer, Holder, and Verifier.

The central component of the framework is the `Veramo Agent`, a configurable object that coordinates the functionalities of the application. The Agent is built through a plugin-based architecture, where each plugin adds a specific functionality. In this way, it is possible to configure the Agent by including only the components needed for the system to be developed.

Among the main components is the `Key Manager`, which is responsible for managing cryptographic keys and the signing operations required by the other modules. The `DID Manager`, instead, allows DIDs to be created and managed using different DID providers. The `DID Resolver` is responsible for resolving a DID and retrieving the associated information, such as the corresponding `DID Document` and the verification methods needed to identify the public keys associated with the subject.

For credential management, Veramo provides the `Credential Plugin`, which offers the operations needed to create and verify VCs and VPs. An Issuer can use the Veramo Agent to create a VC containing the claims related to the Credential Subject and protect it cryptographically using the keys managed by the Agent. The credential can later be presented to a Verifier, which can use Veramo to verify its origin, integrity, and cryptographic validity.

5.3 Hardhat

Hardhat[43] is a development environment used to create, test, and deploy Ethereum smart contracts. One of its main functionalities is smart contract compilation. Code written in Solidity is compiled into bytecode, which can then be executed by the Ethereum Virtual Machine. `Hardhat` also allows automated tests to be executed, which are useful for verifying that contract functions produce the expected behavior before deployment on a real network. `Hardhat` also provides a simulated local Ethereum network, which allows smart contracts and transactions to be executed in a controlled environment. In this way, it is possible to test the behavior of the system without using a public blockchain and without spending real ETH. During local testing, `Hardhat` also provides useful debugging information, such as error messages related to failed transactions.

`Hardhat` also supports the deployment of smart contracts, that is, their publication on an Ethereum network. For this phase, `Hardhat Ignition` can be used, allowing the deployment process of the different contracts to be defined and managed in a structured way.

Chapter 6

Implementation

This chapter describes how the proposed governance model is concretely implemented. It first introduces the main system versions and presents the project structure and the role of the main smart contracts.

Following the protocol architecture defined in Chapter 4, the implementation is then presented according to the five main protocols of SAGE: the DAO Membership Protocol, the Stake Protocol, the Competence Certification Protocol, the Governance Protocol, and the Investment Execution Protocol.

6.1 Implementation Versions

During the development of the proposed architecture, different versions of the system were implemented, each representing the governance implementation with the core features introduced at that stage. Four main versions of the system were created:

- **V0 – No Topic With Unique Token:** The initial V0 version uses a single `ERC20Votes` governance token to represent the voting power, both the part derived from stake and the part derived from skills. In addition, it does not use the topic mechanism.
- **V1 – No Topic With Override:** The V1 version separates the two components of voting power. The part derived from stake is represented by

the `ERC20Votes` token, while the part derived from skills is stored through checkpoints. The two components are added at voting time through the override of the `_getVotes()` function.

- **V2 – Topic-Based DAO without Bitmap:** The V2 version introduces the topic mechanism, in which the same skill can have a different value depending on the topic of the proposal.
- **V3 – Topic-Based DAO with Bitmap:** The V3 version optimizes the storage of the skills possessed by a user through the use of a bitmap.

The rest of this chapter describes the implementation of the V3 version, which represents the final version of the proposed system.

6.2 Project Structure

The project uses Solidity and TypeScript, together with the Hardhat and Veramo frameworks, and the APIs provided by the OpenZeppelin library. The complete source code of the project is publicly available on GitHub[44].

The governance system is composed of several smart contracts and libraries, each responsible for a specific part of the architecture, as illustrated in Figure 6.1. These components cooperate to implement the five protocols defined in Section 4.4.

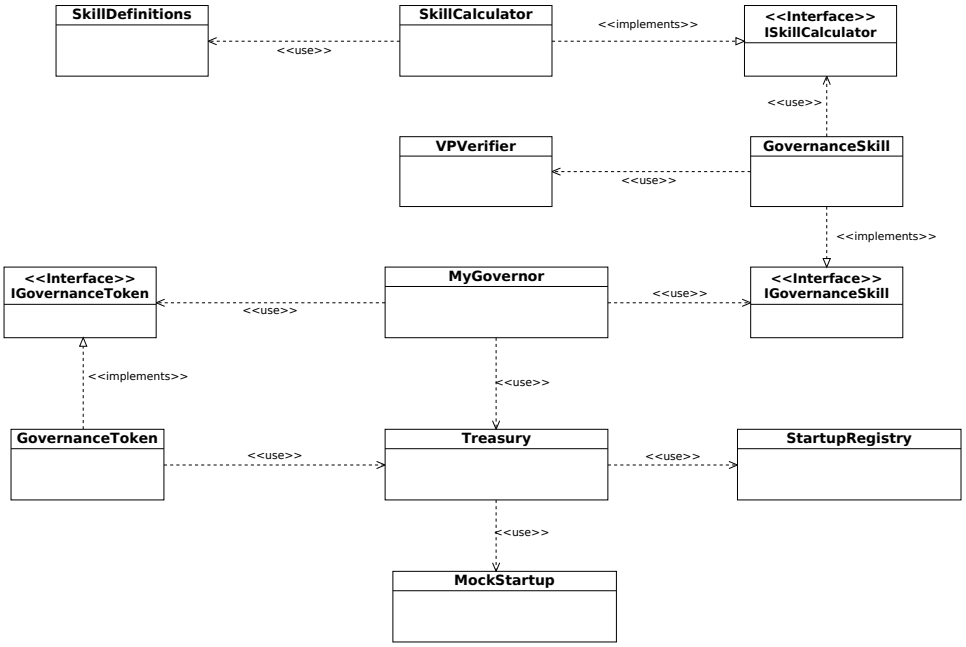


Figure 6.1: Class diagram of the main smart contracts and interfaces in the SAGE implementation.

The main modules, their respective responsibilities, and their relation to the SAGE protocols are summarized below:

- **MyGovernor.sol** is the contract that implements the *Governance Protocol* described in Subsection 4.4.4. It inherits from the OpenZeppelin `Governor` library and some of its extensions and manages the entire lifecycle of proposals, from their creation and voting to the verification of the approval conditions and their subsequent execution. The contract calculates the voting power of members at the snapshot by combining the component derived from stake with the component derived from the skills related to the topic of the proposal. It also manages the calculation of the total voting power supply for a given topic, as well as the quorum and superquorum.

- **IGovernanceToken.sol** defines the interface used by the other system com-

ponents to access the functionalities related to membership and to the component of voting power derived from stake. `GovernanceToken.sol` implements the `ERC20Votes` token that represents this economic component and mainly implements the *DAO Membership Protocol*, described in Subsection 4.4.1, and the *Stake Protocol*, described in Subsection 4.4.2. It manages the entry of members into the DAO, the restrictions on token transfers and delegation toward non-member accounts, the recording of deposited ETH, the assignment of governance tokens at the time of deposit, and the transfer of the deposited funds to the Treasury. The `ERC20Votes` checkpoints are also used by the *Governance Protocol* to retrieve the voting power derived from stake at the proposal snapshot.

- **IGovernanceSkill.sol** defines the interface used by the other system modules to access the functionalities related to the component of voting power derived from competences. `GovernanceSkill.sol` implements part of the on-chain functionalities of the *Competence Certification Protocol*, described in Subsection 4.4.3, managing the association between members and DIDs, the Issuers considered trusted, the registration of skills certified through VCs, and the update of the voting power derived from competences. This component is maintained through separate checkpoints for each topic, which are also used by the *Governance Protocol* to retrieve, at the snapshot, the historical voting power of the member and the total skill supply related to the topic of the proposal.

- The **VPVerifier.sol** library implements the on-chain cryptographic verification phase of the *Competence Certification Protocol*. In particular, it reconstructs the EIP-712 digest of the VC presented by the member and verifies its signature, recovering the address of the Issuer that signed it. The result of the verification is then used by `GovernanceSkill.sol` to check that the credential was issued by an Issuer recognized as trusted and that the VC has not been tampered with, before registering the certified competences.

- The part of the *Competence Certification Protocol* that translates competences into numerical scores is implemented in the **SkillCalculator.sol** con-

tract, which implements the `ISkillCalculator.sol` interface. The use of an interface makes it possible to replace the calculation formula in the future without modifying the rest of the system. The shared definitions of topics and competences recognized by the DAO are centralized in the `SkillDefinitions.sol` library, which can be accessed by the other contracts without additional storage costs.

- The *Investment Execution Protocol*, described in Subsection 4.4.5, is implemented by the **Treasury.sol**, **StartupRegistry.sol**, and **MockStartup.sol** modules. Fund management is handled by the `Treasury.sol` contract, a vault that can transfer ETH to external addresses only upon instruction from the governance system, ensuring that no single member can move the funds independently. The addresses eligible to receive investments are maintained in the `StartupRegistry.sol` contract, an on-chain registry that can only be modified through a collective decision. The `MockStartup.sol` contract provides a mock implementation used to simulate the behavior of a startup receiving funds.

6.3 DAO Membership Protocol

The *DAO Membership Protocol*, described in Subsection 4.4.1, is mainly implemented in the `GovernanceToken.sol` contract. DAO membership status is represented by the `isMember` mapping, which associates each address with a boolean value indicating whether the account is a member of the DAO.

```
1 mapping(address => bool) public isMember;
```

joinDAO(). The `joinDAO()` function implements the entry of a new account into the DAO. It can be called by any external account without requiring the approval of a proposal, making access to the DAO permissionless. The function can only be executed by an account that is not already registered as a member and requires an initial ETH deposit.

As shown in Code 6.3, the function first verifies that the Treasury has been configured, that the caller is not already a member, and that the deposited amount

is greater than zero. The caller's address is then registered as a member by setting the corresponding value in the `isMember` mapping. Once the new member has been registered, `joinDAO()` calls the `increaseStake()` function, which is responsible for minting governance tokens based on the deposit made and for transferring the deposited ETH to the Treasury.

```
function joinDAO() external payable {
    if (treasury == address(0)) revert TreasuryNotSet();
    if (isMember[msg.sender]) revert AlreadyMember();
    if (msg.value == 0) revert ZeroDeposit();
    isMember[msg.sender] = true;
    uint256 tokenAmount = increaseStake();
    emit MemberJoined(msg.sender, msg.value, tokenAmount);
}
```

The transfer and delegation of governance tokens are restricted to DAO members only. Through the override of the `_update` and `_delegate` methods of `ERC20Votes`, checks are introduced to ensure that the recipient of the tokens is actually a member of the DAO. Otherwise, respectively, the transfer or delegation is reverted, preventing a user who is not a member of the DAO from receiving tokens, acquiring voting power, and participating in the voting process.

6.4 Stake Protocol

The *Stake Protocol*, described in Subsection 4.4.2, is mainly implemented in the `GovernanceToken.sol` contract, which manages the economic component of voting power through an `ERC20Votes` token.

`GovernanceToken.sol` inherits from `ERC20` for the basic token functionalities and from `ERC20Votes` for the management of voting power and the related checkpoints. For stake management, the contract stores the deposit made by each member through the `stakeDeposited` mapping, which associates each address with the amount of ETH deposited, and defines the `MAX_DEPOSIT` constant as the maximum amount of ETH that can be deposited by a single member. The `weightStake` parameter, expressed in basis points, determines the weight of the economic component in the overall voting power.

```
mapping(address => uint256) public stakeDeposited;
uint256 public constant MAX_DEPOSIT = 100 ether;
uint256 public immutable weightStake;
```

_calculateStakeTokens(). The `_calculateStakeTokens()` function calculates the amount of governance tokens to mint when a new deposit is made, according to the formula of the stake component of voting power defined in Equation 4.2. Since a member can increase their stake multiple times, the function considers only the increase in the `stake score` with respect to the previous deposit.

For this purpose, the `stake score` is calculated by applying Equation 4.3 both to the previous deposit and to the new total deposit. The difference between the two values determines the increase in the score produced by the new deposit.

The number of new governance tokens is then calculated by applying `weightStake` to the increase in the score, according to Equation 4.2. In the contract, the result is represented with a precision of 10^{18}.

```
function _calculateStakeTokens(uint256 depositAmount, uint256
    previousDeposit) internal view returns (uint256) {
    uint256 oldEffective = previousDeposit > MAX_DEPOSIT ?
        MAX_DEPOSIT : previousDeposit;
    uint256 newTotal = previousDeposit + depositAmount;
    uint256 newEffective = newTotal > MAX_DEPOSIT ?
        MAX_DEPOSIT : newTotal;
    uint256 effectiveDiff = newEffective - oldEffective;
    return (effectiveDiff * 100 * weightStake * 1e18) / (
        MAX_DEPOSIT * BASIS_POINTS);
}
```

For example, considering $weightStake = 5000$ basis points, a previous deposit of 5 ETH and a new deposit of 3 ETH, the score increases from 5 to 8 and therefore $\Delta scoreStake = 3$. The number of new governance tokens is:

$$\textbf{newTokens} = \frac{3 \times 5000}{10000} = 1.5 \tag{6.1}$$

which in the contract corresponds to 1.5×10^{18} token units.

increaseStake(). The `increaseStake()` function is used to allow a member to mint additional tokens after joining the DAO by depositing ETH.

The function checks that the caller is actually a member of the DAO, that the Treasury has an assigned address, and that the submitted deposit is greater than 0. It also checks that the ETH previously deposited by the member, added to the amount they are about to deposit, does not exceed `MAX_DEPOSIT`, in order to prevent a possible violation of the ETH deposit limit for minting tokens. If the deposit limit has already been reached or would be exceeded by the new deposit, the function reverts.

After that, it calculates the number of tokens to be received based on the deposit made, using `_calculateStakeTokens`. The amount of ETH deposited by the user is then updated. The tokens are minted and sent to the member, and the ETH is transferred directly to the Treasury.

```
1     function increaseStake() public payable returns (uint256
          newTokens) {
2         if (!isMember[msg.sender]) revert NotMember();
3         if (msg.value == 0) revert ZeroDeposit();
4         if (treasury == address(0)) revert TreasuryNotSet();
5         if (stakeDeposited[msg.sender] >= MAX_DEPOSIT) revert
              MaxDepositReached();
6         if (stakeDeposited[msg.sender] + msg.value >
              MAX_DEPOSIT) revert ExceedsMaxDeposit();
7         newTokens = _calculateStakeTokens(msg.value,
              stakeDeposited[msg.sender]);
8         if (newTokens == 0 && weightStake > 0) revert
              DepositTooSmall();
9         stakeDeposited[msg.sender] += msg.value;
10        if (newTokens > 0) _mint(msg.sender, newTokens);
11        (bool ok, ) = treasury.call{value: msg.value}("");
12        if (!ok) revert TreasuryTransferFailed();
13        emit StakeIncreased(msg.sender, msg.value, newTokens)
              ;
14    }
```

In this section of the code, a check is performed against a possible attack that attempts to bypass the maximum stake limit. A malicious user with a large amount of funds could mint tokens, reach the maximum amount of stake that can be pur-

chased, send the tokens to another account they own, and start minting them again, violating the imposed limit. By recording not only the minted stake but also the ETH deposited by a given user, and checking that this amount does not exceed the maximum deposit limit, the system ensures that an attacker cannot perform this type of violation.

6.5 Competence Certification Protocol

The *Competence Certification Protocol*, described in Subsection 4.4.3, is implemented mainly in the `GovernanceSkill.sol` contract, with the support of the `VPVerifier.sol` library for the cryptographic verification of VCs, the `SkillCalculator.sol` contract for the calculation of the score derived from competences, and the `SkillDefinitions.sol` library for the definition of the skills recognized by the DAO.

To implement the protocol, `GovernanceSkill.sol` maintains in its internal state a `memberDID` mapping, which associates each member with their DID; a `didToAddress` mapping, which associates each DID with an address, preventing the same DID from being registered by multiple accounts; a `trustedIssuers` mapping, which stores the addresses of the Issuers recognized as trusted by the DAO and authorized to sign the VCs containing the competences of the members; and a `memberSkillBitmap` mapping, which associates each member with a bitmap representing the competences they possess.

```
1 mapping(address => bool) public trustedIssuers;
2 mapping(address => bytes32) public memberDID;
3 mapping(bytes32 => address) public didToAddress;
4 mapping(address => uint256) public memberSkillBitmap;
```

To reduce storage costs and simplify DID comparisons, the contract does not store DIDs as strings. Instead, each DID is converted into a fixed-size `bytes32` value by computing its `keccak256` hash. This representation requires less storage than maintaining the complete DID string and allows efficient comparisons between the hashes associated with the stored DIDs.

Bitmap operations. As described in Subsection 4.2.4, the skills possessed by each member are represented through a bitmap. In `GovernanceSkill.sol`, this representation is maintained through the `memberSkillBitmap` mapping, which associates each member address with the corresponding bitmap.

An alternative implementation would have been to use a mapping that associates each member address with an array of skill hashes, in order to retrieve the list of skills possessed by that member, together with a second mapping that associates each member address with a mapping between skill hashes and Boolean values, allowing the system to verify in $O(1)$ whether a given skill is possessed by the member. As discussed in Section 4.2.4, the bitmap-based solution provides a significant reduction in gas costs.

The bitmap is manipulated through bitwise operations, which are used to add new skills, verify whether a skill is possessed, and count the number of skills. A new skill is added through a bitwise OR operation between the current bitmap and a flag representing the user's new skills. This does not modify the skills that are already present and automatically makes the insertion of the same skill multiple times idempotent.

The possession of a skill is verified through a bitwise AND operation between the member's bitmap and the corresponding skill flag. If the corresponding bit in the result is set to 1, the member possesses that skill.

The number of skills possessed can be obtained by counting the set bits, while the difference between two sets of skills can be identified through a bitwise XOR operation.

Checkpoint for maintaining historical voting power. `GovernanceSkill.sol` maintains a checkpoint representing the voting power derived from the skills of a user for a specific topic at a given timepoint. It is implemented as a mapping that associates a user address with an integer representing a topic and with a `Trace208` checkpoint that historically stores an integer value associated with a timepoint.

A second checkpoint is maintained for each topic to store the historical total skill-based voting power of all DAO members for that topic.

```
1 // _skillVotesCheckpoints[account][topicId]
```

```
2 mapping(address => mapping(uint256 => Checkpoints.Trace208))
    private _skillVotesCheckpoints;
3 // _totalSkillSupplyCheckpoints[topicId]
4 mapping(uint256 => Checkpoints.Trace208) private
    _totalSkillSupplyCheckpoints;
```

registerDID(). Before presenting a VC, the member associates their DID with the Ethereum account used in the DAO through the `registerDID()` function. The `registerDID()` function registers the DID of a DAO member. A member can register only one DID, and the same DID cannot be associated with two different addresses. The function verifies that `msg.sender` is a member of the DAO, that the member has not already registered a DID, and that the provided DID corresponds to the Ethereum address of the member. It then calculates the hash of the DID and checks that it has not already been associated with another address. If all checks are passed, it stores the address-to-DID and DID-to-address associations in the mappings.

```
1  function registerDID(string calldata _did) external {
2      if (!governanceToken.isMember(msg.sender)) revert
           NotMember();
3      if (bytes(_did).length == 0) revert EmptyDID();
4      address holderAddress = _parseSupportedEthrDID(_did);
5      if (holderAddress != msg.sender) revert
           DIDControllerMismatch();
6      if (memberDID[msg.sender] != bytes32(0)) revert
           DIDAlreadyRegistered();
7      bytes32 didHash = keccak256(bytes(_did));
8      if (didToAddress[didHash] != address(0)) revert
           DIDAlreadyBound();
9      memberDID[msg.sender] = didHash;
10     didToAddress[didHash] = msg.sender;
11     emit DIDRegistered(msg.sender, didHash);
12     }
```

setTrustedIssuer(). The VCs accepted by the DAO must be signed by an Issuer recognized as `trusted`. The addresses of the authorized Issuers are stored in the

`trustedIssuers` mapping. The `setTrustedIssuer()` function allows the configuration of the addresses of the Issuers that the DAO considers trusted and that are therefore authorized to sign the VCs certifying the competences of members. The first Issuer can be configured only by the deployer. After this initial configuration, new Issuers can be added only by the governance through the Timelock Controller.

upgradeSkillWithVC(). The `upgradeSkillWithVC()` function performs the upgrade of a member's skills through a VC. It uses the `VPVerifier.sol` library to verify a signed VC and applies the upgrade if the VC is valid. The function checks whether the member exists and whether at least one trusted Issuer is configured in the DAO. It checks whether the member's DID is consistent with the DID contained in the Credential Subject. The EIP-712 domain separator is precomputed as a constant, requiring 0 gas for hashing.

The function recovers the address of the signer using the functions of the `VPVerifier` library on the EIP-712 signature contained in the VC. It checks whether the recovered Issuer is equal to one of the trusted Issuers and is consistent with the Issuer's DID. A valid signature also makes it possible to verify the integrity of the signed data contained in the VC. If these checks succeed, it calls the `_mergeSkills()` method to add the verified skills to the bitmap of the skills possessed by the member. It then computes a proof hash associated with the verified Credential Subject, which is emitted through the SkillUpgraded event. If the bitmap has changed, it updates the voting power derived from the member's skills by directly calling `_performUpgrade()`, without going through the governance process.

```
1 function upgradeSkillWithVC(
2 VPVerifier.VerifiableCredential memory _vc,
3 bytes memory _issuerSignature
4 ) external {
5 if (!governanceToken.isMember(msg.sender)) revert NotMember()
    ;
6 if (trustedIssuerCount == 0) revert TrustedIssuerNotSet();
7 bytes32 didHash = memberDID[msg.sender];
8 if (didHash == bytes32(0)) revert NoDIDRegistered();
```

```
9  if (keccak256(bytes(_vc.credentialSubject.id)) != didHash)
       revert DIDMismatch();
10 address recovered = VPVerifier.recoverIssuer(_vc,
       _issuerSignature);
11 address issuerAddress = _parseSupportedEthrDID(_vc.issuer.id)
       ;
12 if (issuerAddress != recovered) revert IssuerDIDMismatch();
13 if (!trustedIssuers[recovered]) revert UntrustedIssuer();
14
15 (uint256 bitmap, bool changed) = _mergeSkills(msg.sender, _vc
       .credentialSubject.skills);
16 bytes32 issuerDidHash = keccak256(bytes(_vc.issuer.id));
17 bytes32 proofHash = _credentialSubjectProofHash(_vc.
       credentialSubject);
18 if (changed) _performUpgrade(msg.sender, bitmap);
19 emit SkillUpgraded(msg.sender, proofHash);
20 emit SkillUpgradedWithVC(msg.sender, issuerDidHash);
21 }
```

_mergeSkills() The helper function `_mergeSkills()`, given the address of a member and a list of skill identifiers, checks that the received skills are valid and adds them to the bitmap of the skills possessed by the member. For each skill, it checks that the skill is valid, generates the corresponding flag, and performs a bitwise OR operation between the user's bitmap and the skill flag.

```
1      function _mergeSkills(address member, string[] memory
           skillNames) internal returns (uint256 bitmap, bool
           changed) {
2          uint256 oldBitmap = memberSkillBitmap[member];
3          bitmap = oldBitmap;
4          for (uint256 i = 0; i < skillNames.length; i++) {
5              bytes32 skillId = SkillDefinitions.skillId(
                   skillNames[i]);
6              uint256 flag = SkillDefinitions.skillFlag(skillId
                   );
7              if (flag == 0) revert InvalidSkill(skillId);
8              bitmap |= flag;
9          }
10         changed = bitmap != oldBitmap;
```

```
11          if (changed) {
12              memberSkillBitmap[member] = bitmap;
13              emit MemberSkillsMerged(
14                  member,
15                  SkillDefinitions.countSkills(bitmap ^
                        oldBitmap),
16                  SkillDefinitions.countSkills(bitmap)
17              );
18          }
19      }
```

_performUpgrade(). The `_performUpgrade()` function is responsible for updating the member's skill-based voting power after the VC has been successfully verified.

The function retrieves the current timepoint through `clock()`, which is used as the temporal key for the checkpoints that record the updated voting power values.

For each topic, the skill-based voting power is calculated according to Equation 4.4. To calculate the skill score associated with each topic, the function uses the `ISkillCalculator` interface. The function receives the bitmap representing the skills possessed by the member and passes it to `SkillCalculator`, which returns the corresponding score for each topic.

Then, for each topic, the resulting score is multiplied by `weightSkill`, normalized with respect to `BASIS_POINTS`, and finally stored in the corresponding checkpoint through `_writeSkillVotes()`.

```
1  function _performUpgrade(address _member, uint256
       _skillBitmap) internal {
2  uint48 blk = clock();
3  uint256[] memory scores = skillCalculator.
       calculateAllVPFromBitmap(_skillBitmap);
4  if (scores.length != SkillDefinitions.TOPIC_COUNT) revert
       InvalidCalculator();
5  for (uint256 topicId = 0; topicId < SkillDefinitions.
       TOPIC_COUNT; topicId++) {
6  uint256 score = scores[topicId];
7  if (score > 100) revert InvalidCalculator();
8  uint256 newVP = (score * weightSkill * 1e18) / BASIS_POINTS;
```

```
9  _writeSkillVotes(_member, topicId, newVP, blk);
10 }
11 }
```

_writeSkillVotes(). The `_writeSkillVotes()` function adds, after a skill upgrade, the additional voting power derived from skills for a given topic and a given member to the checkpoints associated with the member and to the total supply of each topic. The function creates a record in the checkpoint related to the member's voting power for a given topic, storing the new value together with the corresponding timepoint.

It then creates a record for the total supply checkpoint. It compares the member's current voting power with the voting power stored in the latest checkpoint. If the member's voting power has increased, it adds the difference, while if it has decreased, it subtracts the difference.

```
1  function _writeSkillVotes(
2  address account,
3  uint256 topicId,
4  uint256 newVP,
5  uint48 checkpointKey
6  ) internal {
7  Checkpoints.Trace208 storage userTrace =
       _skillVotesCheckpoints[account][topicId];
8  uint208 oldVotes = userTrace.latest();
9  uint208 newVotes = SafeCast.toUint208(newVP);
10 if (newVotes == oldVotes) return;
11 Checkpoints.Trace208 storage totalTrace =
       _totalSkillSupplyCheckpoints[topicId];
12 uint208 oldTotal = totalTrace.latest();
13 userTrace.push(checkpointKey, newVotes);
14 if (newVotes > oldVotes) {
15 totalTrace.push(checkpointKey, oldTotal + (newVotes -
       oldVotes));
16 } else {
17 totalTrace.push(checkpointKey, oldTotal - (oldVotes -
       newVotes));
18 }
19 }
```

Supported topics and skills. The topics and skills supported by the DAO are defined in the `SkillDefinitions.sol` library. Each topic is associated with a numerical identifier, while each skill is associated with a fixed position in the bitmap used to represent the competences of a member. If the bit at that position is set in a bitmap, the bitmap contains that skill.

The library also provides functions to convert a string into the corresponding `skillId`, verify whether a topic or a skill is supported by the DAO, retrieve the flag associated with a skill, count how many skills are present in a bitmap, and return the list of skills contained in a bitmap.

Table 6.1: Voting topics supported by the DAO and their numerical identifiers

Topic	**Identifier**
AI & Data	0
Cloud & Cybersecurity	1
FinTech & Blockchain	2
Enterprise Software	3

In this implementation, the supported voting topics are `AI & Data`, `Cloud & Cybersecurity`, `FinTech & Blockchain`, and `Enterprise Software`, as shown in Table 6.1. The skills that a member can have are `machineLearning`, `dataEngineering`, `cyberSecurity`, `cloudArchitecture`, `distributedSystems`, `blockchain`, `softwareArchitecture`, and `startupFinance`, as shown in Table 6.2.

Table 6.2: Skills supported by the DAO and their corresponding positions in the skill bitmap

Skill	Bit position
machineLearning	0
dataEngineering	1
cyberSecurity	2
cloudArchitecture	3
distributedSystems	4
blockchain	5
softwareArchitecture	6
startupFinance	7

Skill score calculation. After the competences contained in a verified VC have been added to the member's bitmap, the system calculates the competence score associated with each supported topic through the SkillCalculator.sol contract.

GovernanceSkill.sol accesses the calculator through the ISkillCalculator.sol interface, which exposes the `calculateAllVPFromBitmap()` function, that is required to calculate, based on a user's skills, the voting power scores for each topic. The use of an interface separates the competence evaluation logic from the rest of the governance system, In this way, `GovernanceSkill` does not depend on a specific implementation of `SkillCalculator`, but only requires compliance with the `ISkillCalculator` interface. Users who reuse the project modules can therefore develop their own competence calculator, defining different formulas, weights, and evaluation criteria for each topic, without having to modify the governance contract. It is sufficient for the new contract to implement the `calculateAllVPFromBitmap()` function defined by the interface.

The `calculateAllVPFromBitmap()` function receives the bitmap representing the competences of a member and returns a score between 0 and 100 for each supported topic. The function initializes an array containing the score for each topic and iterates over all supported skills. For each skill, it generates the corresponding flag and uses a bitwise AND operation to check whether the skill is present in the member's bitmap. If the skill is present, the function calls

`_addRelevance()` to add its contribution to the scores of the different topics. After all skills have been processed, `_addBoosts()` applies additional bonuses for specific combinations of complementary skills.

```
function calculateAllVPFromBitmap(uint256 skillFlags) public
    pure override returns (uint256[] memory scores) {
scores = new uint256[](SkillDefinitions.TOPIC_COUNT);
for (uint256 i = 0; i < SkillDefinitions.SKILL_COUNT; i++) {
uint256 flag = 1 << i;
if ((skillFlags & flag) != 0) _addRelevance(scores, flag);
}
_addBoosts(scores, skillFlags);
for (uint256 topicId = 0; topicId < SkillDefinitions.
    TOPIC_COUNT; topicId++) {
if (scores[topicId] > MAX_SCORE) scores[topicId] = MAX_SCORE;
}
}
```

The value of each skill depends on the topic being considered. If a competence is central to a topic, a score of 35 is assigned; if it is strongly relevant, a score between 25 and 30 is assigned depending on the skill and the topic; if it provides support for a topic, a score between 15 and 20 is assigned; and if it is considered marginal, a score between 5 and 10 is assigned.

In this way, possessing pairs of complementary skills, with one central skill and one strongly relevant skill, or two strongly relevant skills, gives a score between 60 and 65. When the user possesses specific combinations of skills that are relevant to a particular topic, the score receives a boost of +10, limiting the score of two competences that receive the boost to a value of 75.

The scores are weighted for each topic so that, even if a user possesses the best combination of skills for that topic, their score cannot exceed 100. The utility function `_addRelevance()` assigns, for each skill flag, a different contribution to the score of each topic, as shown in Table 6.3, while `_addBoosts()` assigns the additional boosts for specific combinations of complementary skills.

Table 6.3: Relevance score of each skill for the supported topics.

Skill	AI & Data	Cloud & Cyber.	FinTech & Blockchain	Enterprise Software
machineLearning	35	5	5	15
dataEngineering	30	20	15	20
cyberSecurity	15	35	25	20
cloudArchitecture	15	30	20	30
distributedSystems	20	30	25	30
blockchain	5	15	35	15
softwareArchitecture	15	25	20	35
startupFinance	15	10	30	25

6.5.1 VC Issuance and On-Chain Cryptographic Verification

On-Chain Cryptographic Verification

As part of the *Competence Certification Protocol*, the VC is first issued and signed off-chain by a trusted Issuer using Veramo. The credential contains the competences certified for the member and is signed according to the `EIP-712`[45] standard.

When the credential is presented to the DAO through the previously described `upgradeSkillWithVC()` function, its cryptographic verification is performed on-chain through the `VPVerifier.sol` library, using the EIP-712 standard. The library takes the credential data, reconstructs the same digest that was signed off-chain by the Issuer, and uses the provided signature to recover the Ethereum address of the signer.

The recovered address is then used by `GovernanceSkill.sol` to verify that the credential was signed by an Issuer recognized as trusted by the DAO. If the verification succeeds, the protocol can proceed with the registration of the certified competences and the subsequent update of the member's skill-based voting power, as shown in Code 6.5.

EIP-712 type hashes. The library uses constants to construct the `typeHash`, that is, the hash of the structure of the fields that compose the VC. The names, types, and order of the fields must be identical to those used off-chain during the issuance of the VC. The library hashes the structure of the Credential Subject,

which contains the fields `id`, `organization`, `unit`, and `skills`.

It then reconstructs the `typeHash` of the main `VerifiableCredential` struct, which contains `issuer`, `issuanceDate`, and the nested `credentialSubject`, as required by EIP-712.

```
bytes32 internal constant ISSUER_TYPEHASH = keccak256("Issuer
    (string id)");
bytes32 internal constant CREDENTIAL_SUBJECT_TYPEHASH =
    keccak256(
        "CredentialSubject("
        "string id,"
        "string organization,"
        "string unit,"
        "string[] skills"
        ")"
    );

bytes32 internal constant VERIFIABLE_CREDENTIAL_TYPEHASH =
    keccak256(
        "VerifiableCredential("
        "Issuer issuer,"
        "string issuanceDate,"
        "CredentialSubject credentialSubject"
        ")"
        "CredentialSubject("
        "string id,"
        "string organization,"
        "string unit,"
        "string[] skills"
        ")"
        "Issuer("
        "string id"
        ")"
    );
```

VC data structures. The library then defines structs used to contain the VC data. It creates a struct containing the identity of the Issuer, represented by its DID. It then creates a struct containing the information related to the Holder, that

is, the Credential Subject, whose fields are the same as those already described.

Finally, it creates a struct containing all the information of the VC to be verified. It includes the two previous structs and the issuance date as fields.

```
struct Issuer {
string id;
}
struct CredentialSubject {
    string id;
    string organization;
    string unit;
    string[] skills; // Dynamic array of skills
}
struct VerifiableCredential {
    Issuer issuer;
    string issuanceDate;
    CredentialSubject credentialSubject;
}
```

Credential hashing. The first function performs the EIP-712 hash of the `Issuer` struct by hashing the `typeHash` and the Issuer ID.

The second function performs the EIP-712 hash of the `CredentialSubject` struct by hashing the `typeHash` together with the hashes of all the values contained in the Holder's information fields.

The third function performs the EIP-712 hash of the complete credential. For nested struct fields, it uses their corresponding hashes by calling the two previous functions on the complete data.

```
function hashIssuer(Issuer memory issuer) internal pure
    returns (bytes32) {
return keccak256(
abi.encode(
ISSUER_TYPEHASH,
keccak256(bytes(issuer.id))
)
);
}

```

```
function hashCredentialSubject(
    CredentialSubject memory cs
) internal pure returns (bytes32) {
    return keccak256(
        abi.encode(
            CREDENTIAL_SUBJECT_TYPEHASH,
            keccak256(bytes(cs.id)),
            keccak256(bytes(cs.organization)),
            keccak256(bytes(cs.unit)),
            hashSkills(cs.skills)
        )
    );
}

function hashVerifiableCredential(
    VerifiableCredential memory vc
) internal pure returns (bytes32) {
    return keccak256(
        abi.encode(
            VERIFIABLE_CREDENTIAL_TYPEHASH,
            hashIssuer(vc.issuer),
            keccak256(bytes(vc.issuanceDate)),
            hashCredentialSubject(vc.credentialSubject)
        )
    );
}
```

Signer recovery. The Recover Signer function recreates the EIP-712 digest using the functions implemented previously. The EIP-712 digest is recreated by concatenating the standard prefix `0x1901`, the `domainSeparator`, and the digest. The final digest must match bit-by-bit with the one used by the off-chain signer. If it does not match, the VC data has been altered. Starting from the digest and the signature, the address associated with the public key that produced the signature is recovered. The caller then compares this address with the trusted Issuers. If it matches a trusted Issuer, the VC is valid.

```
function recoverIssuer(
VerifiableCredential memory vc,
```

```
3  bytes memory signature
4  ) internal pure returns (address signer) {
5  bytes32 structHash = hashVerifiableCredential(vc);
6  bytes32 digest = keccak256(
7  abi.encodePacked(
8  "\x19\x01",
9  UNIVERSAL_DOMAIN_SEPARATOR,
10 structHash
11 )
12 );
13 signer = ECDSA.recover(digest, signature);
14 }
```

Generation and Signing of Verifiable Credentials

As part of the *Competence Certification Protocol*, the Veramo module is responsible for the issuance of VCs by an Issuer that certifies competences to DAO members acting as Holders.

Each Holder receives a VC containing one or more skills recognized by the Issuer, which can later be verified on-chain and used for the calculation of competence-based voting power.

Architecture of the Credential Issuance Module. The central component of this part of the protocol is a Veramo agent, configured with different modules, each responsible for a specific function. The `DIDManager` plugin manages the decentralized identity DID of the Issuer, while the `KeyManager` plugin manages the cryptographic key used to sign the credentials.

The persistent information of the agent is stored in a SQLite database through the `DataStore` plugin. The database stores the Issuer DID, the references to the related keys, the saved VCs, and the cryptographic material required for their use.

The private key of the Issuer is not stored in plaintext, but is entrusted to a local Key Management System (KMS), which encrypts it using a secret key external to the program source before storing it in the database. In this way, the `KeyManager` can request the KMS to perform a signature without having to directly manage the private key in plaintext.

The `did:ethr` method is used to represent identities, in which the DID is derived directly from the Ethereum address of the subject. Each Ethereum address is associated with an identifier of the form `did:ethr:0x...`.

The agent also uses a `CredentialPlugin`, responsible for creating the VCs, to which a custom provider, `DaoEip712CredentialProvider`, is associated. This provider makes it possible to produce credentials signed according to the EIP-712 format.

The Issuer is imported into the agent starting from its Ethereum wallet. The DID is constructed from its address, a correspondence check between the DID and the address is performed, and a controller key identified by the suffix `#controller` is defined.

The related private key is then entrusted to the KMS and associated with the DID, allowing the agent to use that identity to sign the credentials.

The Holders, instead, are not imported into the agent and their private keys are never stored by Veramo. For them, only the public DID derived from the corresponding Ethereum address is used.

The main scripts used by this part of protocol are `agent.ts`, `credentials.ts`, `DaoEip712CredentialProvider.ts`, and `issue-for-dao.ts`. The `agent.ts` file is responsible for configuring the Veramo agent, while `credentials.ts` defines the VC model. `DaoEip712CredentialProvider.ts` implements the custom provider used to create and sign the credentials according to EIP-712, while `issue-for-dao.ts` coordinates the actual credential issuance process using the Veramo agent.

Structure of Verifiable Credentials. The issued credentials follow the W3C VC model and are identified by the types `VerifiableCredential` and `SkillCredential`. Each VC contains the DID of the Issuer, the issuance date, and the `credentialSubject`, that is, the set of certified information related to the recipient. The `credentialSubject` contains the DID of the member, the organization that certified the competences, the reference organizational unit, and the set of certified skills. This information represents the content that the Issuer declares to be authentic through its digital signature.

The following credential example certifies that the Holder identified by the

specified DID, associated with the organization University of Pisa and the Digital Economy unit, possesses the skills `blockchain` and `startupFinance`, issued by the Issuer University of Pisa.

```
{
"@context": ["https://www.w3.org/2018/credentials/v1"],
"type": ["VerifiableCredential", "SkillCredential"],
"issuer": {"id": "did:ethr:0
   x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC"},
"issuanceDate": "2026-09-21T09:07:55Z",
"credentialSubject": {
"id": "did:ethr:0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc",
"organization": "University of Pisa",
"unit": "Digital Economy",
"skills": ["blockchain", "startupFinance"]
},
"proof": {
"type": "EthereumEip712Signature2021",
"created": "2026-09-21T09:07:55Z",
"proofPurpose": "assertionMethod",
"verificationMethod": "did:ethr:0
   x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC#controller",
"proofValue": "0xdf35a4972b3da04fa2b97a9d6c61bec9df
909c1aa5eaab19fdaa88d12ebb54110bb2c6e
526a9cc4d5e66521f17b3094a2f5d9d5a455d
d2a20f7632832b655fe01b",
"eip712": {
"domain": {
"name": "Universal VC Protocol",
"version": "1"
},
"types": {
"Issuer": [{ "name": "id","type": "string"}],
"CredentialSubject": [
{"name": "id", "type": "string"},
{"name": "organization", "type": "string"},
{"name": "unit","type": "string"},
{"name": "skills","type": "string[]"}
],
"VerifiableCredential": [
```

```
35 {"name": "issuer", "type": "Issuer"},
36 {"name": "issuanceDate", "type": "string"},
37 {"name": "credentialSubject", "type": "CredentialSubject"}
38 ]
39 },
40 "primaryType": "VerifiableCredential"
41 }
42 }
43 }
```

The Verifiable Credential is structured as follows:

@context: Field that indicates the context of the credential and specifies that the structure follows the W3C Verifiable Credentials model.

type: Field that indicates the type of the credential. In this case, the credential is both a standard VerifiableCredential and a SkillCredential, that is, a credential used specifically to certify competences.

issuer: Field that identifies through a DID the organization that issues and signs the credential.

issuanceDate: Field that represents the date and time of issuance of the VC.

credentialSubject: Field that contains the information related to the Holder. It includes the DID of the subject, the organization that certifies the competences, the reference unit within the organization, and the list of certified skills.

proof: Composite field that contains the information required to verify the cryptographic signature of the credential. It is composed of the following fields:

type: Field that indicates the type of signature used. It allows the Verifier to understand which algorithm must be used to verify it. In this case, the signature is based on EIP-712.

created: Field that indicates when the signature was generated.

proofPurpose: Field that indicates the purpose of the signature. The value `assertionMethod` specifies that the Issuer uses its own key to attest the validity of the information contained in the credential.

verificationMethod: Field that indicates which public key must be used to verify the signature. In this case, a DID linked to a public key is specified.

proofValue: Field that contains the actual digital signature produced using the Issuer's private key.

types: Composite field that contains the structure of the fields of the VC.

In this experimental implementation, the organization name used is University of Pisa, while the competences that can be certified are those supported by the DAO and defined in `SkillDefinitions.sol`.

Credential Signature. To allow the smart contracts to verify the signature of the credential, the solution uses EIP-712, an Ethereum standard for signing structured data. Unlike the signature of a simple sequence of bytes, EIP-712 explicitly defines which fields compose the message and which type is associated with each field.

Three main types are defined in the project. `Issuer` contains the DID of the entity that issues the credential, `CredentialSubject` contains the DID of the recipient, the organization, the unit, and the array of skills, while `VerifiableCredential` connects the Issuer, the issuance date, and the CredentialSubject.

An EIP-712 domain, identified by a name and a version, is associated with these types. In the implementation, the name `Universal VC Protocol` was chosen, with the objective of defining a schema that is not explicitly linked to a single blockchain and of supporting the portability of the credential across different Ethereum-compatible networks.

```
1  export const CREDENTIAL_CONTEXT = ["https://www.w3.org/2018/
       credentials/v1"] as const;
2  export const CREDENTIAL_TYPE = ["VerifiableCredential", "
       SkillCredential"] as const;
3  export const EIP712_DOMAIN = {
4  name: "Universal VC Protocol",
5  version: "1",
6  } as const;
7  export const VC_TYPES: Record<string, Array<{ name: string;
       type: string }>> = {
8  Issuer: [{ name: "id", type: "string" }],
9  CredentialSubject: [
10 { name: "id", type: "string" },
11 { name: "organization", type: "string" },
12 { name: "unit", type: "string" },
```

```
13 { name: "skills", type: "string[]" },
14 ],
15 VerifiableCredential: [
16 { name: "issuer", type: "Issuer" },
17 { name: "issuanceDate", type: "string" },
18 { name: "credentialSubject", type: "CredentialSubject" },
19 ],
20 };
21 export interface CredentialSubject {
22 id: string;
23 organization: string;
24 unit: string;
25 skills: SkillName[];
26 }
```

The structure and the order of the fields are kept the same in the Veramo component and in the Solidity verifier, so that both calculate the same cryptographic message to be verified. To obtain this compatibility, a custom signing provider called `DaoEip712CredentialProvider` was implemented, replacing the standard Veramo provider that automatically generates the EIP-712 schema.

When Veramo has to create a credential, the provider first retrieves the DID of the Issuer and the Secp256k1 key associated with that identity. Before signing, it checks that the Ethereum address obtained from the public key matches the address embedded in the Issuer DID, in order to guarantee that the key used for the signature actually belongs to the identity that the credential declares as the Issuer.

In a way that is symmetric to the on-chain verification, the EIP-712 signature is generated starting from the structured data defined in the credential. For each type, such as `Issuer`, `CredentialSubject`, and `VerifiableCredential`, EIP-712 defines a textual representation of the structure and calculates a type hash, that is, the hash that uniquely identifies that type and the order of its fields. The actual values of the credential are then encoded together with the corresponding type hashes, obtaining the hashes of the internal structures and finally the hash of the complete VC. This value is combined with the domain separator, calculated from the EIP-712 domain, obtaining the final digest to be signed.

In the implementation, these steps are not calculated manually. Instead, the

provider passes the domain, the types contained in the VC, and their values to the `KeyManager`, and the implementation of the `eth_signTypedData` algorithm internally performs the EIP-712 encoding and the calculation of the corresponding hashes. The resulting digest is then signed with the Issuer's Secp256k1 private key stored in the KMS, producing the signature stored in the `proofValue` field. For the on-chain verification to succeed, the Solidity contract must use the same type hashes, the same domain, and the same field structure, so that it can reconstruct exactly the same digest signed off-chain.

```
async createVerifiableCredential(
....
const signingPayload = {
issuer: { id: did },
issuanceDate,
credentialSubject: args.credential.credentialSubject,
};
const proofValue = await context.agent.keyManagerSign({keyRef
    : key.kid, algorithm: "eth_signTypedData",
data: JSON.stringify({domain: EIP712_DOMAIN, types: VC_TYPES,
     primaryType: "VerifiableCredential", message:
    signingPayload,}),});
return {
"@context": [...CREDENTIAL_CONTEXT],
type: [...CREDENTIAL_TYPE],
...signingPayload,
proof: {
type: DAO_EIP712_PROOF,
created: issuanceDate,
proofPurpose: "assertionMethod",
verificationMethod: key.kid,
proofValue,
eip712: {
domain: EIP712_DOMAIN,
types: VC_TYPES,
primaryType: "VerifiableCredential",
},
},
} as VerifiableCredential;
}
```

The result is an Ethereum signature that makes it possible, during verification, to reconstruct the address of the wallet that produced it and check that it corresponds to the Issuer declared in the credential.

The result of the signature is stored in the `proofValue` field of the VC. A proof object is then added to the credential, specifying the signature type `EthereumEip712Signature2021`, the creation date, the purpose of the signature, the key used through `verificationMethod`, the actual cryptographic signature, and the description of the EIP-712 schema used.

Credential issuance orchestration. The actual VC issuance process is coordinated by the `issue-for-dao.ts` script. For each Holder, the script constructs the DID from the corresponding Ethereum address and invokes the Veramo `createVerifiableCredential()` method, specifying the Holder information, the certified skills, the EIP-712 proof format, and the Issuer controller key.

After the credential has been created and signed through the custom provider described previously, it is verified through Veramo's `verifyCredential()` method. Only credentials that successfully pass this verification are stored in the agent's `DataStore` and exported as JSON documents so that they can subsequently be presented to the DAO.

```
...
const credential = await agent.createVerifiableCredential({
credential: {
"@context": [...CREDENTIAL_CONTEXT],
type: [...CREDENTIAL_TYPE],
issuer: { id: issuerIdentity.did },
issuanceDate,
credentialSubject: {
id: holderDid,
organization: DEFAULT_ORGANIZATION.name,
unit: holder.unit,
skills: holder.skills,
},
},
proofFormat: DAO_EIP712_PROOF,
keyRef: issuerIdentity.keyId,
});
```

```
18 ....
19 ....
20 const vcVerification = await agent.verifyCredential({
       credential });
21 if (!vcVerification.verified) throw new Error(`VC ${holder.
       alias} non verificabile da Veramo`);
22 await agent.dataStoreSaveVerifiableCredential({
       verifiableCredential: credential });
23 const fileName = outputName(index, holder.alias);
24 fs.writeFileSync(path.join(sharedCredentials, fileName), JSON
       .stringify(credential, null, 2));
25 ....
```

6.6 Governance Protocol

The `Governance Protocol`, described in Subsection 4.4.4, is mainly implemented in the `MyGovernor.sol` contract, which manages the DAO governance process and coordinates the entire lifecycle of proposals, from their creation and voting to their possible execution through the Timelock Controller.

The contract inherits and combines several OpenZeppelin governance modules, reusing functionalities already available for proposal management (`Governor.sol`), vote counting (`GovernorCountingSimple.sol`), governance parameters (`GovernorSettings.sol`), quorum (`GovernorVotesQuorumFraction.sol`), superquorum (`GovernorVotesSuperQuorumFraction.sol`), historical voting
power (`GovernorVotes.sol`), and the timelock (`GovernorTimelockControl`).

The main extension introduced by `MyGovernor.sol` concerns the management of governance based on topics and competences. Each proposal is associated with a `topicId`, which identifies the area to which it belongs. The voting power of each member is therefore calculated by combining two components: the voting power derived from stake, obtained from the governance token, and the voting power derived from skills, retrieved from `GovernanceSkill` according to the topic of the proposal.

Both components are read at the snapshot timepoint, in order to prevent later

changes in stake or competences from modifying the voting weight already determined for that proposal. The calculation of quorum and superquorum is also adapted to the multi-topic model. The thresholds are not applied only to the historical supply of the token, as would happen without these modifications, but to the total voting supply of the topic, obtained by adding the total supply of the stake component and the skill component related to that topic.

The internal state contains two immutable references to `IGovernanceToken` and `IGovernanceSkill`, and a mapping that associates each `proposalId` with its `topicId`. This mapping is used to retrieve the `topicId` of a proposal, in order to retrieve the supply of voting power related to the skills of the topic associated with that proposal.

```
1 IGovernanceToken public immutable governanceToken;
2 IGovernanceSkill public immutable governanceSkill;
3 mapping(uint256 => uint256) public proposalTopic;
```

The constructor takes as input `IGovernanceToken`, `IGovernanceSkill`, the `TimelockController`, the number of blocks to wait before voting begins, the duration of the voting window, the minimum voting threshold required to create a proposal, the quorum as a percentage of the total voting supply at the snapshot block of the proposal, and the superquorum. The inherited contracts are initialized using these parameters.

proposeWithTopic(). The `proposeWithTopic()` function creates a proposal and associates it with a `topicId`. It replaces the `propose()` function by adding support for the `topicId` parameter, which is not present in the original implementation, because each proposal must declare its competence area through a `topicId`. The voting power derived from skills is then calculated based on this topic, and consequently the quorum and superquorum are also calculated from it.

The function checks that the `topicId` is valid, calls `super.propose()` to create the proposal, and then stores the `topicId` in `proposalTopic[proposalId]`.

```
1 function proposeWithTopic(
2 address[] memory targets,
3 uint256[] memory values,
4 bytes[] memory calldatas,
```

```
5  string memory description,
6  uint256 topicId
7  ) external returns (uint256 proposalId) {
8  _validateTopicId(topicId);
9  proposalId = super.propose(targets, values, calldatas,
       description);
10 proposalTopic[proposalId] = topicId;
11 emit ProposalTopicSet(proposalId, topicId);
12 }
```

Voting power calculation. During a voting process, the `_getVotes()` function retrieves and combines the voting power of a member derived from stake and skills, according to the formula defined in Equation 4.1.

The function takes as input the member's account, the snapshot block of the proposal, and the proposal parameters, which contain the `topicId`. It first checks that the account belongs to a DAO member; otherwise, it returns zero voting power.

The votes derived from stake are retrieved using the `getVotes()` function inherited by `ERC20Votes`. If the proposal parameters are empty, and therefore no `topicId` is present, the function returns only the votes derived from stake.

Otherwise, the `topicId` is decoded from the proposal parameters and its validity is checked. Then, from the checkpoint that stores the voting power derived from the skills of each member, the voting power related to the proposal topic at the corresponding `timepoint` is retrieved.

Finally, the function returns the sum of the voting power derived from stake and the voting power derived from skills.

```
1  function _getVotes(address account, uint256 timepoint, bytes
       memory params) internal view override(Governor,
       GovernorVotes) returns (uint256) {
2      if (!governanceToken.isMember(account)) {
3          return 0;
4      }
5      uint256 stakeVotes = super._getVotes(account, timepoint,
           params);
6      if (params.length == 0) {
```

```
7           return stakeVotes;
8       }
9       if (params.length != 32) revert InvalidVoteParams();
10      uint256 topicId = abi.decode(params, (uint256));
11      _validateTopicId(topicId);
12      uint256 skillVotes = governanceSkill.getPastSkillVotes(
            account, topicId, timepoint);
13      return stakeVotes + skillVotes;
14  }
```

Topic-specific total voting power. Without modifications, the `Governor` contract considers as total voting supply only the total supply related to the ERC20Votes token, without taking into account the component derived from skills. This makes the voting, quorum, and superquorum mechanisms incorrect. Therefore, this mechanism must be adapted to the voting power model used by SAGE.

The `_totalVotingPowerForTopic()` function calculates the total voting supply for a specific topic, considering both the stake component and the skill component. The formula used is:

$$\textbf{supplyTopic} = \textbf{supplyStake} + \textbf{supplySkill}(topicId)$$

To retrieve the two components of the total supply, the checkpoints managed respectively by `ERC20Votes` and `governanceSkill` are used.

```
1   function _totalVotingPowerForTopic(uint256 topicId, uint256
        timepoint) internal view returns (uint256) {
2       uint256 stakeSupply = governanceToken.getPastTotalSupply(
            timepoint);
3       uint256 skillSupply = governanceSkill.
            getPastTotalSkillSupply(topicId, timepoint);
4       return stakeSupply + skillSupply;
5   }
```

Topic-based quorum calculation. The `_quorumForTopic()` function calculates the quorum of a vote related to a specific topic at a given timepoint. It retrieves the total supply through the previous function and applies the quorum

percentage. Without this modification, the quorum would be calculated only on the basis of the total supply derived from stake, invalidating the vote. In the same way, the modification is also applied to pass the total supply to the function that calculates the superquorum.

The `_quorumReached()` function is overridden. This function determines whether a proposal has reached the quorum, replacing `GovernorCountingSimple._quorumReached()`, which would call `quorum()` instead of `_quorumForTopic()`. The function retrieves the snapshot timepoint of the proposal and the `topicId` from the corresponding mapping. It calculates the quorum related to the topic of the proposal using the previous custom function. It retrieves the number of `For` and `Abstain` votes received by the proposal. It compares this number with the quorum calculated on the total voting power supply. If it is greater than or equal to q, the quorum is reached.

```
function _quorumForTopic(uint256 topicId, uint256 timepoint)
    internal view returns (uint256) {
    return Math.mulDiv( _totalVotingPowerForTopic(topicId,
        timepoint), quorumNumerator(timepoint),
        quorumDenominator());
}

function _quorumReached(uint256 proposalId) internal view
    override(Governor, GovernorCountingSimple) returns (bool)
    {
    uint256 snapshot = proposalSnapshot(proposalId);
    uint256 topicId = proposalTopic[proposalId];
    uint256 q = _quorumForTopic(topicId, snapshot);
    (, uint256 forVotes, uint256 abstainVotes) =
        proposalVotes(proposalId);
    return q <= forVotes + abstainVotes;
}
```

Topic-based superquorum calculation. The `_superQuorumReached()` function was removed in OpenZeppelin 5.4, so the topic-specific superquorum logic is incorporated into the `state()` function. The `state()` function returns the current state of a proposal, returning states such as `Pending`, `Active`, `Succeeded`,

`Defeated`, `Queued`, `Executed`, and `Canceled`.

To implement the superquorum logic, the following adaptation is applied: while the proposal is in the `Active` state, the current number of votes is compared with the superquorum calculated on the total supply of the proposal topic. The current state of the proposal is retrieved through the `state()` function of `Governor`. If the state is `Active`, the function checks whether the superquorum has been reached using the custom formula. If the `For` votes are greater than or equal to the superquorum of the total supply and are also greater than the `Against` votes, the proposal moves to the `Succeeded` state using the `state()` function of `GovernorSuperQuorumFraction`. Otherwise, it remains in the `Active` state.

```
function state(
uint256 proposalId
)
public
view
override(
Governor,
GovernorVotesSuperQuorumFraction,
GovernorTimelockControl
)
returns (ProposalState)
{
ProposalState baseState = Governor.state(proposalId);
    if (baseState == ProposalState.Active) {
        (, uint256 forVotes, ) = proposalVotes(proposalId);
        uint256 topicId = proposalTopic[proposalId];
        uint256 sq = _superQuorumForTopic(
            topicId,
            proposalSnapshot(proposalId)
        );

        if (forVotes >= sq && _voteSucceeded(proposalId)) {
            return super.state(proposalId);
        }

        return ProposalState.Active;
    }
```

```
28
29      return super.state(proposalId);
30  }
```

In this implementation, the quorum is set to 20% and the superquorum to 70%. These settings can be modified parametrically.

6.7 Investment Execution Protocol

The `Investment Execution Protocol`, described in Subsection 4.4.5, is mainly implemented through the `Treasury.sol` and `StartupRegistry.sol` contracts. The Treasury stores the funds of the DAO and executes investments approved through governance, while the Startup Registry maintains the startups that can receive these investments.

Treasury. `Treasury.sol` is a contract that stores the funds of the DAO and allows them to be invested in other addresses, in this implementation startups, only if the operation has been approved by the governance and passes through the `TimelockController`. In fact, the Treasury cannot be controlled by the deployer or by any other account, and the only address that can call the `investStartup()` function is the `TimelockController`. This guarantees that no single individual can move the funds without the consent of the DAO.

The fund movement flow is the following. Members deposit ETH by minting tokens. A member proposes an investment through the `Governor`. The community votes. If the proposal is approved, it is added to the Timelock queue. After the delay, the Timelock executes `Treasury.investStartup()`, sending the ETH to the address specified by the proposal.

The state variables include the address of the `TimelockController`, which is the only address that can authorize investments, and a mapping that stores the history of all investments made, using the startup address as the key and the amount of ETH invested in it as the value.

The contract uses a modifier to indicate that only the Timelock can call the decorated function. The smart contract constructor initializes the Treasury by taking the `TimelockController` address as input and setting it.

There is a `deposit()` function that allows anyone to deposit ETH into the Treasury. This function is called by `GovernanceToken` to deposit the funds received when members mint tokens.

The main function of the Treasury is `investStartup()`, which can only be called by the Timelock and is protected by the `nonReentrant` modifier. It allows the DAO to invest ETH in a startup address. It takes as input the address of the destination startup and the amount in wei to invest. It is called by the Timelock after an investment proposal has been approved and the delay has passed. It retrieves the data of the startup in which to invest by calling `StartupRegistry.sol`. It increases the amount invested in the startup and transfers ETH to the startup wallet.

```
1  function deposit() external payable {
2  if (msg.value == 0) revert ZeroAmount();
3  emit Deposited(msg.sender, msg.value);
4  }
5
6  function investStartup(
7      uint256 _startupId,
8      uint256 _amount
9  ) external onlyTimelock nonReentrant {
10     if (address(startupRegistry) == address(0)) revert
           RegistryNotSet();
11     if (_amount == 0) revert ZeroAmount();
12     if (address(this).balance < _amount) revert
           InsufficientBalance();
13
14     (, address startupWallet,, bool active) = startupRegistry
           .getStartup(_startupId);
15
16     if (!active) revert StartupInactive();
17     if (startupWallet == address(0)) revert ZeroAddress();
18
19     investedIn[startupWallet] += _amount;
20
21     (bool success, ) = startupWallet.call{value: _amount}("")
           ;
22     if (!success) revert TransferFailed();
23
```

```
24      emit Invested(_startupId, startupWallet, _amount);
25  }
```

Startup Registry. `StartupRegistry.sol` is a contract that maintains an on-chain registry of addresses related to startups in which the DAO can invest. Instead of proposing investments toward random addresses, the DAO can verify that the startup is registered by the members and is active. Only the governance, through the Timelock Controller, can register or deactivate startups.

Mock Startup. `MockStartup.sol` is a mock contract that simulates a startup receiving investments from the DAO. It is used in tests to verify the correct behavior of the investment logic and to check that the funds have been received.
The `receive()` function allows the contract to receive ETH and records the investment. The `getBalance()` function returns the current ETH balance of the contract.

Chapter 7

Experimental evaluations

This chapter experimentally evaluates the proposed model. In particular, it measures the gas consumption of the main DAO operations and compares the different versions developed, analyzing the impact of the changes introduced over time.

It also functionally studies the behavior of voting power as stake, certified competences, and the topic associated with the proposal change.

7.1 Gas Measurement

To quantitatively evaluate the evolution of the proposed architecture, benchmark tests based on gas consumption were performed on the four implementation versions introduced in Section 6.1. The comparison makes it possible to evaluate the impact of the main architectural changes introduced from V0 to V3 on the cost of the main DAO operations.

Gas consumption was chosen as the evaluation metric instead of execution time because, in Ethereum, the time required to execute and confirm a transaction mainly depends on block production and network conditions. The tests were executed on **Hardhat Network**, using the `gasUsed` value returned by the receipt of each transaction executed during the tests.

The objective of the tests is to measure the gas consumption of the main operations performed by users and of the governance cycle, while keeping the experimental scenario constant across the different implementations. In particular,

for each version, a complete DAO usage flow is simulated, including member entry, delegation of voting power, registration of the decentralized identity (DID), verification of the VCs used to certify the member's competences, creation of a proposal, voting, insertion of the proposal into the `TimelockController` queue, and its subsequent execution.

7.1.1 Experimental Setup

The experimental evaluation is based on a common scenario that is kept constant across all implementation versions, in order to allow a direct comparison of their gas consumption.

The scenario uses 10 members, each of whom joins the DAO by depositing a stake of 5 ETH. The voting power weight distribution assigns 50% to stake and 50% to competences. The quorum is set to 20%, while the superquorum is set to 70%. The `votingDelay` is 1 block, the `votingPeriod` is 50 blocks, and the Timelock delay is set to 3600 seconds. Despite these parameters, on Hardhat the blocks can be advanced artificially. All ten members participate in the vote, with 6 `For` votes and 4 `Against` votes. For each member, a VC containing two competences, `cyberSecurity` and `cloudArchitecture`, is used. The proposal used in the test requires the Treasury to invest 10 ETH in `MockStartup`.

In V2 and V3, the proposal is associated with the `FINTECH_BLOCKCHAIN` topic, identified by $\texttt{topicId} = 2$.

For each transaction, the actual `gasUsed` value is recorded, while the conversion of gas consumption into ETH and USD is performed using the same economic parameters for all versions. Based on historical data from 06/09/2025 to 06/09/2026, an average gas price of 1.03244 Gwei and an average ETH price of 2638.48 USD were calculated and used as fixed reference values. This removes the effect of market fluctuations and makes the results reproducible and comparable.

The analysis considers both the cost of individual operations and the aggregated cost of the entire flow, distinguishing between Member Activation Cost, which includes the operations required to make members operational, and Governance Cycle Cost, which includes proposal creation, voting, queueing, and execu-

tion.

The flow consists, in order, of `joinDAO`, through which all ten users join the DAO; `delegate`, through which each member self-delegates their voting power; `registerDID`, which registers the DID associated with the user; `upgradeSkillWithVC`, which verifies the VC and updates the competences used to calculate the voting power; proposal creation, performed through `propose` in V0 and V1 and through `proposeWithTopic` in V2 and V3; ten calls to `castVote`; `queue`, which adds the approved proposal to the `TimelockController` queue; and `execute`, which finally causes its execution on the Treasury. For operations executed by all users, the value reported in the graphs represents the sum of the gas consumed by the ten transactions, while `propose/proposeWithTopic`, `queue`, and `execute` are executed only once.

The benchmark calculates the Member Activation Cost as the sum of `joinDAO`, `delegate`, `registerDID`, and `upgradeSkillWithVC` for one user and for a set of ten users, while the Governance Cycle Cost includes proposal creation, the ten votes, `queue`, and `execute`.

A scalability analysis of versions V2 and V3 is also performed, measuring the gas cost as the number of skills per user and the number of members using the DAO increase, in order to compare the two versions and evaluate the impact of introducing the bitmap.

7.1.2 Gas Measurement of V0

V0 is the first version considered in the experimental comparison. It uses a single governance token to represent both the voting power derived from stake and from skills.

Under the reference scenario, the total cost of the experimental flow is 7502524 gas, corresponding to approximately 20.44 USD. Of this amount, 6193787 gas are related to the activation of the 10 members, while 1308737 gas are related to the governance cycle.

Table 7.1: Gas consumption and cost of the main operations in version V0

Operation	Gas Used	Cost (ETH)	Cost (USD)
`joinDAO`	149059	0.00015389447396	$0.4060
`delegate`	95711	0.00009881586484	$0.2607
`registerDID`	117297	0.00012110211468	$0.3195
`upgradeSkillWithVC`	254165	0.00026241011260	$0.6924
`propose`	69430	0.00007168230920	$0.1891
`castVote`	87900	0.00009075147600	$0.2394
`queue`	151280	0.00015618752320	$0.4121
`execute`	175003	0.00018068009732	$0.4767

Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48.

Table 7.2: Aggregate costs of version V0

Entry	Gas Used	Cost (USD)
Activation of the 10 members	6193787	$16.87
Governance cycle	1308737	$3.57
Estimated activation cost per member	619378	$1.69
Governance cycle share per member	130873	$0.3565
Estimated total cost per member	750252	$2.04
Total	**7502524**	**$20.44**

A test was performed to measure the gas overhead introduced by on-chain verification compared with off-chain verification based on an external Verifier and a mock `upgradeCompetence` function. This function received the verified competences from outside the contract and updated the voting power within the on-chain contract.

On-chain verification introduces an overhead of 95239 gas, corresponding to $0.2594, increasing the cost from 158914 gas to 254153 gas. Based on this estimate, it was concluded that the overhead introduced by on-chain verification was justified by the security advantages obtained.

Table 7.3: VC verification overhead in the `upgradeCompetenceWithVC` function

Operation	Gas Used	Cost (ETH)	Cost (USD)
`upgradeCompetence` mock	158914	0.00016406917016	$0.4329
`upgradeSkillWithVC`	254153	0.00026239772332	$0.6923
Overhead	**95239**	**0.00009832855316**	**$0.2594**

7.1.3 Gas Measurement of V1

In V1, the voting power of each member is no longer represented by a single token. Instead, it is derived from two separate sources: the governance token, which represents the stake-based component, and checkpoints, which store the skill-based component.

Under the reference scenario, the total cost of the experimental flow increases slightly to 7612478 gas, corresponding to approximately 20.74 USD. Member activation requires 6159782 gas, while the governance cycle reaches 1452696 gas. Compared with V0, the overall cost therefore remains very similar, but the governance cycle becomes slightly more expensive because the voting power must be retrieved and combined from two separate sources at voting time.

Table 7.4: Gas consumption and cost of the main operations in version V1

Operation	Gas Used	Cost (ETH)	Cost (USD)
`joinDAO`	149059	0.00015389447396	$0.4060
`delegate`	95689	0.00009879315116	$0.2607
`registerDID`	117275	0.00012107940100	$0.3195
`upgradeSkillWithVC`	249367	0.00025745646548	$0.6793
`propose`	69382	0.00007163275208	$0.1890
`castVote`	100979	0.00010425475876	$0.2751
`queue`	157889	0.00016301091916	$0.4301
`execute`	181611	0.00018750246084	$0.4947

Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48.

Table 7.5: Aggregate costs of version V1

Entry	Gas Used	Cost (USD)
Activation of the 10 members	6159782	$16.78
Governance cycle	1452696	$3.96
Estimated activation cost per member	615978	$1.68
Governance cycle share per member	145269	$0.3957
Estimated total cost per member	761247	$2.07
Total	**7612478**	**$20.74**

7.1.4 Gas Measurement of V2

V2, which introduces topic-based management without a bitmap, shows a more significant increase. The total cost rises to 9802172 gas, corresponding to approximately 26.70 USD, with 8177845 gas dedicated to member activation and 1624327 gas to the governance cycle. The increase is mainly due to skill management: `upgradeSkillWithVC` requires approximately 491862 gas per member in ordinary cases, making it significantly more expensive than in the previous versions. This is because, during a skill upgrade, the system no longer stores only one voting power value per user, but instead stores a different voting power value for each user and for each voting topic.

This critical cost is optimized in V3 through the use of the bitmap.

Table 7.6: Gas consumption and cost of the main operations in version V2

Operation	Gas Used	Cost (ETH)	Cost (USD)
`joinDAO`	146771	0.00015153225124	$0.3998
`delegate`	95623	0.00009872501012	$0.2605
`registerDID`	74617	0.00007703757548	$0.2033
`upgradeSkillWithVC`	491862	0.00050781800328	$1.34
`proposeWithTopic`	76425	0.00007890422700	$0.2082
`castVote`	112171	0.00011580982724	$0.3056
`queue`	174653	0.00018031874332	$0.4758
`execute`	217515	0.00022457118660	$0.5925

Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48.

Table 7.7: Aggregate costs of version V2

Entry	Gas Used	Cost (USD)
Activation of the 10 members	8177845	$22.28
Governance cycle	1624327	$4.42
Estimated activation cost per member	817784	$2.23
Governance cycle share per member	162432	$0.4425
Estimated total cost per member	980217	$2.67
Total	**9802172**	**$26.70**

7.1.5 Gas Measurement of V3

V3 represents the final implementation of the proposed architecture and is therefore analyzed in greater detail. This version maintains the topic-based governance mechanism introduced in V2, while optimizing skill storage through the use of a bitmap representation.

In addition to the analysis of the main DAO operations and aggregated costs, a scalability analysis is performed to evaluate the effectiveness of the bitmap optimization.

Table 7.8: Gas consumption and cost of the main operations in version V3

Operation	Gas Used	Cost (ETH)	Cost (USD)
`joinDAO`	152905	0.00015786523820	$0.4165
`delegate`	98117	0.00010129991548	$0.2673
`registerDID`	87661	0.00009050472284	$0.2388
`upgradeSkillWithVC`	403614	0.00041670723816	$1.0995
`proposeWithTopic`	96590	0.00009972337960	$0.2631
`castVote` For	115051	0.00011878325444	$0.3134
`castVote` Against	116432	0.00012020905408	$0.3172
`queue`	174653	0.00018031874332	$0.4758
`execute`	217515	0.00022457118660	$0.5925

Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48. Voting scenario: 10 members, 6 For and 4 Against.

The results show that joining the DAO requires an average of 152905 gas, corresponding to approximately 0.4165 USD, while self-delegation requires an average of 98117 gas, corresponding to approximately 0.2673 USD. DID registration requires an average of 87661 gas, corresponding to approximately 0.2388 USD. Among the member activation operations, the most expensive operation is `upgradeSkillWithVC`, which requires an average of 403614 gas, corresponding to approximately 1.10 USD.

This result is consistent with the amount of processing required for credential management, which includes the processing of EIP-712 data and the on-chain update of skill information and the related checkpoints.

The creation of the proposal through `proposeWithTopic`, instead, has a relatively low cost of 96590 gas, approximately 0.26 USD. The voting operation requires an average of 115051 gas for a `For` vote and 116432 gas for an `Against` vote, corresponding to approximately 0.3134 USD and 0.3172 USD, respectively. Finally, `queue` and `execute` require 174653 gas and 217515 gas, with estimated costs of approximately 0.48 USD and 0.59 USD, respectively.

Table 7.9: Aggregate costs of version V3

Entry	Gas Used	Cost (USD)
10 Member Activation Cost	7422954	$20.22
Governance Cycle Cost	1644792	$4.48
Per-Member Activation Estimate	742295	$2.02
Per-Member Cycle Share Estimate	164479	$0.4481
Per-Member Total Estimate	906774	$2.47
TOTAL GAS (activation + cycle)	**9067746**	**$24.70**

Considering the aggregated values, the activation phase of the ten members, including joining the DAO, delegation, DID registration, and competence verification, requires a total of 7422954 gas, with an estimated cost of approximately 20.22 USD. A complete governance cycle, from proposal creation to its execution, requires 1644792 gas, corresponding to approximately 4.48 USD. The entire experimental scenario therefore requires 9067746 gas, with an estimated total cost of approximately 24.70 USD.

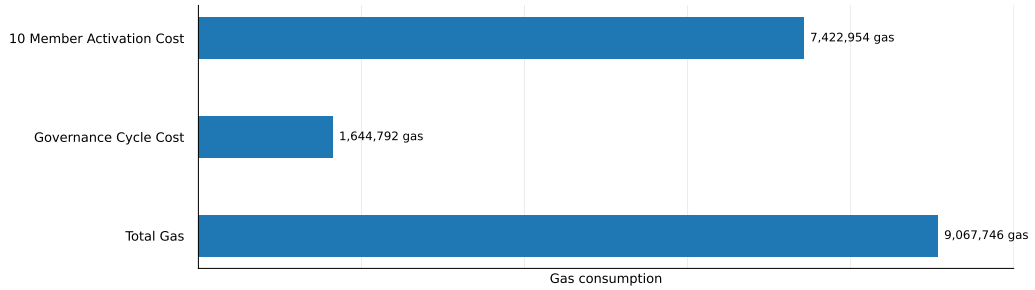


Figure 7.1: Aggregated gas consumption for the activation of ten DAO members, a complete governance cycle, and the overall experimental scenario.

The result highlights that the main cost is not related to the creation or execution of proposals, but to the skill upgrade phase of a member. Considering all ten members, the `upgradeSkillWithVC` operations require a total of 4036136 gas, representing approximately 44.5% of the total gas consumed by the entire experimental scenario.

However, this cost is not recurring, since it is executed only once for each

competence upgrade of a member.

Moreover, the cost is justified by the security advantages obtained by verifying a member's Verifiable Credentials on-chain, rather than relying on an off-chain Verifier, which may introduce trustability issues.

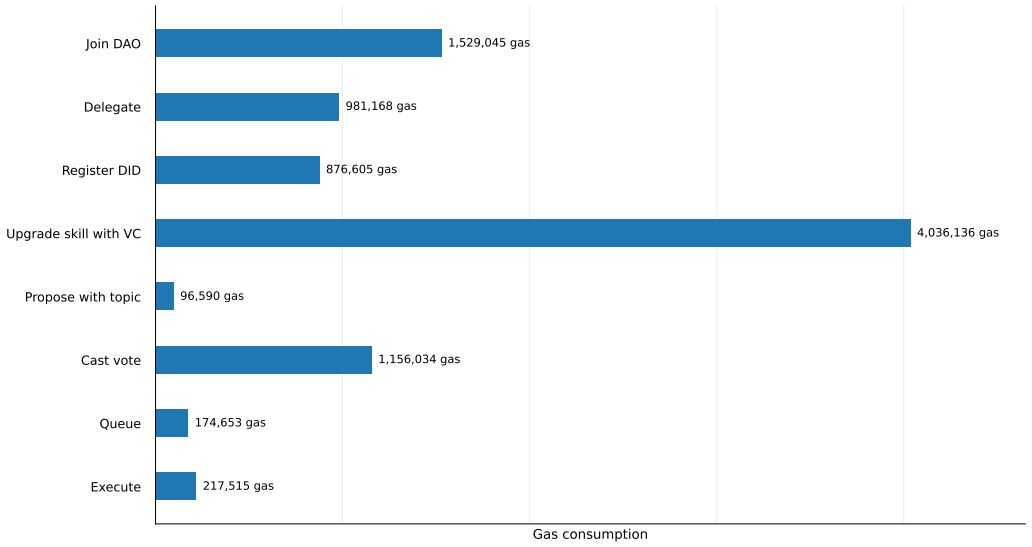


Figure 7.2: Aggregated gas consumption for the main operations performed by ten DAO members, showing that the `upgradeSkillWithVC` operation accounts for the largest share of the total cost.

7.1.6 Bitmap and No-Bitmap Comparison

To evaluate the effectiveness of the optimization introduced in V3, a direct comparison was performed between skill management using a bitmap and the no-bitmap version used in V2. The bitmap optimizes the storage of the skills owned by each user, which are represented by a mapping that associates each address with an integer, instead of using two mappings. The first mapping associates each address with an array of hashes of skill identifiers, while the second associates each address with a submapping that maps each skill to a boolean value, allowing O(1) verification of whether a user owns a specific skill.

The comparison considers only the cost associated with skill management, while keeping all the other benchmark parameters unchanged. Two experiments were performed: the first analyzes how the cost of `upgradeSkillWithVC` changes

as the number of skills increases, while the second measures the cumulative cost as the number of users increases, keeping 8 skills per user fixed.

Scalability with Respect to the Number of Skills

The results show that the advantage of the bitmap increases rapidly with the number of skills managed. With 1 skill, the bitmap version requires 450724 gas, compared with 494712 gas for the no-bitmap version, which is therefore approximately 9.76% more expensive. With 8 skills, the consumption is respectively 474131 gas and 824100 gas, with an overhead of +73.81% for the no-bitmap solution.

The difference becomes even more evident as the number of skills increases further. With 16 skills, the no-bitmap version consumes 138.08% more gas, while with 32 skills the difference reaches 244.64%, and finally with 64 skills it reaches +399.83%, with consumption increasing from 692323 gas with the bitmap to 3460439 gas without the bitmap. The result therefore shows that the bitmap representation becomes progressively more advantageous as the number of skills increases.

Table 7.10: Scalability comparison between bitmap and non-bitmap representations

#Skills	Bitmap Gas	NoBitmap Gas	Delta Gas	Delta	Bitmap USD	NoBitmap USD
1	450724	494712	43988	9.76%	$1.23	$1.35
2	454033	541756	87723	19.32%	$1.24	$1.48
4	460724	635858	175134	38.01%	$1.26	$1.73
8	474131	824088	349957	73.81%	$1.29	$2.24
16	504279	1200621	696342	138.09%	$1.37	$3.27
32	566919	1953789	1386870	244.63%	$1.54	$5.32
64	692311	3460439	2768128	399.84%	$1.89	$9.43

Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48.

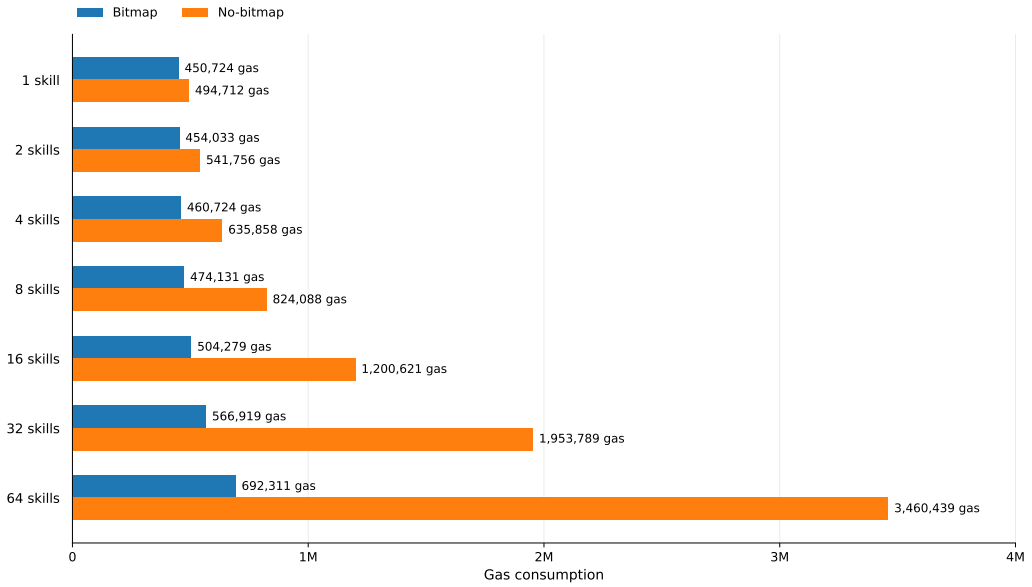


Figure 7.3: Gas consumption comparison between bitmap and non-bitmap skill representations as the number of managed skills increases, highlighting the increasing scalability advantage of the bitmap-based solution.

Scalability with Respect to the Number of Users

The second experiment keeps 8 skills per user fixed and increases the number of users from 1 to 64. With only one user, the bitmap version requires 474131 gas, while the no-bitmap version requires 824100 gas, making it 73.81% more expensive. With 8 users, the total cost is 3389376 gas with the bitmap and 6189128 gas without the bitmap.

The difference remains evident even as the number of users increases further. With 64 users, the total cost reaches 26711348 gas in the bitmap version and 49109328 gas in the no-bitmap version. The average cost per user is therefore approximately 417364 gas with the bitmap and 767333 gas without the bitmap: the no-bitmap solution requires approximately 83.85% more gas.

It should be noted that the delta remains constant, meaning that the percentage overhead caused by not using the bitmap remains approximately fixed around 80% and does not change as the number of users increases, but scales only as the number of skills increases.

Table 7.11: Comparison of cumulative cost between bitmap and non-bitmap representations as the number of users increases

#Users	Bitmap Total Gas	NoBitmap Total Gas	Delta	Bitmap USD	NoBitmap USD
1	474131	824076	73.81%	$1.29	$2.24
2	890598	1590536	78.59%	$2.43	$4.33
4	1723520	3123408	81.22%	$4.69	$8.51
8	3389352	6189104	82.60%	$9.23	$16.86
16	6721040	12320556	83.31%	$18.31	$33.56
32	13384440	24583544	83.67%	$36.46	$66.97
64	26711348	49109340	83.85%	$72.76	$133.78

Each user has 8 skills. Reference gas price: 1.03244 Gwei. Reference ETH price: $2638.48.

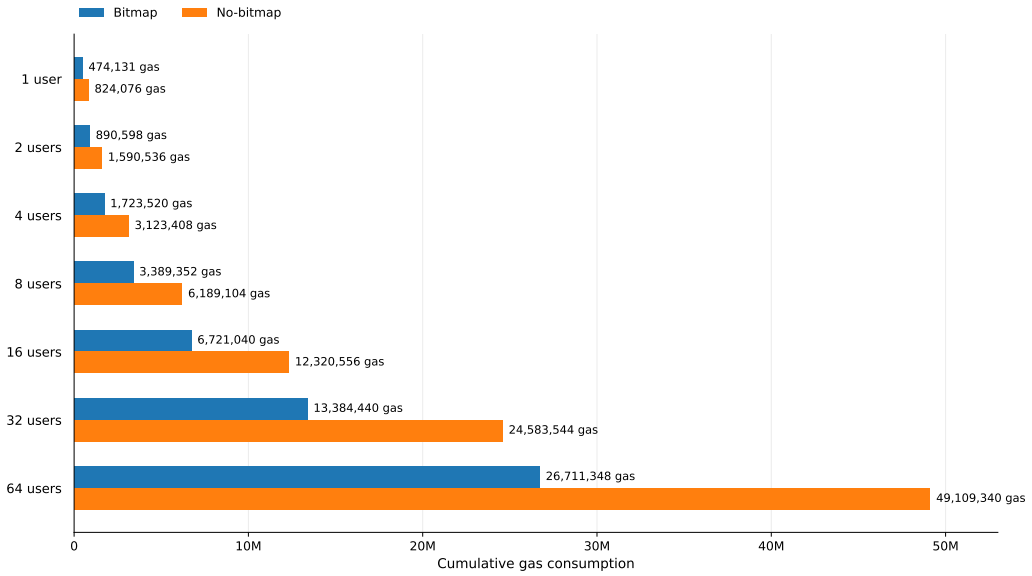


Figure 7.4: Comparison of cumulative gas consumption between bitmap and non-bitmap skill representations as the number of users increases, with 8 skills fixed per user, highlighting the persistent gas-efficiency advantage of the bitmap-based solution.

7.2 Analytical Evaluation of Voting Power Behavior

In addition to the evaluation of gas consumption, a second series of experiments was performed with the objective of verifying how the proposed model concretely

distributes voting power among DAO members, by measuring the behavior of the voting formula as stake, certified competences, and the proposal topic change.

7.2.1 Influence of Stake and Competence Weights

The first experiment analyzes the effect of the `weightStake` and `weightSkill` parameters on the distribution of voting power. The topic is kept fixed on AI & Data, while five configurations are compared: 100/0, 75/25, 50/50, 25/75, and 0/100, where the first value represents the weight of stake and the second the weight of competences.

The dataset is composed of 10 members (U1, U2,...U10) with different stake values and skills, in order to include both users with high economic power but limited competence on the topic, and users with lower stake but more relevant skills.

Table 7.12: Profiles used in the voting power experiment

Profile	**Stake (ETH)**	**Certified skills**
U1	100	blockchain
U2	20	machineLearning + dataEngineering
U3	20	blockchain + startupFinance
U4	40	softwareArchitecture + cloudArchitecture
U5	30	cyberSecurity + cloudArchitecture
U6	15	machineLearning
U7	15	dataEngineering
U8	10	startupFinance
U9	10	distributedSystems
U10	5	Nothing

The results clearly show the transition from a governance model based exclusively on capital to a governance model that also considers competences. For example, user U1 has the highest stake in the group, equal to 100 ETH, but only possesses the `blockchain` skill, which produces a skill score of 5 for the AI & Data topic. With the token-only 100/0 configuration, U1 therefore has 100

VP. With the 50/50 configuration, the voting power decreases to 52.5 VP, until it reaches 5 VP in the limiting 0/100 case.

In contrast, U2 has only 20 ETH, but possesses the `machineLearning` and `dataEngineering` skills, which produce an AI/Data skill score of 75. The voting power therefore increases from 20 VP in the token-only model to 47.5 VP with 50/50 weights, and reaches 75 VP in the 0/100 case.

The same behavior can be observed for the other profiles. As the weight of the skill component increases, a user with a small amount of ETH but with skills that are highly relevant to the topic can obtain a significant voting power compared with users who have a large stake but skills that are less relevant to the topic. Conversely, when the stake weight is increased relative to the weight assigned to skills, the opposite behavior occurs.

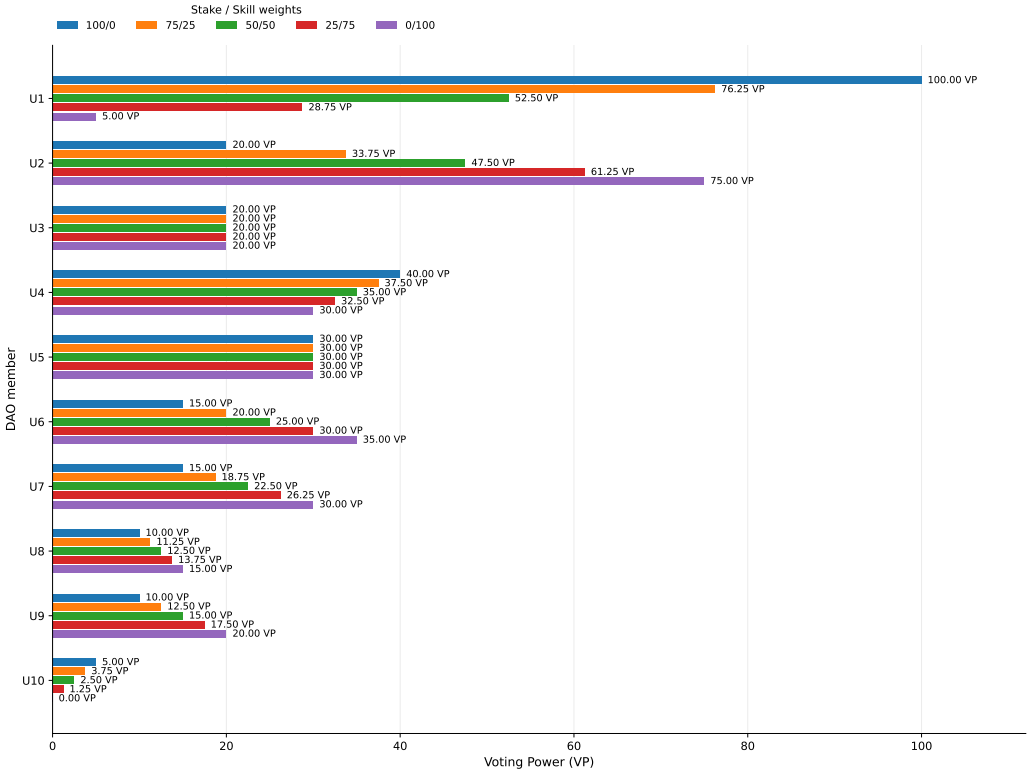


Figure 7.5: Voting power of the ten member profiles under different combinations of `weightStake` and `weightSkill`, with the proposal topic fixed to AI & Data.

7.2.2 Whale and expert minority

The second experiment analyzes a scenario in which a single member, referred to as the Whale, owns a much larger amount of stake than the other users, while a minority of members possesses competences that are particularly relevant to the proposal. The topic is again AI & Data.

The Whale owns 60 ETH and only the `blockchain` skill, which is not highly relevant to the considered topic. The three expert members, instead, collectively own only 25 ETH, but possess skills that are critical for the topic: Expert A has 10 ETH and the `machineLearning` and `dataEngineering` skills, Expert B has 10 ETH and the `machineLearning` skill, while Expert C owns 5 ETH and the `dataEngineering` and `distributedSystems` skills.

Table 7.13: Profiles used in the Whale and expert minority experiment

Profile	**Stake (ETH)**	**Skills certified**
Whale	60	blockchain
Expert A	10	machineLearning + dataEngineering
Expert B	10	machineLearning
Expert C	5	dataEngineering + distributedSystems
Other 1	5	softwareArchitecture
Other 2	4	cloudArchitecture
Other 3	3	startupFinance
Other 4	2	cyberSecurity
Other 5	1	Nothing
Other 6	1	blockchain

With a fully token-based governance model, the Whale alone controls 59.41% of the total voting power, while the three experts together reach only 24.75%. By progressively introducing the contribution of competences, this distribution changes significantly. With 75/25 weights, the Whale's share decreases to 34.71%, while the experts' share increases to 44.10%. In the 50/50 configuration, used as the main configuration of the system, the Whale's share decreases to 19.64%, while the three experts together reach 55.90% of the total voting power. With 25/75 weights, the two shares become 9.48% and 63.85%, respectively.

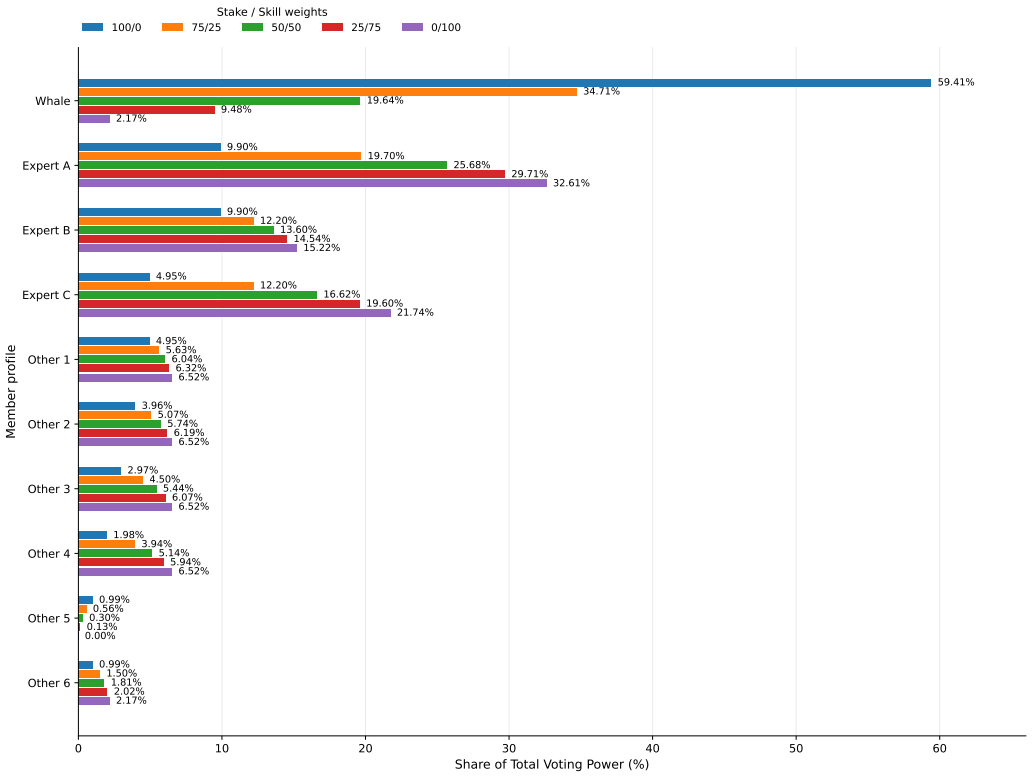


Figure 7.6: Share of total voting power of the Whale, the three expert members, and the remaining members under different combinations of `weightStake` and `weightSkill`, with the proposal topic fixed to AI & Data.

This result shows the main effect targeted by the proposed architecture: by changing the weights assigned to the stake and skill components of the voting power, a member with a large stake but competences that are not very relevant to a specific proposal can lose part of their influence in favor of members with a smaller economic stake but greater competence on the topic.

7.2.3 Sensitivity to the Proposal Topic

The third experiment verifies whether the voting power derived from competences actually changes depending on the topic of the proposal. To isolate this variable, six profiles with the same stake, equal to 40 ETH, are used, while the weights are kept fixed at 50% for stake and 50% for skills. Only the competences of the members and the topic used in the voting power calculation change. The four

topics considered are AI & Data, Cloud & Cybersecurity, FinTech & Blockchain, and Enterprise Software.

Table 7.14: Profiles used in the topic sensitivity experiment

Profile	**Stake (ETH)**	**Skills certified**
AI/Data expert	40	machineLearning + dataEngineering
Cloud/Security expert	40	cyberSecurity + cloudArchitecture
FinTech/Web3 expert	40	blockchain + startupFinance
Enterprise architect	40	softwareArchitecture + cloudArchitecture
Cross-domain engineer	40	distributedSystems + cloudArchitecture
No certified skill	40	Nothing

The results confirm that, with the same stake, the voting power changes according to the relevance of the competences. The AI/Data expert profile, with `machineLearning` and `dataEngineering`, reaches the maximum value of 57.5 VP on the AI & Data topic, while it decreases to 30 VP on the FinTech & Blockchain topic. Similarly, the FinTech/Web3 expert profile, with `blockchain` and `startupFinance`, reaches 57.5 VP on FinTech & Blockchain but only 30 VP on AI & Data. The Enterprise architect, with `softwareArchitecture` and `cloudArchitecture`, also reaches the maximum value of 57.5 VP on the Enterprise Software topic.

An important control case is represented by the No certified skill member. Since all profiles own 40 ETH and stake has a weight of 50%, this user always has 20 VP, independently of the topic. Its value therefore remains 20 for AI & Data, Cloud & Cybersecurity, FinTech & Blockchain, and Enterprise Software.

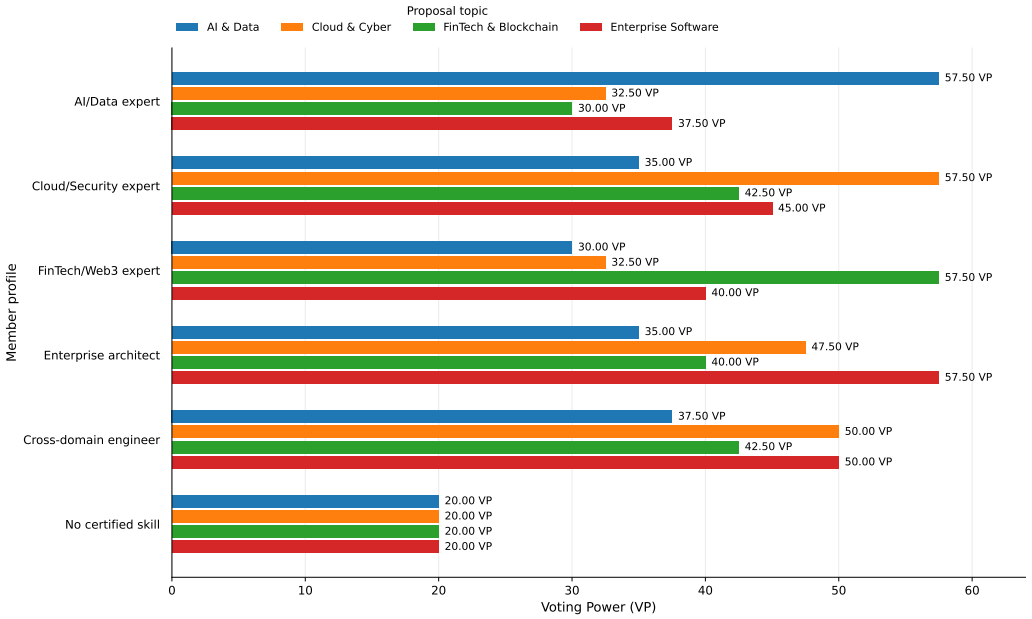


Figure 7.7: Voting power of six expert profiles across different proposal topics, with the stake fixed at 40 ETH for all members and both `weightStake` and `weightSkill` fixed at 50%.

The experiment confirms that the same user, with the same fixed skills, can have different voting power values for proposals belonging to different topics.

Chapter 8

Security Analysis

SAGE manages data and operations that are particularly sensitive from a security perspective. In particular, it controls the voting power of DAO members, the VCs that certify users'competences, the lifecycle of governance proposals, and the ETH stored in the DAO Treasury.

The objective of this chapter is to analyze the security mechanisms used to protect these data and operations, also considering the main remaining security limitations. The analysis is supported by the test suite developed to verify the security and the correct behavior of the different smart contracts.

The chapter first introduces the security assumptions considered in the analysis. It then analyzes, from a security perspective, the mechanisms used to protect the voting power derived from stake and competences, the relationship between members and their corresponding DIDs, the verification of VCs, and the management of Issuers recognized as trusted are then examined. The analysis also considers the security of topic-based governance and the mechanisms used to protect the funds stored in the Treasury. Finally, the main remaining limitations of the prototype are discussed, with particular attention to credential revocation and the privacy of data transmitted on-chain.

8.1 Actors and Security Assumptions

As described in Section 4.3, SAGE involves several actors and system components with different roles. The main actors considered in the security model are DAO members, users external to the DAO, trusted Issuers, the deployer, and the startups. The analysis also considers the security implications of the main system components, including DAO governance, `TimelockController`, and Treasury.

The security analysis assumes that:

1. DAO members and external users may behave maliciously. For example, a member may attempt to exceed the stake limits, present a credential issued to another Holder, or modify the information contained in a VC.

2. Trusted Issuers correctly issue credentials and keep control of their private keys. Similarly, DAO members are assumed to securely protect the private keys associated with their accounts.

3. During the bootstrap phase, the deployer holds a privileged position because it is responsible for the initial configuration of some DAO components. The security analysis therefore assumes that the deployer behaves correctly during this phase. After the bootstrap phase, the deployer loses its privileges and is no longer considered trusted.

8.2 Deployer Privileges and Risks During Bootstrap

During the bootstrap phase, the deployer has some privileges required for the initial configuration of the DAO. In particular, the deployer can configure the Treasury address and set the first trusted Issuer.

The implementation limits the possibility of abusing these privileges after the initial configuration. The Treasury address can be set only once and cannot later be modified by the deployer. Similarly, the deployer can configure only the first trusted Issuer; after this operation, the addition or removal of Issuers can be performed only through a governance proposal.

However, these restrictions do not protect the system from a malicious deployer during the initial configuration, who could set an untrusted Treasury address or an untrusted first Issuer. These abuses could redirect DAO funds to an address controlled by the deployer or allow the acceptance of credentials issued by an untrusted Issuer, assigning invalid competences to members. For this reason, the correct behavior of the deployer during the bootstrap phase remains a strong security assumption of the prototype.

8.3 Protection of Stake-Based Voting Power

The economic component imposes a maximum limit on the amount of tokens that can be minted and on the amount of stake that can be deposited. One security objective of SAGE is to prevent the same member from depositing funds, transferring the obtained tokens to another account, and then depositing again in order to bypass the maximum mintable stake limit. If the deposit limit were calculated based on the current balance of stake tokens owned by a member, the limit could be easily bypassed. A member could transfer the tokens already obtained to another account, thereby reducing their own balance, and then deposit additional ETH to receive new tokens. In this way, the same member could deposit a total amount of ETH greater than the intended limit and obtain a voting power higher than the allowed amount.

To address this problem, `GovernanceToken` does not determine the deposit limit based on the current balance of the ERC-20 token, but instead stores, for each address, the cumulative value of the ETH stake deposited. For this reason, `increaseStake()` checks this value against the maximum deposit limit. As a result, transferring previously minted tokens does not reset the amount already deposited by the address.

This anti-bypass property is directly verified by the tests. In particular, one scenario considers a member who initially deposits ETH, transfers all the generated stake tokens to another member, and then attempts to deposit an amount that would cause the cumulative stake to exceed the maximum allowed value. The transaction is rejected even if the original account no longer owns the previously transferred tokens.

The implementation also prevents token transfers and voting power delegation to users who are not members of the DAO. The internal hook `_update()` rejects transfers to non-members, while `_delegate()` verifies that both the delegator and the delegation recipient are DAO members. The tests cover direct transfers, `transferFrom()`, and token delegation, verifying that voting power cannot simply be transferred to an external address that has never joined the DAO.

These mechanisms do not provide complete protection against Sybil attacks. DAO membership is permissionless and is associated with an Ethereum address, not with a unique physical person. The deposit limit therefore constrains the amount of voting power that can be minted by a single address, but not necessarily the total voting power controlled by the same person in the real world. A subject controlling multiple addresses could join the DAO and mint tokens up to the deposit limit with each address they control.

These security issues are not introduced by the competence-based model proposed by SAGE, but generally concern the management of membership and governance tokens in DAOs. Nevertheless, SAGE introduces specific controls to mitigate these risks.

In the case of Sybil attacks, the mechanism proposed by SAGE can make the attack more difficult, since an attacker controlling multiple accounts would need to obtain valid credentials for each DID associated with those accounts. The presence of credentials containing the same certified claims but associated with different DIDs could represent an indication of a possible attack.

8.4 Security of Historical Voting Power After a Snapshot

Another important type of attack concerns the modification of voting power after the beginning of a proposal. For example, a member could attempt to increase their stake or verify new competences after observing the progress of a vote, in order to influence it with increased voting power.

The proposed system mitigates this problem through historical checkpoints. The stake component inherits the checkpoint mechanism from `ERC20Votes`, while

`GovernanceSkill` maintains checkpoints for the skill component. To address this problem, the `Governor` retrieves both components at the proposal snapshot instead of using their current values.

The test suite verifies both cases. Skills assigned after the proposal snapshot do not increase the voting power related to that proposal and, in the same way, stake added after the snapshot does not increase the voting power.

This security issue is not introduced by the competence-based voting model proposed by SAGE, but represents a general concern in DAO governance. Nevertheless, SAGE mitigates this risk by applying the snapshot mechanism to both stake-based and competence-based voting power.

8.5 DID Binding and Holder Authentication

A possible attack consists of registering a DID that does not correspond to one's own identity. Before using a VC, a member must register a DID. SAGE supports a limited representation of **did:ethr**, in which the Ethereum address of the controller is encoded directly inside the DID. During registration, the contract extracts the address from the DID and requires it to correspond to `msg.sender`. The DID is then stored in hashed form and permanently associated with the member's address.

SAGE's design prevents a member from registering a DID controlled by another Ethereum account and prevents a member from replacing their DID with another one after the first registration.

The tests we conducted explicitly verify these properties. DID registration requires DAO membership, rejects empty or malformed DIDs, prevents subsequent replacement, and verifies that the controller contained in the DID corresponds to the caller.

Another attack is also prevented by the DID verification mechanism implemented in SAGE. An attacker could present a VC that certifies competences for an identity that does not correspond to their DID. When a credential is presented, `upgradeSkillWithVC()` compares the hash of the `credentialSubject.id` contained in the credential with the DID previously registered by the caller. If they do not match, the verification fails. Therefore, a credential issued for a given

Holder cannot be presented by a different account. The test suite verifies that a mismatch between the registered DID and the DID of the credential Holder causes the upgrade to be rejected.

8.6 Addition and Removal of Trusted Issuers

The deployer configures the Treasury address only once as the recipient of the funds deposited into the DAO and also sets the first trusted Issuer of the credentials. Subsequent trusted Issuers can only be added through the governance process, with the corresponding operation executed by the `TimelockController`. SAGE prevents duplicate Issuers, zero addresses, and the removal of the last remaining trusted Issuer. The corresponding tests verify these conditions and check that, after the bootstrap phase, a normal member cannot add addresses to the list of trusted Issuers.

8.7 VC Integrity

A possible attack consists of a user modifying the fields of a VC presented to the DAO in order to upgrade their skills. The integrity of the information related to competences is protected through EIP-712 signatures. The `VPVerifier` library reconstructs the typed-data hash of the credential and uses the OpenZeppelin ECDSA utilities to recover the address that produced the signature.

Modifying any of the fields included in the signed credential produces a digest different from the one originally signed by the Issuer. As a result, the original signature is no longer valid for the new digest, causing the credential verification to fail.

After recovering the signer, the `GovernanceSkill` contract verifies that the recovered Ethereum address belongs to the set of trusted Issuers. If the address of the Issuer recovered that signed the credential is not included in this set, the competence upgrade operation is considered invalid.

The test suite uses real EIP-712 credentials generated by the off-chain Veramo infrastructure used for credential management. A set of 13 credentials is verified

by presenting them to the on-chain verifier and then comparing the calculated topic-specific voting power with the expected values. The tests also modify a signed credential after the signature has been generated and verify that the altered credential is rejected.

Another useful property is *idempotency*. The skills owned by a member are internally represented through a bitmap and are aggregated using a bitwise OR operation. As a result, repeatedly presenting the same valid credential does not duplicate the member's skills and does not increase either the individual voting power or the total skill supply.

This property also holds when two different trusted Issuers issue credentials certifying the same skills for the same member, as the bitwise OR operation ensures that each skill is counted only once.

The test suite explicitly verifies this property. Skills that are not recognized by the DAO scoring system are also rejected before the member state is updated. The tests verify also that a credential containing an unsupported skill causes the entire operation to revert without partially modifying the member's bitmap or voting power.

8.8 Security of Topic-Aware Governance

In SAGE, each proposal must be associated with one of the supported topics. The standard `propose()` function is disabled, and users must instead use `proposeWithTopic()`. This prevents the creation of a proposal without topic information, which is necessary to select the correct voting power derived from skills. In addition, invalid topic identifiers are rejected. The corresponding tests verify both the association of the topic with the proposal and the rejection of calls to the standard proposal creation interface.

8.9 Treasury Security and Reentrancy Protection

The Treasury is the contract characterized by the most direct financial risk, since it stores the ETH deposited by DAO members. The Treasury allows investments

only when the caller is the `TimelockController`, after the DAO has approved the corresponding decision through the governance process. Neither the deployer nor DAO members can directly transfer Treasury funds to another address. An investment must first be encoded as a governance proposal, approved through voting, added to the Timelock queue, and executed after the configured delay.

The tests explicitly verify this restriction by attempting to call `investStartup()` from a member account and expecting the operation to revert. Investments with a zero amount, insufficient Treasury balance, addresses not present in the StartupRegistry, addresses of startups marked as inactive, and non-existing startup identifiers are also tested.

The `investStartup()` function also inherits `ReentrancyGuard` and applies the `nonReentrant` modifier. The internal state is updated before the external ETH transfer is executed, following the Checks-Effects-Interactions pattern. As a result, a malicious recipient contract cannot re-enter `investStartup()` while the first investment is still being executed. If the external transfer fails, the entire transaction is reverted, including the update of the `investedIn` variable.

These protections address general risks associated with smart contracts performing external calls and ETH transfers, rather than risks specifically introduced by SAGE. SAGE therefore relies on standard Solidity security practices to mitigate them.

The use of the Startup Registry also limits the possible destinations of transfers performed by the Treasury. An arbitrary wallet cannot be designated as the recipient within an investment proposal, but must belong to the `StartupRegistry`

8.10 Residual Risks and Limitations

Despite the protections described above, the current prototype still presents some residual security limitations.

A first limitation concerns the absence of a mechanism in SAGE for *revoking* a credential that has already been used. The signed data of a VC include an `issuanceDate` field and an `expiration date`, which limits the period during which the credential remains valid. However, SAGE does not provide a mechanism to revoke a credential before its expiration. As a consequence, there are no

mechanisms to automatically remove the skills obtained through that credential from the member's skill bitmap and to update the corresponding voting power accordingly.

The second limitation concerns *privacy*, which remains constrained by the characteristics of a public blockchain. Much of the information linked to the user is publicly observable on-chain.

For example, the contract does not directly store the member's DID, but only its hash. However, this does not provide anonymity. The DID format accepted by the prototype is deterministic and corresponds to `did:ethr:0x<address>`. The contract directly extracts the Ethereum address from the DID and verifies that it matches `msg.sender`. Since `msg.sender` is publicly visible as the sender of the registration transaction, an observer can directly identify the Ethereum address associated with the member and reconstruct the corresponding DID. Moreover, the `DIDRegistered` event also exposes the member's Ethereum address.

During registration, the DID is also provided in plaintext as a parameter of the `registerDID()` function. As a result, even if only its hash is stored in contract storage, the original DID string remains observable in the transaction calldata.

A similar limitation concerns the presentation of VCs. The upgradeSkillWithVC() function receives the credential in plaintext as transaction input. Since the information contained in the VC is transmitted through a transaction on a public blockchain, it is visible in the calldata.

The skills owned by the member are also not private. The bitmap that stores the skills of a member is public. As a result, by observing the bitmap value, it is possible to deterministically reconstruct the competences associated with a specific member.

Public checkpoints also make it possible to observe the voting power of a member for a specific topic and its evolution over time.

In addition, the stake deposited by a given member and the amount of tokens owned by that member can also be observed.

Finally, the prototype does not implement *selective disclosure* mechanisms. To verify a competence, the entire `CredentialSubject` is presented, rather than only the information strictly required for the calculation of voting power. A system more focused on privacy could instead allow a member to prove possession of

a specific certified competence without revealing the other attributes contained in the VC, or to prove possession of a competence without revealing the competence itself, through *zero-knowledge proof* techniques.

The integration of selective disclosure and zero-knowledge proof techniques is therefore left for future work.

Chapter 9

Conclusions and Future Work

9.1 Conclusions

This thesis addressed some of the limitations of DAO governance models in which the voting power of members mainly depends on the amount of governance tokens owned or on the capital invested in the organization. Although these mechanisms make it possible to represent decision-making power in a simple way, the amount of capital invested by a member does not necessarily represent their ability to correctly evaluate proposals belonging to a specific domain.

To address this problem, the thesis proposed SAGE, a governance model in which the voting power of a member depends both on the capital deposited and on the competences they possess. The main idea is that the influence of a member within the decision-making process should not depend exclusively on their economic contribution, but should also take into account the competences that are relevant to the topic of the decision.

The proposed model therefore separates voting power into two components. The stake-based component represents the economic participation of the member in the DAO, while the competence-based component represents the contribution derived from the certified competences of the member. The two components are combined through configurable weights.

The component derived from competences also depends on the topic of the proposal. Each proposal is associated with a specific topic, and the same com-

petence can contribute differently to the voting power depending on the domain of the decision. As a result, the same member can have different voting power in proposals belonging to different topics.

To make the competences declared by members verifiable, the architecture integrates the concepts of Self-Sovereign Identity and Verifiable Credentials. The competences are certified by external entities recognized by the DAO as trusted Issuers. A member can present to the DAO a VC containing their certified competences. The DAO cryptographically verifies the credential on-chain and uses the verified competences to calculate the voting power.

The SAGE architecture is organized into five main protocols: the DAO Membership Protocol, which manages the entry and membership of members in the DAO; the Stake Protocol, which manages stake deposits and the related voting power component; the Competence Certification Protocol, which manages DID registration, the verification of Verifiable Credentials, and the update of certified competences; the Governance Protocol, which manages proposal creation, voting power, voting, quorum, and superquorum; and the Investment Execution Protocol, which manages the execution of investment decisions approved by the governance through the Timelock Controller and the Treasury.

The proposed model was implemented on Ethereum using Solidity and the libraries provided by OpenZeppelin. The Veramo framework was used for the off-chain management and issuance of Verifiable Credentials, while EIP-712 signatures were used to enable their cryptographic verification by the smart contracts.

The component of voting power derived from stake was implemented through `ERC20Votes`, while the component derived from competences is maintained through separate checkpoints for each topic. In this way, both components can be retrieved at the timepoint associated with the snapshot of a proposal, preventing changes to stake or competences made after the beginning of the voting process from retroactively modifying the voting power used for that proposal.

During development, four main versions of the system were implemented. Version V0 initially used a single governance token to represent both the component derived from stake and the component derived from competences. Version V1 separated the two components, maintaining the competence-based component through dedicated checkpoints. Version V2 introduced the topic mechanism, al-

lowing competences to have a different value depending on the topic of the proposal. Finally, version V3 introduced a bitmap-based representation of competences, with the objective of reducing storage costs and the gas associated with skill management.

The experimental evaluation analyzed both the computational cost of the system and the behavior of the proposed voting power model. Regarding gas consumption, the main DAO operations and the cost of the entire governance cycle were measured, comparing the different versions developed. The optimization introduced in version V3 through the use of the bitmap was also evaluated. The results show that the benefit of this solution becomes increasingly evident as the number of managed competences increases.

A second part of the evaluation analyzed the behavior of the voting power formula, showing how the variation of the weights assigned to the stake and competence components affects the distribution of voting power among the members. The experiments also confirmed that, with the same stake, the contribution of competences varies according to the topic of the proposal, allowing the same member to have different voting power in decisions belonging to different domains.

Finally, the security analysis evaluated the main risks associated with the management of stake, certified competences, and the governance process, verifying the countermeasures introduced in the implementation. The system uses the cumulative value of the deposited stake to prevent the maximum limit from being bypassed through token transfers, while checkpoints make it possible to determine the voting power with respect to the proposal snapshot, preventing subsequent changes to stake or competences from affecting a vote that has already started.

The binding between the member's Ethereum account and the corresponding DID, together with the verification of the Credential Holder's DID, also prevents the use of credentials associated with another member.

The integrity and origin of the VCs are guaranteed through the on-chain cryptographic verification of the credential and by checking that the signer belongs to the set of trusted Issuers recognized by the DAO. Access to the Treasury funds is instead allowed exclusively through the execution, by the Timelock Controller, of decisions approved by the governance. The investment function also includes protection against reentrancy attacks.

Overall, this work proposes a governance model in which certified competences are directly integrated into the calculation of DAO voting power. Compared with approaches in which VCs are mainly used to determine the eligibility of a member to participate in a vote, SAGE instead uses certified competences as a quantitative component of voting power, whose contribution varies according to the topic of the proposal.

9.2 Future Work

The prototype developed in this thesis implements the main mechanisms required to integrate certified competences into the governance process of a DAO. However, some aspects remain open and could be extended in future versions of the system.

Revocation and Expiration of Verifiable Credentials. The current implementation does not include a mechanism to manage the revocation or expiration of VCs that have already been used by a member. As a result, a credential could continue to contribute to the voting power even if it is no longer valid.

A possible future development therefore consists of introducing expiration and credential status mechanisms, in order to verify the validity of credentials over time and update the member's competences and voting power accordingly.

Selective Disclosure. A further limitation concerns the amount of information revealed when presenting a VC. In the current implementation, the entire CredentialSubject is presented, even when only part of the information contained in the credential is required for the calculation of voting power.

A possible extension consists of introducing Selective Disclosure mechanisms, allowing members to reveal only the attributes required for verification. This would reduce the amount of information made public and make the system more consistent with the data minimization principles of SSI.

Zero-Knowledge Proofs. A possible evolution concerns the use of Zero - Knowledge Proofs. In SAGE, the verification of competences requires the on-chain pre-

sentation of the information contained in the credential.

The use of Zero-Knowledge Proofs could allow a member to prove possession of certified competences without directly revealing the credential or the information contained in it.

SAGE shows that SSI and Verifiable Credentials can be effectively integrated into DAO governance to make certified competences part of voting power, while future work can further improve privacy and the management of credential validity.

9.3 Declaration of Generative AI and AI-assisted technologies in the writing process

We acknowledge the use of AI-based tools, such as Grammarly and ChatGPT, to refine the writing. After using these tools, we reviewed and edited the content as needed and take full responsibility for the content of this thesis.

Bibliography

[1] Partha Pratim Ray. "Web3: A Comprehensive Review on Background, Technologies, Applications, Zero-Trust Architectures, Challenges and Future Directions". In: *Internet of Things and Cyber-Physical Systems* 3 (2023), pp. 213–248. DOI: 10.1016/j.iotcps.2023.05.003.

[2] Dylan Yaga et al. *Blockchain Technology Overview*. Tech. rep. NISTIR 8202. National Institute of Standards and Technology, 2018. DOI: 10.6028/NIST.IR.8202.

[3] Vitalik Buterin. *Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform*. https://ethereum.org/whitepaper/. Accessed: 2026-09-16. 2014.

[4] O. K. Rikken, M. F. W. H. A. Janssen, and Z. Roosenboom-Kwee. "The Ins and Outs of Decentralized Autonomous Organizations (DAOs): Unraveling the Definitions, Characteristics, and Emerging Developments of DAOs". In: *Blockchain: Research and Applications* 4.3 (2023), p. 100143. DOI: 10.1016/j.bcra.2023.100143.

[5] Lin William Cong et al. *Centralized Governance in Decentralized Organizations*. Working Paper. SSRN, 2025. DOI: 10.2139/ssrn.5168660.

[6] Robin Fritsch, Marino Müller, and Roger Wattenhofer. "Analyzing Voting Power in Decentralized Governance: Who Controls DAOs?" In: *Blockchain: Research and Applications* 5.3 (2024), p. 100208. DOI: 10.1016/j.bcra.2024.100208.

[7] Alexander Mühle et al. “A Survey on Essential Components of a Self-Sovereign Identity”. In: *Computer Science Review* 30 (2018), pp. 80–86. DOI: 10.1016/j.cosrev.2018.10.002.

[8] Carlo Mazzocca et al. “A Survey on Decentralized Identifiers and Verifiable Credentials”. In: *IEEE Communications Surveys & Tutorials* 27.6 (2025), pp. 3641–3671. DOI: 10.1109/COMST.2025.3543197.

[9] IP Capital Partners. *Web3*. Management Report. IP Capital Partners, Dec. 2021. URL: https://ip-capitalpartners.com/wp-content/uploads/2022/04/22-12_WEB3_IP_eng.pdf.

[10] Huabing Zhao. *Hash Pointers and Data Structures*. Zhaohuabing Blog. Accessed: 2026-09-16. May 2018. URL: https://zhaohuabing.github.io/post/2018-05-12-cryptocurrency_week1_hash_pointer_and_data_structures/.

[11] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. https://bitcoin.org/bitcoin.pdf. Accessed: 2026-09-16. 2008.

[12] Ethereum Foundation. *Proof-of-Stake (PoS)*. https://ethereum.org/developers/docs/consensus-mechanisms/pos/. Accessed: 2026-09-16.

[13] Ethereum Foundation. *Ethereum Accounts*. https://ethereum.org/developers/docs/accounts/. Accessed: 2026-09-16.

[14] Ethereum Foundation. *Introduction to Smart Contracts*. https://ethereum.org/developers/docs/smart-contracts/. Accessed: 2026-09-16.

[15] Ethereum Foundation. *Gas and Fees*. https://ethereum.org/developers/docs/gas/. Accessed: 2026-09-16.

[16] Ethereum Foundation. *Transactions*. https://ethereum.org/developers/docs/transactions/. Accessed: 2026-09-16.

[17] Pixis. *Ethereum*. Hackndo. Accessed: 2026-09-16. July 2023. URL: https://en.hackndo.com/ethereum/.

[18] Ethereum Foundation. *Merkle Patricia Trie*. https://ethereum.org/developers/docs/data-structures-and-encoding/patricia-merkle-trie/. Accessed: 2026-09-16.

[19] Ethereum Foundation. *Ethereum Virtual Machine (EVM)*. https://ethereum.org/developers/docs/evm/. Accessed: 2026-09-16.

[20] Solidity Team. *Solidity Documentation*. https://docs.soliditylang.org/. Accessed: 2026-09-16.

[21] Brave Software, Inc. *Brave Rewards*. https://brave.com/brave-rewards/. Accessed: 2026-09-16.

[22] Decentraland Foundation. *Decentraland 101*. https://docs.decentraland.org/faqs/decentraland-101. Accessed: 2026-09-16.

[23] Fabian Vogelsteller and Vitalik Buterin. *ERC-20: Token Standard*. Ethereum Improvement Proposals, EIP-20. Accessed: 2026-09-16. 2015. URL: https://eips.ethereum.org/EIPS/eip-20.

[24] Arvind Narayanan et al. *Bitcoin and Cryptocurrency Technologies: A Comprehensive Introduction*. See Section 1.1, Cryptographic Hash Functions. Princeton University Press, 2016. Chap. 1. ISBN: 9780691171692. URL: https://bitcoinbook.cs.princeton.edu/.

[25] Neil Daswani, Christoph Kern, and Anita Kesavan. *Foundations of Security: What Every Programmer Needs to Know*. Apress, 2007. DOI: 10.1007/978-1-4302-0377-3.

[26] Sanjay Goel. *Computer Security: Cryptography and Secure Communication*. Lecture slides, School of Business, University at Albany. Accessed: 2026-09-16. URL: https://www.slideserve.com/morgan/computer-security.

[27] Manu Sporny, Dave Longley, and David Chadwick. *Verifiable Credentials Data Model v1.1*. W3C Recommendation. Accessed: 2026-09-16. World Wide Web Consortium (W3C), Mar. 2022. URL: https://www.w3.org/TR/vc-data-model-1.1/.

[28] Manu Sporny et al. *Decentralized Identifiers (DIDs) v1.0: Core Architecture, Data Model, and Representations*. W3C Recommendation. Accessed: 2026-09-16. 2022. URL: https://www.w3.org/TR/did-core/.

[29] Compound. *Compound v2 Documentation: Governance*. Compound Documentation. Accessed: 2026-09-16. URL: https://docs.compound.finance/v2/governance/.

[30] Ivan Fan. "Startup Investing and Venture DAOs". In: *The University of Chicago Business Law Review Online* (May 2023). Accessed: 2026-09-16. URL: https://businesslawreview.uchicago.edu/online-archive/startup-investing-and-venture-daos.

[31] Uniswap Labs. *Governance Overview*. Uniswap Developers Documentation. Accessed: 2026-09-16. URL: https://developers.uniswap.org/docs/ecosystem/governance/overview.

[32] Nouns DAO. *Nouns DAO Documentation*. Nouns DAO Docs. Accessed: 2026-09-16. URL: https://docs.nouns.wtf/.

[33] Ethereum Name Service. *ENS DAO Governance Process*. ENS Documentation. Accessed: 2026-09-16. URL: https://docs.ens.domains/dao/governance/process/.

[34] Yixuan Fan et al. "Insight into Voting in DAOs: Conceptual Analysis and a Proposal for Evaluation Framework". In: *IEEE Network* 38.3 (2024), pp. 92–99. DOI: 10.1109/MNET.137.2200561.

[35] Yoshiro Saito and John A. Rose. "Reputation-Based Decentralized Autonomous Organization for the Non-Profit Sector: Leveraging Blockchain to Enhance Good Governance". In: *Frontiers in Blockchain* 5 (2023), p. 1083647. DOI: 10.3389/fbloc.2022.1083647.

[36] DeXe Protocol. *User Roles*. DeXe Protocol Whitepaper. Accessed: 2026-09-16. URL: https://whitepaper.dexe.network/dexe-protocol-overview/user-roles.

[37] Dario Castellano, Roberto De Prisco, and Pompeo Faruolo. "Toward a Compliant Token-Based e-Voting System with SSI-Granted Eligibility". In: *Proceedings of the 20th International Conference on Security and Cryptography (SECRYPT 2023)*. SciTePress, 2023, pp. 752–757. ISBN: 978-989-758-666-8. DOI: 10.5220/0012121100003555.

[38] Vid Keršič et al. "Using Self-Sovereign-Identity Principles to Prove Your Worth in Decentralized Autonomous Organizations". In: *Proceedings of the Ninth Workshop on Software Quality Analysis, Monitoring, Improvement, and Applications (SQAMIA 2022)*. Vol. 3237. CEUR Workshop Proceedings. CEUR-WS.org, 2022, 9:1–9:12. URL: https://ceur-ws.org/Vol-3237/paper-ker.pdf.

[39] OpenZeppelin. *OpenZeppelin Contracts*. https://docs.openzeppelin.com/contracts/5.x. Accessed: 2026-09-16.

[40] OpenZeppelin. *Governance*. https://docs.openzeppelin.com/contracts/5.x/api/governance. Accessed: 2026-09-16.

[41] OpenZeppelin. *ERC20Votes*. https://docs.openzeppelin.com/contracts/5.x/api/token/erc20. Accessed: 2026-09-16.

[42] Veramo. *Veramo Agent*. https://veramo.io/docs/basics/introduction. Accessed: 2026-09-16.

[43] Nomic Foundation. *Hardhat: Ethereum Development Environment*. https://hardhat.org/. Accessed: 2026-09-16.

[44] Mattia Prestifilippo Colombrino. *VCdao: Verifiable Credentials for DAO Governance*. https://github.com/mattiaprestifilippocolombrino/VCdao. Source code repository, accessed: 2026-09-16. 2026.

[45] Remco Bloemen, Leonid Logvinov, and Jacob Evans. *EIP-712: Typed Structured Data Hashing and Signing*. Ethereum Improvement Proposals, EIP-712. Accessed: 2026-09-16. 2017. URL: https://eips.ethereum.org/EIPS/eip-712.